QWEN

Helix Constitution — Universal QWEN.md

Field	Value
Revision	32
Created	2026-05-20
Last modified	2026-06-22T00:00:00Z
Status	active

Status summary	Added §11.4.170 — Device-independent host-side rendered-UI visual-proof mandate (User mandate 2026-06-25): every change to ANY user-facing UI surface MUST be proven by DEVICE-INDEPENDENT host-side RENDERED PIXELS before claimed correct — the real component rendered to a PNG ON THE HOST (no device/emulator/ running app; Compose→Roborazzi/Paparazzi, web→Playwright/Storybook, SwiftUI→snapshot-testing, equiv per stack) for EVERY screen×state×{light,dark} theme dual-validated by (i) golden image-diff AND (ii) an OCR/vision oracle reading rendered text+labels+control bounds (NO overlap / label-over-label / clipping / off-screen / collapsed-or-giant-unbounded widget). VALUE/token-equality / property-assertion UI tests are FORBIDDEN as the PROOF a UI is correct (forensic FACT 2026-06-25: hex/ sp/dp value-equality unit tests stayed GREEN while the operator opened a broken gian button screen) — may supplement, NEVER substitute the rendered-pixel proof; self-validated golden-good/golden-bad analyzer §11.4.107(10); device-offline is NEVER a valid skip (host-render IS the device-independent path); COMPLEMENTS not replaces §11.4.153/.158/.159/.160 live-device recording + §11.4.162 OpenDesign tokens + §11.4.168 exported-doc visual validation. Propagation gate CM-COVENANT-114-170-PROPAGATION + recommended gate CM-HOST-RENDERED-UI-VISUAL-PROOF + paired §1.1 mutation. Classification: universal (§11.4.17). Prior round: §11.4.166 REPEALED (operator decision 2026-06-22): Universal Semgrep static-analysis mandat repealed — Semgrep no longer mandatory; scaffolding, submodules/semgrep, MCP/pr commit/PATH wiring, docs_chain semgrep_status context, and CM-COVENANT-114-166-PROPAGATION/CM-SEMGREP-WIRED gates removed. Added §11.4.163 mirror — Universal Media Validation: every recorded artifact MUST pass MEDIA VALIDATION pipeline (OCR/transcription/text parsing vs patterns, self-validated analyzer, structured verdict, pinpoint on FAIL, real-time trigger, §1.1 mutation). Added §11.4.164 mirror — Universal Auto-Propagation Hook: post_update_hook.sh detects/ registers/installs skills/MCP/hooks/scripts; consumer invokes after every constitutional pull. Added §11.4.165 mirror — Universal Independent Verification Agent: every output passes INDEPENDENT verifier (§11.4.70/.20), zero-finding GO. All Classification: universal (§11.4.17). Prior round: Added §11.4.162 mirror — OpenDesign UI design system mandate (User mandate 2026-06-21): every project producing user-facing interfaces MUST use OpenDesign as the mandatory UI design-and-refinement system — NOT ad-hoc CSS or one-off design tools; install as a project dependency and use its design tokens/themes for color palette (light+dark), typography, spacing, and component-level tokens; extend upstream per §11.4.74 for missing patterns; every UI component ships light+dark variants with project brand colors from canonical assets; elements MUST NOT overlap or overlay labels; all UI changes covered by standard tes types including visual regression. Classification: universal (§11.4.17). Prior round: Adde §11.4.161 mirror — Rootless container runtime mandate (User mandate 2026-06-21):

every project MUST use Podman in rootless mode (or equivalent rootless container runtime) for ALL containerized workloads — Docker in rootful mode, sudo, or any escalation to root is FORBIDDEN unless the target platform has no rootless option AND that constraint is documented per §11.4.112; the `vasic-digital/containers` submodule (§11.4.76) MUST be used as the sole container orchestration layer — no ad hoc docker/podman commands outside `pkg/boot` / `pkg/compose` / `pkg/health`; missing capabilities extend the container Submodule upstream per §11.4.74; integration tests boot infra on-demand via the Submodule. Classification: universal (§11.4.17). Prior round: Added §11.4.160 mirror — Vision-verified recording + HelixQA bridge mandate (User mandate 2026-06-21): every video recording for feature/QA evidence MUST be processed through a vision/OCR pipeline that reads on-screen content and confirms expected results BEFORE acceptance; the recording system MUST provide a bridge feeding captured frames to HelixQA's test infrastructure (or equivalent) for content verification — automated read-the-screen against specified expected patterns; the bridge MUST capture frames at ≤5s intervals, run OCR/vision analysis with self-validated analyzer (§11.4.107(10)), compare against SPECIFY-phase patterns, produce per-frame PASS/FAIL with evidence path, surface failures immediately for re-record per §11.4.159(L). Honest boundary: vision verification confirms on-screen content — does NOT replace §11.4.5 quality analysis nor §11.4.108 runtime-signature; FLAG_SECURE surfaces use the §11.4.117 proxy oracle. Classification: universal (§11.4.17). Prior round: Added §11.4.153 mirror — Crashlytics-recorded-data continuous monitoring + systematic-debug + regression-test-coverage mandate (User mandate 2026-06-13): every project with Firebase Crashlytics enabled/wired MUST continuously monitor ALL four recorded surfaces (fatal crashes, ANRs, performance traces, AND non-fatals), systematic-debug each (reproduce-before-fix §11.4.102/.115), fix/improve, and cover every closure with a permanent §11.4.135 regression guard (RED-on-broken→GREEN-on-fixed) + a closure log citing the console issue id/URL + the validation/verification test paths; regular cadence (the §11.4.47 five-trigger set), no false results, no bluff — a console-resolved Crashlytics issue with no falsifiable regression test is FORBIDDEN (th silent-recurrence vector). STRENGTHENS+COMPLETES §11.4.47 (review/dedup finds it; §11.4.152 fixes it + proves it stays fixed). Project-level reference instantiation = a consuming project's §6.O + §6.AC. Composes §11.4/.1/.5/.6/.9/.34/.40/.43/.47/.69/.90/.102/.107/.108/.115/.118/.123/.130/.135/.138/.15 §1.1. Propagation gate `CM-COVENANT-114-152-PROPAGATION` + recommended gate `CM-CRASHLYTICS-ISSUE-FULLY-COVERED`. Classification: universal (§11.4.17). Prior round: Added §11.4.151 mirror — Project-prefixed release-tag/version-naming mandate (User mandate 2026-06-12): every release tag AND version name on the main repo AND every owned submodule MUST be prefixed `<PREFIX>-<version>` (e.g. `myproject-1.0.0-dev-0.0.1`); prefix resolution order (1)

`HELIX_RELEASE_PREFIX` from `.env` (authoritative, git-ignored §11.4.30, documented in tracked `.env.example` §11.4.77) else (2) lowercased snake_case project-root dir name §11.4.29; SAME prefix across main + all owned submodules in one release = greppable across every repo; version codes increment monotonically. Composes §2.1/§11.4.28/.29/.30/.40/.77/.113/.126. Propagation gate `CM-COVENANT-114-151-PROPAGATION` + recommended gate `CM-RELEASE-PREFIX-NAMING`. Classification: universal (§11.4.17). Prior round: Added §11.4.148 + §11.4.149 mirrors — workable-iten integrity + testing-diary family (User mandate 2026-06-10). §11.4.148 BINDS+STRENGTHENS §11.4.15/.16/.21/.54/.91/.93/.95/.106 into one DB↔docs↔external-tracker integrity contract: (D1) no item without a valid status+type+stable id on ALL three surfaces; (D2) comprehensive structured description (what/manifest/repro/acceptance); (D3) BLOCKED items carry WHY + enumerated unblock CHOICES (tightens §11.4.21; `Blocked` / `BLOCKED` = documented alias of canonical `Operator-blocked`); (D4) regular never-missed bidirectional DB↔docs↔tracker sync, §11.4.86-fingerprinted + §11.4.106 docs_chain-bound; (D5) generic idempotent external-tracker push (statuses/types/assignee-from-env/sub-tasks, sink-side proof §11.4.69, machinery project-agnostic §11.4.28). §11.4.149 per-workable item TESTING DIARY (date/time + tested-by + result + observations + action+why), distinct from item_history/Reopens: append-only `test_diary` table with PASS-requires-evidence schema constraint + in-depth diary + derived summary VIEW + four-format per-item exports §11.4.86-fingerprinted §11.4.106-bound + external-tracker SUB TASK `{TODO|In-progress|Completed}` lifecycle + minimal-LLM deterministic tooling 100% test-covered §11.4.27. Both Classification: universal (§11.4.17); project-specific dual per-display brightness/auto-dim/font-size landed ONLY in the consumer layer (RK3588/Presenter), NOT here. Propagation gates `CM-COVENANT-114-148-PROPAGATION` + `CM-COVENANT-114-149-PROPAGATION` + paired §1.1 mutations. Prior round: Added §11.4.147 mirror — Crashed-agent respawn-until-complete + no-work-los registry mandate (User mandate 2026-06-10): every dispatched agent/subagent MUST be tracked through its full lifecycle so a crash NEVER loses/forgets/corrupts its work (crash ≠ done, §11.4.6) — (a) durable append-only agent REGISTRY (status CLOSED SET `{dispatched|in-flight|crashed|respawned|complete}`) on the §11.4.116 sync substrate, SINGLE SOURCE OF TRUTH for “is this work owed?”; (b) mechanical CRASH-DETECTION (transient rate-limit/socket-close) → flip to `crashed` + keep OPEN → RESPAWN until `complete` autonomously per §11.4.101 with ALREADY-DEFINED backoff; (c) PARTIAL-STATE preserve → §11.4.84-check → resume-or-clear restart (§9.2 backup) in a per-agent worktree; (d) the §11.4.87/.94/.97/.126 loop done-condition MUST NOT read satisfied while any entry is non-`complete`. Forensic FACT this session: 5 subagents killed by rate-limit + 2 by socket-close + one PARTIAL dual-sliders edit, re-dispatched by pure conductor vigilance with NO mechanical guarantee;

agent-side analogue of §11.4.144. Composes §11.4.6/.58/.84/.87/.94/.97/.101/.116/.102/.108/.125/.142/.40/.128/.144/§9.2/§1.1. Classification: universal (§11.4.17). Prior round: Added §11.4.144 mirror — Tracked/recorded-device availability-following mandate (User mandate 2026-06-10): every devic the project is tracking / following / recording (test / debug / manual-testing, across every reachable transport — USB / wireless ADB / SSH / serial / network introspection API) MUST be availability-FOLLOWED — connection state continuously monitored, any drop handled, never silently abandoned and never presented as a continuous recording; the §11.4.128 always-on recorder KNOWS a tracked device is absent (its loop guards on th reachable state) but, lacking a following discipline, merely spins idle (no data, no offline event, no resume, no escalation) → silent corpus hole = §11.4 PASS-bluff at the recording-integrity layer. On a drop the system MUST automatically (a) DETECT + log an honest offline event (§11.4.6 fabricated-continuity bluff; §11.4.128 always-on gap), (b WAIT using the project's ALREADY-DEFINED reconnection timings — never invented (§11.4.6), the SAME grace/reconnect/poll budgets the recovery path already uses, (c) RE-ATTACH + log an honest online/resume event the moment the device returns, (d) ESCALATE to the sanctioned device-recovery path (per §11.4.69 feature class `device_recovery`) if the device does not return within the defined timeout — through the sanctioned, authorization-gated recovery entry point ONLY, never bypassing its gate never performing a destructive recovery (power-cycle) autonomously without that authorization (§11.4.21/.101 — high-blast-radius recovery is gated; while blocked keep following + log the blocked-escalation honestly). Composes §11.4.128/.69/.6/.14/.21/.101. Classification: universal (§11.4.17). Prior round: Added §11.4.143 mirror — Real-user-journey mandate for video-streaming-app full-automation tests (User mandate 2026-06-10, verbatim "All video streaming apps … require to choose some title and to press proper UI button to start or resume playing! Proper UI interaction to play exact show with proper content and subtitles is MANDATORY! Without it we just eventually play on 2nd display sample, and that's it mostly!"): any full-automation test asserting a video player / streaming app plays content MUST drive the REAL end-user journey through the app's OWN UI — launch → BROWSE the actual catalog → choose a SPECIFIC title → press the real Play/Resume button → confirm THAT chosen content plays with its correct subtitles on the intended routing target — NEVER a sample / demo / loop-clip / deep-link / `am start -a VIEW` / synthetic shortcut (those validate ROUTING only, not real chosen-content playback → a §11.4 PASS-bluff at the user-journey layer); non-introspectable streaming UIs use the §11.4.117 CV/OCR pixel oracle; login via the §11.4.10 credential single-source; chosen content confirmation via §11.4.107 liveness + §11.4.136 real-content + §11.4.137 subtitl correctness; honest `operator_attended` SKIP-with-reason per §11.4.52/.3 ONLY where autonomous is genuinely infeasible (hard login / geo-block / secure-surface

blanking) — NEVER a faked routing-only PASS. Composes §11.4.48/.107/.117/.136/.137/.52/.3/§107/§1.1. Classification: universal (§11.4.17). Prior round: Added §11.4.142 mirror — Universal code-review mandate — every change reviewed, always, no exception (User mandate 2026-06-09, verbatim “ALL changes we do MUST pass through the code review step!!! ALWAYS!!!”): EVERY change to ANY governed repository — no exception (source/fixes/tests/gates/mutations/docs/doc-tooling/build-CI/config/governance/conductor-edits/sub-agent-output/refactors/one-liner — MUST pass an INDEPENDENT code-review step BEFORE acceptance/commit/build the ABSOLUTE form of §11.4.125 (no “just a doc edit”/“just a one-liner”/“self-review §11.4.92 suffices”/“trivial change” carve-out); independence load-bearing (reviewer structurally separated from author per §11.4.70/.20, §11.4.92 self-review PRECEDES never satisfies); anti-bluff (§11.4/.1, conclusions captured-evidence §11.4.5/.69) + iterates to clean GO per §11.4.134; honest boundary §11.4.6 — does NOT replace §11.4.108/.40, it is the FIRST of MULTIPLE STRONG LAYERS. Composes §11.4.1/.4/.5/.6/.20/.40/.69/.70/.92/.108/.110/.125/.134/§107/§1.1. Classification: universal (§11.4.17). Prior round: Added §11.4.132 mirror — Risk-ordered validation priority mandate (User mandate 2026-06-07): tests/validations/verifications MUST run in RISK-DESCENDING order — highest-risk set FIRST (closed factors: (a) most-recently-worked, (b) historically most-problematic, (c) highest crash/break/regress likelihood, (d) most-reopened per §11.4.55), each GREEN with real (physical) captured evidence (§11.4.5/.69/.107, no bluff §11.4.6), and ONLY AFTER that set is GREEN does the rest of the suite run; arbitrary-order or lower-risk-before-highest-risk-GREEN = §11.4.132 violation; REFINES/STRENGTHENS §11.4.130 + §11.4.46 + §11.4.42 by adding explic risk-ordering to VALIDATION. Composes §11.4.4/.5/.6/.7/.40/.42/.46/.50/.55/.69/.107/.130. Classification: universal (§11.4.17). Prior round: Added §11.4.131 mirror — Standing session-resumption file mandate (Use mandate 2026-06-07): every project MUST maintain a SINGLE canonical always-curren session-resumption FILE at a fixed project-declared standard path (declared once per §11.4.35, never moved without a §11.4.66 operator decision) — the OUT-OF-THE-BOX entry point for any fresh session; promotes §11.4.127 (PREPARE-on-demand) into a STANDING version-controlled ARTIFACT, ALWAYS present + in sync, SHORT+FULL variants + live-state anchors + §11.4.65 export + §11.4.44 revision header. Composes §12.10/.127/.65/.44/.6/.66/.126. Classification: universal (§11.4.17). Prior round: Added §11.4.127 mirror — Session-handoff resumption-prompt mandate (User mandate 2026-06-06): when a fresh session is needed (context limits/degradation) OR the operator asks whether a new session is needed, the agent MUST ALWAYS prepare + proactively provide a ready-to-paste resumption prompt valid for that EXACT moment + project phase (SHORT first-sentence + FULL block on demand) pointing to the live handoff docs (.remember/remember.md if present + docs/CONTINUATION.md §12.10,

read FIRST + git fetch –all), stating PHASE + NEXT action + terminal goal, embedding exact live-state anchors (build IDs/MD5, device serials, HEAD, in-flight PIDs+logs, evidence paths) + binding constraints (anti-bluff §11.4, no-force-push §11.4.113, exact version/naming, hardware gotchas); MUST be moment-valid never generic; handoff docs current BEFORE the prompt; missing/stale/generic = §11.4.127 violation. Composes §12.10/.6/.66/.87/.103/.126. Classification: universal (§11.4.17). Prior round: Added §11.4.126 mirror — Default autonomous-loop working mode from first prompt (User mandate 2026-06-04): the endless fully-autonomous loop is the DEFAULT workin mode, engaged automatically the moment the operator sends the FIRST request/promp of a session — no per-session activation handshake; the loop continues until EITHER (A) a new fully-validated-and-verified version (tag) is created AND published across all owned submodules + main repo to all remotes (per §2.1/§11.4.40/§11.4.113), OR (B) fo non-release main-stream work that work is fully completed AND all side work streams done AND nothing is left in the working queue for the current scope; STOPS ONLY on explicit operator STOP, empty current-scope queue, or §12 host-safety; goal ABSOLUT EFFICIENCY; mimicking/imitation/false-results/bluff of ANY kind ABSOLUTELY FORBIDDEN (composes the §11.4 anti-bluff covenant — actually perform the work, never narrate/pretend it); §11.4.126 is the CAPSTONE promoting §11.4.87 from opt-in t always-on. Composes §11.4.87/.94/.97/.101/.103/.66/.6/.40/.42/.72/.113 + §2.1 + §12. Classification: universal (§11.4.17). Prior round: Added §11.4.125 mirror — Code-review agent gate before pre-build + main build (mandatory multi-layer review) (User mandate 2026-06-04): after all batch fixes/changes/implementations are done and BEFORE the pre-build test sweep + main (artifact) build, the agent MUST dispatch dedicated code-review agent(s) (subagent-driven per §11.4.70/.20) that analyze all batch work + all existing data/facts/captured-evidence + the existing codebase (blast radius) + current g history, determine quality + safety + will-it-REALLY-work, and validate+verify every covering test genuinely exercises the work-under-test and catches its negation with ZERO false-result/bluff possibility; any finding (defect / error-prone change / safety risk will-not-work / bluff-capable test / coverage gap) MUST be fixed+polished+improved+test-covered (four-layer §11.4.4(b), TDD-RED-first §11.4.43/.115) BEFORE the build proceeds; review iterates until no blocking findings; one of MULTIPLE STRONG LAYERS complementing (never replacing) the pre-build sweep + §11.4.92 multi-pass (author-side) + §11.4.108 + §11.4.110. Composes §11.4/.1/.4/.6/.40/.43/.50/.70/.20/.92/.102/.107/.108/.110. Classification: universal (§11.4.17). Prior round: Added §11.4.124 mirror — Dead/unwired-code investigate-before-remove mandate (User mandate 2026-06-04): “zero importers ⇒ dead ⇒ delete” is a guess (§11.4.6), never a finding — before removing ANY seemingly-dead element (zero-importer / never-called / unwired function/type/file/module/package/asset/config/ build-target) the agent MUST FIRST investigate via git history (`git log --follow` ,

`-S` / `-G` pickaxe, blame on the deleted call-site) and capture as FACT where/how it was wired in + when/how it became dead + whether “no references” is real or a hidden reference (reflection/DI/build-tags/codegen/plugin/FFI/config) the static tool cannot see; removal is permitted ONLY with captured PROOF it is genuinely unneeded AND MUST be its own separate descriptive commit citing the git-history evidence (+ §11.4.122 operator-confirmation for end-user capabilities; tracked via §11.4.90 Obsolete); with NO such proof the element MUST NOT be removed — instead wire it in properly (§11.4.114 and add any missing/unwired tests (§11.4.27/.43/.115); ALWAYS extra careful — when uncertain, default to NOT removing per §11.4.6/.101/.122. Classification: universal (§11.4.17). Prior round: Added §11.4.123 mirror — Rock-solid-proof-or-deep-research mandate (User mandate 2026-06-03): every reported issue / fix / claimed completion MUST be 100% validated with rock-solid CAPTURED proof (§11.4.5/.69/.107) before fixed/implemented/completed; metadata-only / config-only / absence-of-error / grep-without-runtime PASS forbidden (§11.4/.1); when UNSURE how to validate, the agent MUST ALWAYS first perform deep web research (§11.4.8+§11.4.99) to discover/build an evidence-producing method — declaring “untestable” or accepting a metadata-only PASS without exhausting that research is itself a §11.4.123 violation; forensic case study (FACT) 1.1.8-dev FLAG_SECURE + non-introspectable-UI validation unclear → deep research yielded the CV/OCR/liveness/sink-probe oracle stack (§11.4.107/.112/.117). Classification: universal (§11.4.17). Prior round: Added §11.4.122 mirror — No-silent-removal-of-existing-components-without-operator-confirmation mandate (User mandate 2026-06-03): no application/component/service/package/feature/driver/module/prebuilt — any already-existing end-user capability — may be removed from the existing codebase/System without FIRST interactively asking the operator (§11.4.66, never silent/autonomous) + an EXPLICIT keep-or-remove decision; silent removal is a release blocker; forensic case study F2 Apple-TV-class app + F4 Huawei HMS component removed without asking, operator reversed both; tracked DROP path ask→approve→ `Obsolete` reason `feature-removed` +citation (§11.4.90)→remove. Classification: universal (§11.4.17). Prior round: Added §11.4.114–§11.4.121 mirrors (eight new universal anchors from the 1.1.8-dev live-defect remediation, 2026-06-03): §11.4.114 last-known-good-tag regression isolation; §11.4.115 RED-baseline-on-the-broken-artifact + polarity-switch (refines §11.4.43); §11.4.116 real-time conductor↔autonomous-test-framework sync channel (append-only event stream + atomically-rewritten status snapshot, verdict events carry evidence paths); §11.4.117 CV/OCR pixel-oracle fallback for non-introspectable UIs (refines §11.4.48/.52/.107); §11.4.118 discovery-pressure to confirm known-issue-set completeness; §11.4.119 single-resource-owner partitioning for parallel hardware testing (refines §11.4.58/.103); §11.4.120 fix-breaks-its-own-gate reconciliation (rewrite the gate, never fake-pass / never revert the fix); §11.4.121 no-commit-while-build-writes-tracked-artifacts (build-

Field	Value
	output analogue of §11.4.84). All Classification: universal (§11.4.17). §11.4.108–§11.4.113 + §11.4.78–§11.4.107 mirrors continue from earlier Revisions.
Issues	none
Issues summary	—
Fixed	§11.4.108 mirror
Fixed summary	§11.4.107 lands in lockstep with the Constitution.md §11.4.107 addition.
Continuation	—

Table of contents

How inheritance works

This `QWEN.md` is for the **Qwen Code CLI agent** (`qwen-code`). It documents the universal Helix Constitution from the Qwen agent's perspective — the same content seen by Claude Code via `CLAUDE.md` and by OpenCode / Cursor / generic AI tooling via `AGENTS.md` .

The **authoritative source** is `Constitution.md` in this repository. When this file diverges from `Constitution.md` , the latter wins. This file exists because:

1. Qwen Code does not always honor `@import` directives across files; restating the critical rules inline ensures the agent reads them on every session.
2. Cross-agent parity: Claude Code (CLAUDE.md), OpenCode/Cursor (AGENTS.md), Qwen Code (QWEN.md) all start from the same constitutional baseline.

Discover this file via the canonical parent-walk pattern documented in every consuming project's `CLAUDE.md` / `AGENTS.md` :

```
find_constitution_root() {
    local cur="${1:-$PWD}"
    while [[ "${cur}" != "/" ]]; do
        if [[ -f "${cur}/constitution/Constitution.md" ]]; then
            echo "${cur}/constitution"; return 0
        fi
        cur="$(dirname "${cur}")"
    done
    return 1
}
```

Identity & posture

When the Qwen Code agent loads this file as part of session bootstrap, it operates under the Helix Constitution with the same identity, anti-bluff posture, and end-user quality covenant as any other CLI agent in the catalogue. There is no "Qwen-lite" interpretation — Qwen Code is held to the full §11.4 covenant.

Top-level invariants for every agent

1. **Read order on a cold start.** `CLAUDE.md` / `AGENTS.md` / `QWEN.md` (this file) for the AI-agent posture; `Constitution.md` for the canonical universal rules; then the consuming project's `CLAUDE.md` for project-specific extensions.
2. **Multi-mirror push.** Every commit on this submodule MUST land on all four canonical mirrors (GitHub + GitLab + GitFlic + GitVerse) per §2.1.
3. **Anti-bluff is non-negotiable.** Passing tests MUST imply working features. Bluffing PASSes (and FAILs that hide root cause) is a release blocker (§11.4 + §11.4.1).
4. **Submodule-catalogue-first.** Before scaffolding anything, survey the `vasic-digital` + `HelixDevelopment` orgs for an existing Submodule (§11.4.74). Reuse → extend → no-match in that order.
5. **Containers-submodule for ALL container workloads.** Use `vasic-digital/containers` (§11.4.76) — never reinvent compose/boot orchestration in-project.

Critical base rules restated (for agents that don't honour @imports)

§11.4 — Anti-bluff covenant — END-USER QUALITY GUARANTEE

Forensic anchor — verbatim user mandate (2026-04-28, reasserted 2026-05-21, 2026-05-29):

> "all existing tests and Challenges do work in anti-bluff manner - they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

Operative rule. Captured tests MUST exercise the behavior they claim to verify. PASS-without-execution paths, mock-only tests that don't round-trip, skip-by-default integration tests that mask real failures, metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and grep-without-runtime PASS are all critical defects regardless of how green the summary line looks. Each test failure in CI MUST imply a real broken feature; each PASS MUST imply the feature actually works for the end user. Tests and HelixQA Challenges are bound EQUALLY.

The verbatim covenant MUST be present in every consumer governance file (project Constitution.md, CLAUDE.md, AGENTS.md, QWEN.md) and in every owned submodule's equivalent. Tools that don't expand `@imports` still read the literal text.

§1.1 — Mutation-paired gates

Every captured-evidence gate MUST ship with a paired mutation test that mutates the gate's anchor + runs the gate + asserts the gate now FAILs. Absence of the paired mutation is itself a bluff.

§11.4.10 — Credentials-handling mandate

Credentials are NEVER committed to git, NEVER logged, NEVER printed to stdout/stderr. `.env` files are `.gitignore`d; committed siblings are `.env.example` placeholders only. Pre-store leak audit (§11.4.10.A) scans every PR for credential patterns before merge.

§11.4.17 — Universal-vs-project classification

Every new rule MUST declare itself **universal** (applies to every project consuming this Constitution) or **project** (applies only to one specific project). Misclassification (universal rule landing in a project's local CLAUDE.md instead of the constitution submodule) is a release blocker.

§11.4.20 — Subagent-driven-by-default

When a CLI agent (Qwen Code included) has a subagent / Task tool available, the default mode of operation is to dispatch subagents for discrete units of work rather than perform them inline. This protects context, enables parallel work, and matches the §11.4.27 no-fakes-beyond-unit-tests posture (each subagent runs real tools).

§11.4.73 — Main-specification document versioning + revision discipline

Projects with a main specification (`docs/specs/.../specification.V<N>.md` -shaped) maintain TWO version axes:

- **Primary** (V1 / V2 / V3 …) bumps for major rewrites only.
- **Secondary** (`Revision N` in the metadata table) bumps for additive changes within a primary version.

Spec edits trigger mandatory comprehensive planning and implementation in the consuming project — they are not isolated doc tweaks (the “spec ripple” rule).

§11.4.74 — Submodule-catalogue-first discovery + extend-don’t-reimplement

Direct user mandate (verbatim, 2026-05-20): “We MUST ALWAYS check which already developed features / functionalities do exist as a part of our comprehensive Submodules catalogue located in `vasic-digital` and `HelixDevelopment` organizations on GitHub and GitLab both! Project MUST BE aware of all its existence so we do not implement same things multiple times. For any missing features we MUST IMPLEMENT them properly and extend those Submodules further!”

Before scaffolding any new module / package / helper / utility, the Qwen Code agent MUST: (1) survey `vasic-digital` + `HelixDevelopment` on GitHub + GitLab — the canonical inventory is `submodules-catalogue.md` (142 repos categorised); (2) reuse an existing Submodule when it covers ≥ 80%; (3) extend in-place via upstream PR when 80%+ matches; (4) document the survey result via `Catalogue-Check: reuse|extend|no-match <org/repo>@<sha>` in the tracker row.

Every Submodule in the catalogue is subject to the same development-cycle rules (§11.4 anti-bluff, §1.1 paired mutations, §11.4.10 credentials, §11.4.61 metadata + ToC, §11.4.65 universal export, §11.4.73 spec versioning, §2.1 multi-mirror push, §3 propagation order).

§11.4.76 — Containers-submodule mandate

Direct user mandate (verbatim, 2026-05-20): “For any work or requirements of running services or codebase inside the Containers (Docker / Podman / Qemy / Emulators, and so on) we MUST USE / INCORPORATE the Containers Submodule properly: `https://github.com/vasic-digital/containers` (`git@github.com:vasic-digital/containers.git`). Containers Submodule contains all means for us to Containerize our code and services! If any feature or Containing System is missing or not supported we MUST EXTEND IT properly like we do all of our projects! No bluff work is allowed of any kind!”

For ANY containerized workload (Docker / Podman / Qemu / Kubernetes / container-backed emulators), the Qwen Code agent MUST: (1) install `vasic-digital/containers` (`digital.vasic.containers`) as a Git submodule when scaffolding the consuming project; (2) consume via `replace` directive during development + pinned commit SHAs in production; (3) boot infra on-demand via the Submodule’s `pkg/boot` + `pkg/compose` + `pkg/health` APIs so the operator is never required to start `podman machine` / `docker compose up` manually — the boot is part of the test entry point (**on-demand-infra invariant**); (4) extend the Submodule via upstream PR when a runtime / lifecycle primitive is missing — never reimplement in-project (per §11.4.74); (5) anti-bluff: integration tests claiming to exercise containerized components MUST actually boot them via the Submodule. Short-circuit fakes that bypass boot are a §11.4 violation. A passing test MUST imply the infra was up.

Tracker rows touching containerization MUST record `Catalogue-Check: extend vasic-digital/containers@<sha>` (or `reuse`); `no-match` requires demonstrating the Submodule cannot model the workload.

Planned anti-bluff gate `CM-CONTAINERS-USED` scans container-touching PRs for `digital.vasic.containers/...` imports. Paired mutation strips the import + asserts FAIL.

Canonical authority: `Constitution.md` §11.4.76.

Non-compliance: reinventing compose orchestration in-project is a release blocker.

§11.4.77 — Regeneration-mechanism-required mandate

Direct user mandate (verbatim, 2026-05-20): “We must be sure that after excluding anything from Git versioning we still have the mechanism which will out of the box obtain or re-generate missing content! Add this mandatory safety rule / constraint into ours root (constitution Submodule) Constitution.md, CLAUDE.md, AGENTS.md, QWEN.md or any other required document / file from the constitution Submodule!”

Forensic anchor. 2026-05-20T15:00Z: a consuming project’s audio Tier 1 `commit_all.sh` stalled 4 h on `git add -A` scanning 274 GiB `.git-backup-*` + 159 GiB `RKTools/linux/` + 167 GiB `qa-results/` — all untracked but un-gitignored. Bare `.gitignore` fix would orphan every fresh clone (missing RKTools, missing test infra, missing build outputs).

Every `.gitignore` entry the Qwen Code agent considers adding that excludes (a) >~100 MiB OR (b) any artefact essential to building / running / testing the project MUST carry a documented + automated mechanism to either **re-obtain** (vendor tarball, SDK installer, package registry, git submodule, object store) OR **re-generate** (build pipeline, code-gen, asset render, captured-evidence replay, container build).

Required artefacts per qualifying `.gitignore` entry: (1) `.gitignore-meta/<entry-slug>.yaml` declaring pattern + mechanism-type + script-path + expected-disk-usage + vendor-url-or-source + integrity hash + requires-network + requires-credentials; (2) post-clone bootstrap entry (`scripts/setup.sh` or canonical equivalent) running the mechanism non-interactively; (3) pre-build gate verifying regenerated content present OR stamp `.gitignore-meta/.regenerated/<slug>.ok` recent; (4) README + `docs/guides/*.md` describing the mechanism + manual fallback + time/disk budget + per-§11.4.10 credentials.

No escape hatch: bare `.gitignore` additions without the mechanism are §11.4 PASS-bluff variants — codebase appears complete but fresh clone cannot build / run. No `--skip-regen-mechanism` , `--gitignore-is-enough` , `--operator-already-has-content` flag.

Planned anti-bluff gate `CM-GITIGNORE-REGEN-MECHANISM` scans every `.gitignore` addition for matching `.gitignore-meta/` sibling. Paired §1.1 mutation strips a required YAML key → gate FAILs.

Composes with §11.4.6, §11.4.65, §11.4.66, §11.4.71, §11.4.74, §11.4.75, §11.4.76, §9 / §9.2, §3.

Canonical authority: `Constitution.md` §11.4.77.

Non-compliance is a release blocker regardless of context.

§11.4.78 — CodeGraph code-intelligence mandate

Direct user mandate (verbatim, 2026-05-20): “Make codegraph MANDATORY CHOICE for this purpose for all of our project … All project which do not have configured and installed codegraph yet MUST DO IT and MUST USE IT!”

Every consuming project worked on by AI coding agents — Qwen Code included — MUST install, initialize, and use **CodeGraph** (`https://github.com/colbymchenry/codegraph` , npm `@colbymchenry/codegraph`): a local SQLite semantic code-knowledge-graph exposed to agents over MCP, 100% local. Install globally via npm (no `sudo`). `codegraph init` + `codegraph index` : `.codegraph/config.json` is tracked, `.codegraph/codegraph.db` is gitignored with `codegraph index` as its §11.4.77 regeneration mechanism; the `exclude` list MUST exclude every §11.4.10 credential/secret path. **CRITICAL — CodeGraph filter mechanism:** CodeGraph v1.x is zero-config — it uses built-in skip lists + root `.gitignore` for file filtering, NOT `config.json` include/exclude. Use root-anchored `.gitignore` patterns to exclude only RUBBISH (build outputs, caches, credentials). **ALL SOURCE CODE MUST BE INDEXABLE** — AOSP platform trees, third-party vendored trees, and all tracked source code are part of the index. See `Constitution.md` §11.4.78 for the complete mechanism description. Wire the `codegraph serve --mcp` server into every CLI agent the developers use — for Qwen Code via `.qwen/settings.json` (`qwen mcp add codegraph codegraph serve --mcp --scope project --transport stdio`). Cover the integration with an anti-bluff verification suite using an unforgeable per-agent challenge (a fact obtainable only by calling a CodeGraph MCP tool); un-

runnable agents are documented SKIP gaps, never faked PASSes. Document in `docs/CODEGRAPH.md`. CodeGraph is consumed as the npm package (§11.4.74), not a git submodule.

Composes with §11.4.3, §11.4.10, §11.4.12, §11.4.65, §11.4.30, §11.4.74, §11.4.77, §11.4, §1.1.

Canonical authority: `Constitution.md` §11.4.78.

Non-compliance is a process violation; a project worked on by AI agents without CodeGraph installed, wired, and anti-bluff-verified is in breach.

§11.4.79 — Own-org submodules MUST be included in the CodeGraph index

Direct user mandate (verbatim, 2026-05-21): “All Submodules we use in the project and that are part of organizations to which we have the full access via GitHub, GitLab and other CLIs MUST BE included into the codegraph database and initialized / scanned / synced!”

Refines §11.4.78 step 2 with a per-submodule-ownership split. **Own-org submodules** — full write access via the project’s CLIs (canonical orgs: `vasic-digital` + `HelixDevelopment`) — MUST be INCLUDED in the index so Qwen Code (and every other AI agent) sees a unified call-graph across the project’s domain + own-org infra code. **Third-party submodules** (the §11.4.74 `no-match → vendor` path) MUST be EXCLUDED. Operational steps: (1) `git submodule update --remote --merge` to pull latest before re-indexing — respect load-bearing third-party pins. (2) Adjust `.codegraph/config.json` exclude list. (3) Re-index via `scripts/codegraph_setup.sh`. (4) Validate via `scripts/codegraph_validate.sh` probe resolving a symbol that lives ONLY inside an own-org submodule. (5) Paired §1.1 mutation: temporarily add own-org path to exclude → validate FAILs → restore.

Composes with §11.4.74, §11.4.78, §11.4.10, §1.1, §107.

Canonical authority: `Constitution.md` §11.4.79.

Non-compliance is a process violation; severe cases (own-org submodules silently excluded without an audit trail in `.codegraph/config.json`) are release blockers.

§11.4.80 — CodeGraph regular-update + sync automation mandate

Direct user mandate (verbatim, 2026-05-21): “We MUST regularly check for the updates and execute codegraph npm updates so the latest version of it is always installed […] Make sure we have proper full automation bash scripts which will run regularly and that these are part of the constitution Submodule […] Make sure all updates, sync processes we do and important codegraph related events are all documented under docs/codegraph in Status and Status_Summary documents […]”

Three deliverables in this constitution submodule: (1) `scripts/codegraph_update.sh` — npm-installs latest; §107 anti-bluff verifies `codegraph --version` reflects the new version. (2) `scripts/codegraph_sync.sh` — runs `status` → `sync` → `status` → project’s validate; appends to both project’s + constitution’s `docs/codegraph/Status.md`. (3) `docs/codegraph/Status.md` + `Status_Summary.md` ledgers, exported per §11.4.65 to `.html` + `.pdf`.

Cadence: weekly floor (§11.4.45). Scripts inherited by reference (§3) — Qwen Code consuming projects invoke them at `${CONST_DIR}/scripts/codegraph_*.sh`, never copy.

Composes with §11.4.78, §11.4.79, §11.4.10, §11.4.45, §11.4.65, §107, §1.1.

Canonical authority: `Constitution.md` §11.4.80.

Non-compliance is a process violation; severe cases (>2 weeks since last update + open AI-agent work) are release blockers.

§11.4.81 — Cross-platform-parity mandate (User mandate, 2026-05-21)

Direct user mandate (verbatim, 2026-05-21): “Any Linux-only blocker / issue we have MUST BE created macOS and other supported platforms equivalent! So, depending on platform proper implementation will be used for particular OS! EVERYTHING MUST BE PROPERLY EXTENDED AND UPDATED!”

Every consuming project whose supported-platforms manifest lists more than one OS MUST ship per-OS-equivalent implementations + tests for every feature/gate/challenge/mutation that depends on platform-specific primitives. Runtime dispatch

via `uname -s` (or equivalent platform detection). Three sub-mandates: **(A)** Per-OS implementation REQUIRED — cgroup/systemd/ `/proc` primitives have documented equivalents (POSIX `setrlimit` / `ulimit`, launchd, `rctl`, Job Object). **(B)** Per-OS tests REQUIRED — every gate test has `case "$(uname -s)" in` branches with positive captured evidence per §11.4.2 + §11.4.5 in each branch; SKIP-with-reason only when the platform genuinely cannot enforce. **(C)** Honest kernel-gap citation + adjacent equivalent test REQUIRED where no equivalent exists (canonical: XNU does NOT enforce `RLIMIT_AS` for unprivileged processes → SKIP with exact reproducer + adjacent test of what IS enforced, e.g. `RLIMIT_CPU` + `SIGXCPU`). The adjacent test is itself anti-bluff per §11.4 with a paired §1.1 mutation.

Per-OS equivalence catalogue (canonical): `systemd-run --user --scope` ↔ POSIX `ulimit -t -u` / launchd; cgroup `MemoryMax` ↔ XNU gap (use `RLIMIT_CPU` adjacent) / `rctl` / Job Object; cgroup `TasksMax` ↔ `RLIMIT_NPROC`; `/proc/<pid>/oom_score_adj` ↔ no Darwin/BSD equivalent.

Composes with §11.4.1 / §11.4.2 / §11.4.3 (strictened — SKIP only when kernel cannot) / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.20 / §11.4.27 / §11.4.69 / §11.4.70 / §107. Pre-build gate `CM-CROSS-PLATFORM-PARITY` + paired §1.1 mutation. No escape hatch.

Canonical authority: `Constitution.md` §11.4.81.

Non-compliance is a release blocker on multi-platform projects.

§11.4.82 — Iteration-speedup discipline mandate (User mandate, 2026-05-22)

Direct user mandate (verbatim, 2026-05-22): “How can we speed-up this whole development and fixing process? … all speed optimizations critical rules and mandatory constraints MUST BE all added into our root (constitution Submodule) Constitution.md, CLAUDE.md, AGENTS.md and QWEN.md!”

Iteration cycle time bounds the project’s defect-discovery rate. Slow cycles ship fewer validated features per unit of operator time. The 2026-05-22 forensic session witnessed ~3 hours of operator wait on a job whose intrinsic compute was ~90 min — every wasted minute came from a missing speedup discipline.

Every consuming project's build / test / commit / debug pipeline MUST adopt the following 9 speedup disciplines AS MANDATORY:

- **(A) Phase 1 forensic before any speculative source patch.** Speculative patches without root-cause evidence are §11.4.6 + §11.4.82 violations.
- **(B) Live-ADB-First (or live-equivalent) before any rebuild.** §11.4.51 strengthened to release-blocker. Skipping live-probe for LIVE_ADB_TESTABLE changes = §11.4.82 violation.
- **(C) Pre-flight before rebuild orchestrators.** 30-sec readiness check verifies device + sink reachability, memory budget, lock files, orphan processes.
- **(D) Persistent build caches outside containers.** `ccache` + Soong / sccache / Gradle daemon state bind-mounted to host. Subsequent rebuilds drop from ~40 min to 5-15 min.
- **(E) Module-only rebuild for `CONFIG_*=m` driver patches.** `make modules` on single driver dir saves 15-20 min/cycle when applicable.
- **(F) Parallel multi-device testing.** Every owned validation device runs the autonomous cycle concurrently with separate output dirs. Catches per-device defects one cycle earlier.
- **(G) Subagent scope ≤30 min + worktree isolation + single-responsibility.** Empirical pattern: 8-task subagents stall, 2-3-task subagents complete.
- **(H) Lock-file + stale-process hygiene.** Clean `.git/index.lock` + orphan processes on session start. Disable auto git-`gc` in concurrent multi-agent repos.
- **(I) Cycle telemetry per §11.4.24.** Every cycle logs commit, per-phase wall-clock, speedup flag set, outcome. Weekly aggregation surfaces empirical ROI per discipline.

Composes with §11.4.4 / §11.4.6 / §11.4.9 / §11.4.20 / §11.4.24 / §11.4.42 / §11.4.43 / §11.4.50 / §11.4.51 / §11.4.52 / §11.4.58 / §11.4.70 / §12.7 / §107. Pre-build gate `CM-ITERATION-SPEEDUP-DISCIPLINE` (when implemented) + paired §1.1 mutation. No escape hatch — no `--skip-phase1`, `--no-pre-flight`, `--rebuild-everything`, `--unlimited-subagent-scope`, `--ignore-locks`, `--no-telemetry` flag exists.

Canonical authority: `Constitution.md` §11.4.82.

Non-compliance is a release blocker.

§11.4.83 — docs/qa/ end-user evidence mandate (User mandate, 2026-05-22)

Direct user mandate (verbatim, 2026-05-22): “every feature that ships MUST carry a recorded e2e communication transcript + any attached materials under `docs/qa/<run-id>/` (per-feature subdirectories). A feature with no QA transcript is itself a §107 PASS-bluff — it claims to work but has no auditable runtime evidence. Bot-driven automation MUST preserve full bidirectional communication threads as proof.”

Every feature that ships MUST carry a recorded end-to-end communication transcript plus attached materials committed under `docs/qa/<run-id>/`. Transcripts MUST be full bidirectional. Attached materials live in-repo (no external-only links — §11.4.13 sink-side violation). Bot-driven / agent-driven QA MUST preserve the full conversation thread as the proof artefact. CI release gates MUST refuse to tag a version whose feature-shipping commit lacks its `docs/qa/<run-id>/`.

Composes with §11.4.2, §11.4.5, §11.4.13, §11.4.65, §11.4.69, §107, §1.1.

Canonical authority: `Constitution.md` §11.4.83.

Non-compliance is a release blocker.

§11.4.84 — Working-tree quiescence rule for subagent commits (User mandate, 2026-05-22)

Short tag: `working-tree quiescence`.

Direct user mandate (verbatim, 2026-05-22): “no subagent commit may proceed while any concurrent mutation gate is in flight in the same checkout. Before `git add`, the committing agent MUST `grep` its own working tree for mutation markers (`MUTATED for paired`, `// always pass`, `return json.Marshal` shortcut paths, etc.). Any unexplained file in the staging area triggers ABORT.”

No subagent commit may proceed while any concurrent mutation gate is in flight in the same checkout. Pre-`git add`: grep for mutation markers (`MUTATED for paired`, `// always pass`, `return json.Marshal` shortcuts, `// MUTATION` annotations, `_mutated_*` filenames). Cross-check `git status --porcelain` against declared scope → unaccounted entries ABORT.

Lesson (forensic). A consuming project's logo-fix subagent (Herald commit `72e81ab`, 2026-05-21) swept a `// always pass` JWT-bypass mutation residue into an unrelated commit, pushed to all four mirrors before being caught. Fix `d5bd360` landed within the hour, but the security-defect window is real and the lesson is constitutional.

Operative rule. (1) Pre-`git add` grep + scope-match → unaccounted ABORT. (2) Active mutation gates MUST be serialised before unrelated commits. (3) Concurrent subagents in same checkout MUST use lockfile (`.git/MUTATION_IN_PROGRESS`) or `git worktree add`. (4) Pre-push `mutation-residue-scanner` BLOCKS pushes containing mutation markers.

Composes with §1.1, §11.4.20, §11.4.70, §11.4.27, §11.4.10, §11.4.71, §107.

Canonical authority: `Constitution.md` §11.4.84.

Non-compliance is a release blocker.

§11.4.85 — Stress + Chaos Test Mandate (User mandate, 2026-05-24)

Short tag: `stress-chaos-mandate`.

Direct user mandate (verbatim, 2026-05-24): "Every fix or improvement you do MUST BE covered with full automation stress and chaos tests so we are sure nothing can break the functionality and all edge cases are monitored and polished and additionally fixed if that is needed! Everything must produce rock solid proofs and follow fully no-bluff policy!"

Every fix MUST ship with full-automation stress + chaos test suites. Happy-path-only coverage is a §11.4 / §107 PASS-bluff at the resilience layer.

Stress = sustained load (N ≥ 100 iters OR ≥ 30 s) + concurrent contention (N ≥ 10 parallel) + boundary conditions (empty/max/off-by-one). **Chaos** = failure-injection per §11.4.69 closed-set (process-kill, network-drop, input-corruption, OOM, disk-full, state-corruption) + verifiable recovery.

Anti-bluff: every stress + chaos PASS cites a captured-evidence artefact (latency.json / categorised_errors.txt / state_delta_snapshot.json) per §11.4.5 + §11.4.69. Helper library `stress_chaos.sh` provides `ab_stress_run` / `ab_stress_concurrent` / `ab_chaos_*` primitives. Chaos cleanup non-negotiable — corrupt-restore / disk-fill-cleanup / process-restart MUST run in trap EXIT.

4-layer per §11.4.4(b): pre-build gate (test files exist + parse) + paired meta-test mutation + on-device test (if LIVE_ADB_TESTABLE) + HelixQA Challenge entry. Composes with §11.4 / §11.4.1 / §11.4.5 / §11.4.6 / §11.4.43 / §11.4.50 / §11.4.52 / §11.4.69 / §11.4.83.

Canonical authority: `Constitution.md` §11.4.85.

Non-compliance is a release blocker. No `--skip-stress`, `--no-chaos`, `--happy-path-suffices` flag exists.

§11.4.86 — Roster/corpus-backed Status-doc auto-sync mandate (User mandate, 2026-05-25)

Forensic anchor (verbatim): “Make sure that assets and players Status docs are ALWAYS regularly updated and in sync like all others Status docs — any time we add or modify the assets content(s) or we change or add new / remove existing pre-installed video and audio player apps! This MUST WORK OUT OF THE BOX!”

Status docs (§11.4.45) backed by a **tracked roster** (installed apps/components) or **asset corpus** (test/media directory) MUST resync out of the box the moment a member changes — Status doc + Status_Summary + HTML + PDF, mechanically. Mechanism (all must hold): (1) drift-proof **fingerprint** = sha256 of sorted member list (NOT mtime), persisted in a sidecar; (2) **sync helper** regenerates fingerprint + re-exports (via §11.4.65); (3) **pre-build gate** FAILs on fingerprint drift (mirrors §11.4.12 + §11.4.45); (4) paired §1.1 mutation. Composes with §11.4.12 / §11.4.45 / §11.4.53 / §11.4.56 / §11.4.57 / §11.4.59 / §11.4.60 / §11.4.65 / §11.4.6. Universal (§11.4.17) — consuming project supplies the specific docs/roster/helper/gate per §11.4.35.

Canonical authority: `Constitution.md` §11.4.86.

Non-compliance is a release blocker. No `--skip-roster-sync`, `--allow-status-drift` flag exists.

§11.4.87 — Endless-loop autonomous work + zero-idle agent dispatch + anti-bluff testing mandate (User mandate, 2026-05-26)

When operator instructs an AI agent to “continue in endless loop fully autonomously” (or semantically-equivalent), the agent MUST treat as HARD-CONTRACT covenant: (A) continue until `docs/Issues.md` non-terminal Status entries = 0 AND `docs/CONTINUATION.md` §3 Active work empty AND no subagent in-flight AND no external dep in-flight; (B) dispatch background subagents for parallelisable work — main + subagents concurrent, “waiting for results” is the ONLY idle reason; (C) every closure lands four-layer test coverage per §11.4.4(b) with captured-evidence “physical proofs” (audio/video/network/UI/sysfs) — metadata-only / config-only / absence-of-error / grep-without-runtime PASS are critical defects; (D) §11.4 anti-bluff covenant family operative end-to-end (tests AND HelixQA Challenges bound equally per forensic anchor “tests pass but features don’t work”); (E) loop terminates ONLY on all-conditions-met, explicit operator STOP, host-safety demand, or scheduled wake on known-future-actionable signal.

Composes with §11.4 / §11.4.1 / §11.4.2 / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.7 / §11.4.20 / §11.4.27 / §11.4.42 / §11.4.43 / §11.4.50 / §11.4.52 / §11.4.58 / §11.4.68 / §11.4.69 / §11.4.70 / §11.4.83 / §11.4.85 / §11.4.86 / §12.10. Pre-build gate `CM-COVENANT-114-87-PROPAGATION` + paired §1.1 mutation.

Canonical authority: `Constitution.md` §11.4.87.

Non-compliance is a release blocker. No `--idle-OK`, `--skip-endless-loop`, `--bluff-permitted-for-this-task`, `--metadata-only-test-suffices`, `--no-physical-proof-required` flag exists.

Companion documents

- `Constitution.md` — authoritative universal Constitution. ALWAYS the tie-breaker.
- `CLAUDE.md` — Claude Code agent’s view of the universal constitution.
- `AGENTS.md` — OpenCode / Cursor / generic-tooling view.
- `README.md` — high-level overview + multi-mirror contract.

When operating in a consuming project, ALSO read the project's local `QWEN.md` / `CLAUDE.md` / `AGENTS.md` for project-specific extensions; these MUST NOT weaken any universal rule but MAY add stricter project-specific constraints.

§11.4.88 — Background-push mandate: commit-lock release immediately after commit, push runs detached (User mandate, 2026-05-26)

Forensic anchor: 2026-05-26 commit_all.sh held flock ~5h on synchronous post-commit push. Mandate: (A) flock release IMMEDIATELY after commit; (B) push detached via nohup + disown; (C) push_all.sh per-remote flock — same-remote serializes, different-remote parallel; (D) failures → `qa-results/push_failures/`, autonomous loop checks per §11.4.87(A); (E) `--sync-push` escape for §11.4.41 force-push only. Composes §2.1 / §9.2 / §11.4.41 / §11.4.42 / §11.4.71 / §11.4.87. Gates `CM-COVENANT-114-88-PROPAGATION` + `CM-BACKGROUND-PUSH-WIRED` + paired §1.1 mutations.

Canonical authority: `Constitution.md` §11.4.88. Non-compliance is a release blocker.

§11.4.89 — Background test execution mandate (User mandate, 2026-05-27)

Forensic anchor 2026-05-27: conductor invoked long pre_build synchronously, blocking main stream 6-7 min on §JV/§JW/§JX/§JY scaffolding. Symmetric to §11.4.88 at test-execution layer. Mandate: (A) long tests (>30s) run via `nohup ... > <log> 2>&1 &` + `disown`; (B) main stream proceeds to §11.4.42 priority queue immediately; (C) hard-dependency gating via poll/pgrep — surface §11.4.66 options if still running; (D) failures land in log files; (E) foreground only <30s OR operator authorisation; (F) per-script flock. Composes §11.4.42 / §11.4.66 / §11.4.82 / §11.4.84 / §11.4.85 / §11.4.87 / §11.4.88. Gates `CM-COVENANT-114-89-PROPAGATION` + `CM-BACKGROUND-TEST-EXECUTION-WIRED` + paired §1.1 mutations.

Canonical authority: `Constitution.md` §11.4.89. Non-compliance is a release blocker.

§11.4.90 — Obsolete status + obsolescence audit (User mandate, 2026-05-27)

§11.4.15 Status closed-set + terminal `Obsolete (→ Fixed.md)`. Reasons: `superseded-by-design-change | superseded-by-later-mandate | feature-removed | duplicate-of | unsupported-topology | not-reproducible` (`not-reproducible` = a reported defect that does NOT reproduce on the canonical tree/baseline — an environment/isolated-worktree artifact, not a real product defect). `**Obsolete-Details:**` line mandatory (Since + Reason + Superseding-item + Triple-check evidence). Colorizer `cell-status-obsolete` = light-gray + strikethrough. Audit cadence per §11.4.40 + §11.4.42. Triple-check non-negotiable.

Canonical authority: `Constitution.md` §11.4.90. Release blocker.

§11.4.91 — Summary-doc clarity (User mandate, 2026-05-27)

Every summary entry’s description: self-contained, meaningful, ≥ 6 words / ≥ 40 chars, SUBJECT + PROBLEM/GOAL. Anti-patterns forbidden: section labels, bare metadata, §-letter alone. Generators extract from H1/H2 heading. Pre-build gate `CM-SUMMARY-CLARITY-DESCRIPTIONS`.

Canonical authority: `Constitution.md` §11.4.91. Release blocker.

§11.4.92 — Multi-pass change-evaluation (User mandate, 2026-05-27)

5-pass evaluation BEFORE commit-ready: (1) Main-task verification; (2) Regression-blast-radius; (3) Cross-feature interaction; (4) Deep-research per §11.4.8; (5) Anti-bluff confirmation. Doc per pass. Trivial exemption: typo/revision/ MD-export ONLY if zero source touched.

Canonical authority: `Constitution.md` §11.4.92. Release blocker.

§11.4.93 — SQLite-SSoT for workable items (User mandate, 2026-05-27)

`docs/.workable_items.db` SQLite — DB authoritative, MD derived. Schema: items + item_history + obsolete_details + operator_block_details + firebase_metadata + meta. Go binary `cmd/workable-items/` sync md↔db + diff + validate + add + close. Bidirectional regen byte-identical. Anti-bluff: unit+integration+stress+chaos per §11.4.85. Go binary in constitution submodule per §11.4.74. 6-phase migration §LA.

Canonical authority: `Constitution.md` §11.4.93. Release blocker.

§11.4.94 — Zero-idle priority-first parallel-by-default (User mandate, 2026-05-27)

Binds §11.4.20/42/58/70/72/82/87/88/89 — always-on. Idle ONLY: externally blocked, operator STOP, §12 host-safety. Before any wake/sleep survey parallel-work queue. Priority MANDATORY (§11.4.72 audio first). Subagent-driven default non-trivial. Background default >30s. Stability per §11.4.92 + §11.4.84 + §12.6-9. Updates at milestones.

Canonical authority: `Constitution.md` §11.4.94. Release blocker.

§11.4.95 — Workable-items DB TRACKED in git (User mandate, 2026-05-27)

§11.4.93 amendment: `docs/workable_items.db` TRACKED, NEVER gitignored. Authoritative source. Every mutation → stage+commit+push DB + MD. WAL-checkpoint before commit. §11.4.77 does NOT apply. Pre-build gates `CM-COVENANT-114-95-PROPAGATION` + `CM-WORKABLE-ITEMS-DB-TRACKED`.

Canonical authority: `Constitution.md` §11.4.95. Release blocker.

§11.4.96 — Safe-parallel-work-with-long-build (User mandate, 2026-05-27)

SAFE during AOSP build: (A) docs/MD; (B) scripts/; (C) pre-build gates; (D) on-device tests; (E) constitution edits+push; (F) submodule push per §11.4.88; (G) read-only ADB probes; (H) subagent dispatch; (I) web research; (J) workable-items DB ops; (K) pre-build + meta-test execution backgrounded.

UNSAFE: (α) git checkout/reset on source; (β) mass deletes/renames under built source; (γ) APK-built submodule pointer; (δ) out/; (ε) make clean; (ζ) container destruction; (η) disk-fill; (θ) §12 host-safety.

Conductor dispatches ≥1 (A)-(K) per pause. “Build running, nothing else to do” NEVER true.

Canonical authority: `Constitution.md` §11.4.96. Release blocker.

§11.4.97 — Max-use-of-idle + progress-update cadence (User mandate, 2026-05-27)

1. Every minute progressable idle = §11.4.97 violation, dispatch CONTINUOUSLY; (B) 1-line operator updates at commit/subagent/anchor/evidence/migration — no prompt; (C) continuous physical-proof gathering per §11.4.5/6/69; (D) composes with operating-mode anchor family; (E) idle-only-when-blocked closed-set unchanged from §11.4.94(A). Pre-build gates `CM-COVENANT-114-97-PROPAGATION` + `CM-IDLE-TIME-AUDIT`.

Canonical authority: `Constitution.md` §11.4.97. Release blocker.

§11.4.98 — Full-Automation Anti-Bluff (User mandate, 2026-05-28)

Forensic anchor: “Make sure we have full automation testing of all scenarios with real bot, main group and users without any manual intervention or contribution of real user! … No bluff is allowed!”

Closes the manual-intervention gap §11.4+85+87+89+94 didn’t forbid. (A) every test MUST self-drive end-to-end — PASS/FAIL/SKIP-with-reason without further human action; (B) single exception = one-time credential bootstrap OUTSIDE test execution (.env, ~/.bashrc, OAuth, MTProto-session-activation); (C) concrete live

requirements: (1) no “operator MUST type” prompts — drive via MTProto/real-user-API/webhook-fixture/in-process loopback; (2) no UUIDs colliding with active dev session (Herald 2026-05-28: silent exit -1 lesson); (3) no 60s human-response windows (§11.4.50 violation); (4) re-runnability `-count=3` with self-cleaning state; (5) audit COMPLIANT vs NON-COMPLIANT; (6) no false-positive PASS — silent-skip-as-PASS + stale-evidence forbidden, §11.4.3 SKIP-with-reason correct; (D) composes §11.4.85+89+87+94 = continuously-validated fully-automated non-flake anti-bluff regime; (E) inheritance per §11.4.35 — restate literal `11.4.98` in every consumer; pre-build gate `CM-COVENANT-114-98-PROPAGATION`; paired §1.1 mutations strip → FAIL; (F) enforcement: manual-action commit BLOCKED; NON-COMPLIANT after 30 days → §11.4.90 Obsolete citing §11.4.98.

Canonical authority: `Constitution.md` §11.4.98. Release blocker.

§11.4.99 — Latest-Source Documentation Cross-Reference (User mandate, 2026-05-28)

Forensic: “ALWAYS check against latest versions of services we use web/online docs before creating instructions! … mandatory rules / constraints, result is consistency and safety of created instructions, guides and manuals!”

Case study (Herald 2026-05-28): MTProto guide recommended VoIP + omitted `recover@telegram.org` pre-login email; contradicted official Telegram + gotd/td docs; could have caused permanent ban. Misguidance-by-stale-docs = §11.4 PASS-bluff at documentation layer.

1. Pre-commit cross-reference: fetch LATEST official online docs (WebFetch/MCP/direct browsing — NEVER training data) for every operator-facing instruction doc; cite source URL+date in “## Sources verified” footer AND commit-message footer; seek secondary authoritative sources for sparse official docs. (B) Negative findings documented explicitly. (C) STALE after 6 months (90 days for risk-classified services); re-verify at vN.0.0, on breaking-change, on operator-error. (D) Risk-classified: messengers/cloud/payment/AI/code-hosting/package-managers. (E) Composes with §11.4.92 Pass 4 INDEPENDENT — cannot substitute. (F) Inheritance per §11.4.35 — restate literal `11.4.99` in every consumer’s CLAUDE/AGENTS/QWEN. (G) Enforcement: missing-footer BLOCKED; stale-beyond-grace → §11.4.90 Obsolete.

Canonical authority: `Constitution.md` §11.4.99. Release blocker.

§11.4.100 — RETIRED. Demoted to consumer project (a consuming project's video-color/visual-quality fidelity) per §11.4.17/§11.4.35 — project-specific (RK3588/MPV/Arvus), not universal. See the consuming project's Constitution/CLAUDE/AGENTS/QWEN.

§11.4.101 — Autonomous-decision-over-blocking (User mandate, 2026-05-28)

Forensic: "when working in endless working loop fully autonomously try to decide most properly about points which would block execution and wait for us. If we haven't answered now work would be blocked whole night! If possible and if that will not cause any issues make proper and most reliable and safe decision so we achieve maximal efficiency and work gets fully done!"

In autonomous / endless-loop mode (per §11.4.87) the agent MUST minimize operator-blocking and make the safe, reliable, reversible decision itself — work NOT stalled overnight waiting for input. §11.4.87 = keep working; §11.4.101 = HOW to clear the decision points.

Decision rule (proceed autonomously when ALL): (a) reversible OR pre-op backup per §9.2; (b) safe choice determinable from captured evidence per §11.4.6 (`LIKELY` ≠ determination); (c) wrong-choice blast radius bounded AND recoverable; (d) composes with §11.4 + §12 + §9. Block-only-when (BLOCK via §11.4.66 ONLY when ALL): irreversible AND high-blast-radius AND choice undeterminable from evidence — external-account state, inaccessible hardware, destructive-op-without-backup, force-push (§9.2 + §11.4.41), spending / third-party send. `Operator-blocked` per §11.4.21 only after this fires + self-resolution-exhaustion audit. Maximize-progress-while-blocked: a block parks one work unit, not the loop — keep progressing NON-blocked items per §11.4.87 + §11.4.94; pose-and-idle = §11.4.94 + §11.4.97 violation. Composes §11.4.6/21/40/41/66/87/94/§9.2/§12. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-101-PROPAGATION` (literal `11.4.101`) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.101. Release blocker. No `--always-block-on-decision` / `--never-decide-autonomously` / `--skip-decision-rule` / `--block-without-self-resolution` escape.

§11.4.102 — Systematic-debugging always-on + always-loaded skill-discovery + plugin availability (User mandate, 2026-05-29)

Forensic: “ALWAYS trigger / start the”/superpowers:systematic-debugging” skills when any issues happen! … we MUST activate the skill(s) and make strongest efforts in full in depth analisys / debugging and determine root causes … “/using-superpowers” skill is ALWAYS loaded, applied and used! All dependencies (plugins) that Claude Code or other market places are offering MUST BE installed if these are not already available for loading and use!”

1. On ANY spotted issue/bug/test-failure/gate-failure/regression/misalignment/inconsistency/unexpected-behaviour the agent MUST activate `superpowers:systematic-debugging` (or platform-equivalent) BEFORE proposing/writing/applying any fix — Iron Law: NO FIXES WITHOUT ROOT CAUSE INVESTIGATION FIRST. Four-phase arc: root-cause → pattern → hypothesis (prove/disprove with captured evidence) → implementation (against proven cause only). Guess-and-retry / symptom-patch / re-run-hoping-it-passes (“probably transient/flaky”) without completed investigation = §11.4.102 violation; calling a failure `transient` / `flaky` / `intermittent` without captured forensic evidence = simultaneous §11.4.6 + §11.4.7 violation. (B) `superpowers:using-superpowers` (or platform-equivalent skill-discovery) ALWAYS loaded + applied at session start, consulted before any task — if ANY skill could apply (even 1%) it MUST be invoked, never improvised. (C) Every mandated skill plugin/dependency (Claude Code marketplaces or other runtime ecosystems) MUST be installed + loadable BEFORE dependent work; missing plugin blocking a mandated skill = release-blocker. Install: runtime’s own path — Claude Code `/plugin` flow (add marketplace → install → confirm skill in available-skills list); other runtimes’ documented installer. Anti-bluff: confirm via live capability list (install exit 0 ≠ skill loadable, §11.4.80 lesson). Composes §11.4.4/6/7/8/43/70/82(A)/92. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-102-PROPAGATION` (literal `11.4.102`) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.102. Release blocker. No `--skip-systematic-debugging` / `--guess-and-retry-OK` / `--symptom-patch-permitted` / `--skip-skill-discovery` / `--plugin-optional` / `--missing-plugin-is-warning` escape.

§11.4.103 — Continuous parallel-stream working routine (User mandate, 2026-05-29)

Forensic: “Do this working approach continuously and make it part of regular working routine and add it to the Constitution Submodule documented fully.”

Promotes the proven multi-stream pattern into the project’s standing default working routine (not a per-request opt-in). Load-bearing invariant: main work stream MUST always stay FREE. (A) Main stream FREE — ALL commit AND push run detached (`nohup … &` + `disown` per §11.4.88), never blocking on push or slow mirror. (B) ≥3 parallel background streams at all times + auto-backfill (User mandate 2026-05-29; raised from ≥2) — at least three subagent-driven streams (§11.4.70/§11.4.20, isolated §11.4.58 + §11.4.84) alongside main whenever three-plus non-contending actionable items exist; the moment any one stream is FULLY done a new stream MUST immediately start + take its place (next-highest-priority non-contending item), count NEVER drops below 3 while actionable items remain. Idle below 3 only when no remaining items OR all externally blocked (§11.4.94/97/101). Band 3–6, bounded §12.6 60% mem + §11.4.58 6-agent cap. (C) Most-critical + most-visible first; audio always top §11.4.72 + §11.4.42. (D) Safe-during-build scope only — while heavy `gradle` / `m -j` /42 GB containerised AOSP build (§12.9) runs, streams restrict to §11.4.96 SAFE catalogue; NEVER a second concurrent heavy build (§12.8). (E) Heavy anti-bluff every closure (§11.4.5/6/50/69/102; root cause proven or `UNCONFIRMED:` / `UNKNOWN:` / `PENDING_FORENSICS:`). (F) Idle ONLY when genuinely externally blocked (hardware/network/in-flight build-test-push) OR operator STOP OR §12 host-safety per §11.4.94(A)+§11.4.97; block parks one work unit not the loop (§11.4.101). Composes §11.4.58/70/72/87/88/89/94/96/97/101/102/42/84/§12.6-§12.9/§9.2. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-103-PROPAGATION` (literal `11.4.103`) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.103. Release blocker. No `--block-main-stream` / `--synchronous-commit` / `--synchronous-push` / `--single-stream-only` / `--skip-parallel-streams` / `--serialise-actionable-work` / `--idle-without-queue-survey` escape.

§11.4.104 — Participant identity, attribution & notification-tagging (User mandate, 2026-05-31)

Forensic: “Every supported messenger must relate messages to participants (Subscribers/Users); the same logical person may have a different username on every messenger. Workable items must carry who created them and who they are assigned to. Notifications must @-tag the right participant — but never the operator (who drives the system) and never the system agent.”

MANDATORY for every consumer shipping a messenger/notification surface (Herald + flavor binaries = reference impl; others inherit §11.4.35). Detailed spec: Herald `docs/design/PARTICIPANT_ATTRIBUTION.md` — restated, not redefined. (A) Every messenger MUST relate messages to a **Participant** (logical Subscriber/User); SAME person MAY have a DIFFERENT username per messenger (logical subscriber: canonical messenger-neutral `handle`, `kind ∈ {human,agent,service}`; + per-channel aliases `channel` / `channel_user_id` / `@username` for tagging). Canonical handle closed set: `Claude` (reserved system-agent sentinel; never tagged) OR a subscriber `@username`. (B) Operator = env var `HERALD_<CHANNEL>_OPERATOR_USERNAME`, not a DB flag — the one human who drives via the agent CLI; a normal Participant whose handle equals that value. (C) Workable items MUST carry `created_by` + `assigned_to` (canonical handles): CLI-prompt→Operator; system/agent-detected→ `Claude`; received-message→sender’s resolved `@username`; `assigned_to` defaults to Operator, overridable; legacy items carry `""` and MUST still parse + validate. (D) Tagging matrix: tag `assigned_to` if human ≠ Operator; tag `created_by` if human ≠ Operator ≠ `Claude`; NEVER tag `Claude` or the Operator; de-dup; resolve to channel `@username`, skip if absent there. (E) Anti-bluff (composes §11.4): real SQLite round-trip with the new columns byte-identical (incl. legacy fixtures WITHOUT the fields); tagging matrix proven by a truth-table + cell-flip mutation forcing FAIL; E2E real-event → real-message asserting the exact `@username`s + a NEGATIVE case proving the Operator is NOT tagged; evidence under `docs/qa/<run-id>/`. Composes §11.4 + §11.4.1..16 / §11.4.5 / §11.4.69 / §11.4.50 /

§11.4.91 / §11.4.93 / §11.4.95 / §1.1. Classification: universal (§11.4.17); no-messenger projects inherit latently (§11.4.96 pattern). Propagation gate `CM-COVENANT-114-104-PROPAGATION` (literal `11.4.104`) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.104; detailed spec Herald `docs/design/PARTICIPANT_ATTRIBUTION.md`. Release blocker. No `--skip-attribution` / `--no-participant-tagging` / `--tag-operator-anyway` / `--attribution-later` escape.

§11.4.105 — Natural-language intent recognition & clarification (User mandate, 2026-05-31)

Forensic: “Users must NOT need to know command syntax (no `COMMAND: …` prefix). They send a clear natural-language message; the System determines the intent. The System recognizes the commands it has; if none matches it infers the exact intent; if it is totally unable it replies, tags the user (`@user …`), and asks to clarify precisely. We MUST always do our best to determine exact intent so we never annoy end users. This is a CORE part of the System.”

MANDATORY for every consumer shipping a messenger/command surface (Herald + flavor binaries = reference impl; others inherit §11.4.35). Detailed spec: Herald `docs/design/INTENT_RECOGNITION.md` — restated, not redefined. (A) No required command syntax — users MUST NOT need to know any syntax (no `COMMAND:` prefix); they send plain natural language, the System determines the intent. (B) Three-tier resolution, first that succeeds wins: TIER 1 recognize the System’s existing command set from natural language → action (confident deterministic match); TIER 2 when no command matches, infer the exact intent (LLM maps language → action), NEVER guessing; TIER 3 when neither a command nor a confident intent is determinable, REPLY to the message, TAG the sender (`@username`, via the §11.4.104 IdentityResolver) and ask a PRECISE clarifying question NAMING the candidate intents — no guessing, no silent drop. (C) Never guess, never drop — a wrong action is worse than a clarifying question (composes §11.4.6); a message is NEVER silently dropped; only genuine ambiguity reaches Tier 3, which always replies-tags-and-asks. (D) Anti-bluff (composes §11.4): every tier ships unit + integration + E2E + full-automation tests with real captured evidence — Tier 1 truth-table (natural-language → action+fields + conservative negatives that MUST fall through to “no match”); Tier 3 E2E whose

recorded reply body is EXACTLY `@<sender> <specific question>` + a NEGATIVE proving a clear command does NOT trigger clarify; a paired §1.1 mutation breaking the confidence guard (false-match) OR dropping the clarify tag MUST FAIL a test; evidence under `docs/qa/<run-id>/`. Composes §11.4 + §11.4.1..16 / §11.4.6 / §11.4.104 / §11.4.5 / §11.4.69 / §11.4.98 / §1.1. Classification: universal (§11.4.17); no-messenger projects inherit latently (§11.4.96 pattern). Propagation gate `CM-COVENANT-114-105-PROPAGATION` (literal `11.4.105`) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.105; detailed spec Herald `docs/design/INTENT_RECOGNITION.md`. Release blocker. No `--require-command-syntax` / `--guess-intent-ok` / `--skip-clarify` / `--drop-on-ambiguous` escape.

§11.4.106 — Docs Chain — mechanical documentation/DB sync engine (Operator mandate, 2026-05-31)

Forensic: Docs Chain is the canonical mechanical enforcer of the documentation-sync mandates; consumers MUST use the engine instead of ad-hoc per-project doc-sync scripts, register chains via per-context YAML, and never accept a faked transform.

MANDATORY for every consumer. Docs Chain (the `vasic-digital/docs_chain` engine — a universal Go bidirectional document-and-database dependency-propagation engine) is the canonical mechanical enforcer of the documentation-sync mandates. Detailed spec: engine docs `~/Projects/docs_chain` → `docs/CONSTITUTION_INTEGRATION.md` (distribution + inheritance + anchor mapping table) + `docs/USE_CASE_CATALOGUE.md` (chain recipes) — restated, not redefined. (A) Use the engine, never ad-hoc scripts — consumed by reference (the flat-layout sibling `~/Projects/docs_chain` / the constitution-exposed path), inherited like §11.4.80's `codegraph_*` scripts: referenced, NEVER copied; ad-hoc `sync_*` / `generate_*_summary` / `update_readme_doc_links` scripts superseded + retired per registered context. (B) Consumer-owned contexts — the engine is project-agnostic; the consumer registers chains as data via `.docs_chain/contexts/*.yaml` (§11.4.28 decoupling); `state.json` + `*.docs_chain.tmp` gitignored. (C) Anchors mechanized (per CONSTITUTION_INTEGRATION mapping table): §11.4.12/.53/.45/.56/.57/.59/.60/.65/.86/.93/.95/.44 + §12.10 — content-hash change detection (NOT mtime, §11.4.86), atomic-rename + SQLite-txn commit +

rollback (§9.2), both-dirty `sync` → conflict-not-silent-merge (§11.4.6), `verify` as the deterministic CI/pre-build gate (§11.4.50), per-run captured evidence to `qa-results/docs_chain/<run-id>/` (§11.4.69). (D) NOT a replacement for authoring discipline — the source author still writes the §11.4.44 revision header; the engine only keeps exports in sync. (E) Anti-bluff (composes §11.4): NEVER fakes a transform — a missing pandoc/weasyprint surfaces a typed `ToolAbsentError` + honest §11.4.3 SKIP-with-reason, never a fake PASS / partial write; every `sync` / `verify` carries real captured evidence. Composes §11.4 + §11.4.1..16 / §11.4.6 / §11.4.28 / §11.4.80 / §9.2 / §11.4.50 / §11.4.69 / §11.4.5 / §1.1 + the mechanized sync anchors. Classification: universal (§11.4.17); no derived-export/DB-sync projects inherit latently (§11.4.96 pattern). Status (§11.4.6): engine Phases 1–3 AND the CLI/YAML loader (Phase 4) IMPLEMENTED+tested — `cmd/docs_chain/main.go` registers `sync` / `verify` / `diff` (alias)/ `doctor` , `go test -count=1 ./...` GREEN 7/7 packages; this CLI/loader is in the working tree but UNTRACKED and the engine is NOT yet a registered git submodule (in-tree at `./docs_chain` , HEAD = parent HEAD); only submodule distribution (Phase 6) remains PLANNED + OPERATOR-GATED. Propagation gate `CM-COVENANT-114-106-PROPAGATION` (literal `11.4.106`) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.106; detailed spec Docs Chain engine docs `docs/CONSTITUTION_INTEGRATION.md` + `docs/USE_CASE_CATALOGUE.md` (`vasic-digital/docs_chain`). Release blocker. No `--ad-hoc-sync-ok` / `--skip-docs-chain` / `--fake-transform` / `--sync-evidence-optional` escape.

§11.4.107 — Anti-bluff AV/test-validation techniques mandate (User-driven research, 2026-06-02)

Forensic (genericised, 2026-06-02): a test PASSed on a SINGLE captured frame showing “a picture” — but it was a FROZEN/STALE frame from previously-played content (stale-producer/stuck-decoder), feature broken for the user while green; a sibling FLASHED media on the WRONG output for ~1 s before routing with no test sampling that window; a third shipped a comparator that PASSed its own deliberately-degraded fixture (the analyzer was the bluff). §11.4.5 mandates captured evidence + a presence pass; §11.4.107 raises it to liveness + correct-routing + self-validated-analyzer.

Every test asserting audio/video output is genuinely playing/advancing MUST satisfy ALL: (1) a single captured frame is NOT proof — prove LIVE, ADVANCING frames over a window via a freeze-detection oracle (near-duplicate/ `freezedetect`-class OR perceptual-hash adjacent-frame distance, NOT byte-identical — byte-identity only a pre-filter); (2) an independent frame-advance counter from the platform's compositor/decoder telemetry must increase (different-domain oracle — flat ⇒ stuck decoder ⇒ FAIL even if pixels appear to move); (3) loading/buffering is a distinct state — wait for genuine playback-start, never false-FAIL a still-loading stream nor false-PASS a spinner (timeout+unreachable ⇒ SKIP §11.4.3, timeout+reachable ⇒ FAIL); (4) not-stale-from-previous cross-check (new first frame ≠ previous last frame); (5) measured FPS / no-lost-frames within tolerance; (6) no-flash-on-the-wrong-output — high-frequency sampling of the non-target output during routing transitions, any content frame there ⇒ FAIL (content-protection regime classified, never guessed); (7) drive through the realistic feed/UI path, not deep-link shortcuts (UI-not-introspectable ⇒ operator-attended §11.4.52, never fake-PASS); (8) metamorphic relations for the oracle problem on un-owned streams (same content output-A vs output-B must match; paused ⇒ counter stops; 2× speed ⇒ ~2× advance); (9) full-reference quality metrics (SSIM/VMAF/ΔE2000) vs golden source for owned content; (10) mutation-test every analyzer with a golden-good + golden-bad fixture pair so the analyzer cannot bluff (PASS on golden-bad = bluff gate — §1.1 applied to analyzers); (11) per-channel audio RMS/ loudness (EBU R128) + XRUN census (aggregate RMS misses a dead channel); (12) OCR overlay/subtitle needs a per-word confidence floor + ROI (avoids both false-FAIL and false-PASS); (13) thresholds calibrated on the project's own fixtures, not hardcoded from literature (§11.4.6). Honest gap: true photon FPS at the sink has no clean software oracle — flag it, never claim it.

4-layer §11.4.4(b): pre-build gate (`CM-AV-LIVENESS-NO-FROZEN-FRAME`-class + analyzer-self-validation gate wiring golden-good/golden-bad fixtures into meta-test) + runtime/on-device test + paired §1.1 mutation (single-frame-only → FAILs; analyzer PASSing its golden-bad fixture → self-validation FAILs) + HelixQA Challenge. Every PASS via `ab_pass_with_evidence` citing motion / not-stale / frame-advance / fps / loudness / metamorphic / self-validation artefacts (`video_display` / `audio_output` / `subtitle_render` §11.4.69) — never a single screenshot. Classification: universal (§11.4.17) — platform-neutral, reusable by ANY project validating media playback or any pixel/audio output; project supplies its capture mechanism + calibrated thresholds per §11.4.35. Composes

§11.4.5/.6/.50/.68/.69/.85 + §11.4.2/.3/.13/.48/.52/§1.1. Propagation gate `CM-COVENANT-114-107-PROPAGATION` (literal `11.4.107`) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.107. Release blocker. No `--single-frame-proves-playback` / `--skip-liveness` / `--byte-identical-freeze-OK` / `--no-frame-advance-counter` / `--allow-wrong-output-flash` / `--deep-link-shortcut-OK` / `--unvalidated-analyzer-OK` / `--aggregate-rms-suffices` / `--hardcoded-thresholds-OK` escape.

§11.4.108 — Four-layer fix-verification + runtime-signature-as-definition-of-done mandate (systematic-debugging Phase 4.5, 2026-06-03)

Forensic anchor (genericised, 2026-06-03): across one batch multiple fixes were “green” at every gate yet NOT working for the end user — a source-committed change passed by both the source pre-build gate AND the post-build gate never reached the boot-time command-line embedded in the deployed boot artifact (feature dead despite all-green); sibling fixes correctly built into the system image were masked by stale per-user overlay copies from a prior deployment (running code = stale shadow, not the fresh build); deployment did not wipe the mutable overlay so the shadow survived; validation ran against the stale shadow and reported green on code never exercised. Per `superpowers:systematic-debugging` Phase 4.5: each fix revealing a fresh “fixed-but-not-working” in a DIFFERENT place is ONE architectural VERIFICATION flaw, not independent bugs — patching symptoms is thrashing. A fix crosses FOUR distinct layers, and “fixed” at one does NOT imply fixed at the next: (1) SOURCE (committed — grep-the-source gate), (2) ARTIFACT (the change’s BYTES landed in the produced artifact — image / bundle / installer / embedded command-line), (3) RUNTIME-ON-CLEAN-TARGET (active on a CLEAN/fresh deployment, the layer the end user experiences, no stale overlay shadowing it), (4) USER-VISIBLE (works for the end user — §11.4.5/§11.4.69 captured-evidence). The mandate (ALL hold): (1) **runtime-signature-as-definition-of-done** — a fix is DONE only when its declared runtime signature verifies on a CLEAN deployment; source-committed ≠ artifact-contains-it ≠ active-on-clean-target ≠ user-visible-working; (2) every fix declares ONE machine-checkable runtime signature (running-system property / sink-side report per §11.4.13 / §11.4.5/§11.4.69 captured-evidence / counter delta — NEVER a re-

grep of the source); this registry of per-fix runtime signatures is the SINGLE SOURCE OF TRUTH for “fixed”, REPLACING “a gate greps the source”; (3) gates span all four layers — source (pre-build), artifact (post-build — assert the BYTES landed, not merely that the source still contains the change), runtime-on-clean-target (post-deploy — assert the runtime signature), user-visible (§11.4.5/§11.4.69); (4) eliminate the stale-deployment / shadow layer by construction — deployment MUST yield a CLEAN state (wipe the mutable overlay) OR a pre-validation assertion MUST prove `running-artifact == built-artifact` before any validation; validation against possibly-stale state is INVALID and its PASS is a §11.4 PASS-bluff; (5) meta-rule — ≥3 “fixed-but-not-working” discoveries in one cycle signal an architectural VERIFICATION flaw, not three independent bugs; on the 3rd STOP patching symptoms (§11.4.4), fix the VERIFICATION pipeline, re-certify EVERY item through it on a clean target; (6) a batch is “validated” only after COMPREHENSIVE per-item runtime-signature verification on a clean baseline, NOT after the touched items’ own tests pass. Classification: universal (§11.4.17) — platform-neutral verification-pipeline disciplines reusable by ANY project that builds → deploys → validates an artifact; the project supplies its artifact format, clean-deployment / equal-artifact mechanism, and per-fix runtime-signature observables per §11.4.35. Composes §11.4.1/.2/.4/.5/.6/.27/.40/.46/.50/.52/.69/.102. Propagation gate `CM-COVENANT-114-108-PROPAGATION` (literal `11.4.108`) + recommended per-fix gate `CM-RUNTIME-SIGNATURE-REGISTRY` + paired §1.1 mutations (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.108. Release blocker. No `--source-green-is-done` / `--skip-artifact-byte-check` / `--validate-against-running-state` / `--no-clean-deployment` / `--skip-runtime-signature` / `--spot-validate-touched-only` escape.

§11.4.109 — Mandatory Anti-Forgetting Enforcement: PreToolUse Guard Hook + Subagent Constitutional Preamble + Orchestrator Pre-Action Checklist (Operator mandate)

UNCONFIRMED forensic anchor (pending operator’s verbatim mandate quote). Background: emulator subagents ran raw host-direct `emulator` / `adb` because the orchestrator forgot to inject the Containers-submodule rule at dispatch. A forgotten rule is not enforcement. Two-layer fix: (A) portable bash/awk `PreToolUse` hook (`constitution/scripts/hooks/guard-forbidden-`

`commands.sh`, no `jq`, no project-specific paths) blocks host-direct emulator / force-push bypass / sudo / host-power at the tool-call boundary regardless of agent memory (the floor); (B) `constitution/docs/AGENT_GUARDRAILS.md` — canonical preamble the orchestrator pastes verbatim into every subagent dispatch + Orchestrator Pre-Action Checklist (the ceiling). Consuming projects wire the hook in `.claude/settings.json` or equivalent; maintain `docs/AGENT_GUARDRAILS.md` with `SUBAGENT CONSTITUTIONAL PREAMBLE`, `ORCHESTRATOR PRE-ACTION CHECKLIST`, and the anchor literal `11.4.109`; provide a ≥20-case hermetic hook test. The hook is inherited by reference — NEVER copied locally. Gates: `CM-ANTI-FORGETTING-ENFORCEMENT` + `CM-COVENANT-114-109-PROPAGATION` + paired §1.1 mutations (gate-code = separate work item). Classification: universal (§11.4.17). Composes §11.4.6/.10/.75/.76/.78/.79/.80/.81/.84/.98/.102/§12.

Canonical authority: `Constitution.md` §11.4.109; ref impl `constitution/scripts/hooks/guard-forbidden-commands.sh` + `constitution/docs/AGENT_GUARDRAILS.md`. Release blocker. No `--skip-pretooluse-hook` / `--no-guardrails-doc` / `--anti-forgetting-optional` / `--single-layer-sufficient` escape.

§11.4.110 — Pre-build build-readiness verdict + change-impact clash detection mandate (operator mandate, 2026-06-03)

Forensic anchor (genericised, 2026-06-03): a fix shipped a new system-property *read* but introduced no matching security-policy grant + no property-context type entry — the read was silently denied at runtime + the feature was dead, while every pre-build gate stayed green because the gate only grepped the source file. Generalises: a change introduces a new dependency on a *second* artifact (security policy, context file, service registry, interface freeze-snapshot, symbol table, build-graph node) that the pre-build never cross-checks. This is §11.4.108's SOURCE→ARTIFACT gap shifted LEFT to pre-build — most such clashes are statically catchable from the change diff itself. The mandate (ALL hold): (1) a single deterministic **READY-FOR-BUILD verdict** gates the rebuild (orchestrator refuses to start unless READY); (2) a **diff-driven change-impact + clash detector** cross-checks every newly-introduced second-artifact dependency (new property read ⇄ property-context type + read-grant; new service ⇄ service-context; new init service ⇄ security label; new/changed stable interface ⇄ freeze-snapshot updated; new

native-lib dep ⇄ module/prebuilt resolves; new policy rule ⇄ every type/attribute defined; two batch changes on the same seam ⇄ collision acknowledged); (3) **coverage-completeness is a gate** — every changed file maps to ≥1 gate + ≥1 deployed-target test + ≥1 paired §1.1 mutation, baseline ratchets per §11.4.50; (4) **two-speed honesty** — grep-speed always-on gates vs REQUIRES_BUILD heavy gates (build-graph parse-only dry-run, full neverallow compilation, ABI diff) as diff-gated opt-in stages bounded per §12.6/§12.7; (5) every gate + wired analyzer is anti-bluff by paired §1.1 mutation; (6) **honest boundary** — a READY verdict proves static internal-consistency + ready-to-build, NOT that the feature works on the deployed target; the regime empties the *preventable* class, not the *all-defects* class (runtime/USER-VISIBLE remains §11.4.108's job). Classification: universal (§11.4.17) — platform-neutral pre-build-rigor disciplines reusable by ANY artifact-building project; the project supplies its property/service/policy registries, build-graph parse-only command, interface-freeze mechanism + changed-file→gate mapping per §11.4.35. Composes §11.4.1/.4/.6/.9/.27/.50/.67/.75/.92/.108. Propagation gate `CM-COVENANT-114-110-PROPAGATION` (literal `11.4.110`) + recommended `CM-READY-FOR-BUILD-VERDICT` / `CM-CHANGE-IMPACT-CLASH-DETECTOR` / `CM-COVERAGE-COMPLETENESS-GATE` / `CM-BUILDGRAPH-DRYRUN-WIRED` / `CM-SEPOLICY-NEVERALLOW-WIRED` + paired §1.1 mutations (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.110. Release blocker. No `--source-green-is-ready` / `--skip-clash-detector` / `--skip-coverage-gate` / `--no-ready-verdict` / `--grep-proves-neverallow` / `--skip-buildgraph-dryrun` / `--build-without-ready-verdict` escape.

§11.4.111 — Resolve-by-stable-name-not-by-enumeration-index mandate (research-derived, 2026-06-03)

Forensic anchor (genericised, 2026-06-03): a platform bound an audio output to a kernel-enumerated index (`card=0`); a second device of a different class enumerated FIRST at boot + took slot 0, shifting the intended device to slot 1 — the static index binding now pointed at the wrong device (policy mis-attached the sink, mis-assigned its TYPE, the output switcher labelled the AV receiver "Wired Headphone" + routing collapsed to stereo). The lower layer of the SAME stack already resolved by **name** (controller-name registry) — proving the brittleness was the *index*, not the resolution capability. Generalises to every enumerated resource

whose ordinal is assigned at discovery/boot/hotplug (non-deterministic across reboots, additions, topology changes). The mandate: any binding to a hardware device / resource handle / enumerated entity (audio cards, display connectors, network interfaces, storage devices, GPU render nodes, input/camera devices, container/process slots) MUST resolve by a **stable identifier** (name / UUID / serial / label / controller-name / content-hash / sink-reported identity), NOT by enumeration index / ordinal / slot, UNLESS the platform documents the ordinal as deterministically pinned AND the pin is captured + asserted in the binding. Where a stable identifier exists at one layer, every layer binding the same resource MUST use it (mixed by-name-here / by-index-there is the forbidden weak link). Honest boundary (§11.4.6): when only an ordinal exists, pin deterministically via the platform's mechanism, capture the pin in the §11.4.108 runtime signature, document residual fragility as `UNCONFIRMED:`-class — never silently trust an unpinned ordinal. Classification: universal (§11.4.17) — platform-neutral binding-robustness discipline; the project supplies its stable-identifier mechanism (ALSA card name, DRM connector name, NIC predictable name, block-device UUID, etc.) + the layers that must agree per §11.4.35. Composes §11.4.6/.8/.69/.108/.110. Propagation gate `CM-COVENANT-114-111-PROPAGATION` (literal `11.4.111`) + recommended `CM-RESOLVE-BY-NAME-NOT-INDEX` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.111. Release blocker. No `--allow-index-binding` / `--ordinal-is-stable-enough` / `--skip-name-resolution` / `--trust-unpinned-index` escape.

§11.4.112 — Structural-impossibility won't-fix classification mandate (research-derived, 2026-06-03)

Forensic anchor (genericised, 2026-06-03): deep research (§11.4.8) into relocating protected (content-protection / secure-surface) video to a secondary display PROVED — from authoritative platform/HDCP docs + reproducible captured behaviour (a secure surface is blanked on any output lacking the secure flag; mirror/screencap of a secure layer returns black) — that the goal is **structurally impossible by platform design**, not a missing feature. Without a durable classification it is re-investigated every cycle (re-read the same sources, re-run the same probes, re-derive the same impossibility — compounding wasted effort). The mandate: when deep research per §11.4.8 PROVES (cited authoritative sources

AND, where applicable, reproducible captured evidence) a goal is structurally impossible on the target platform (forbidden by platform design / hardware-protocol constraint / documented kernel-or-API limitation — NOT merely unimplemented or hard), the goal MUST be: (1) classified `Won't-fix` + closed per §11.4.90 with reason `structurally-impossible`; (2) documented with the impossibility evidence — cited source URLs (per §11.4.99) + reproducible probe/captured evidence — in the tracker entry + relevant `docs/` guide; (3) NOT re-attempted in future cycles — a reopen MUST cite NEW evidence the constraint changed (per §11.4.34 + §11.4.7), never re-derive the same impossibility; (4) paired with the correct posture — the entry states what the project DOES instead, so “impossible” is never confused with “broken/unhandled”. Honest boundary (§11.4.6): reserved for *proven* impossibility — “could not find a way” / “very hard” / “no time” are `Operator-blocked` (§11.4.21) or open work, NOT won’t-fix; mislabelling them is a §11.4 planning-layer bluff. A future platform change can make the impossible possible (durable, not eternal). Classification: universal (§11.4.17) — platform-neutral effort-conservation + honesty discipline; the project supplies the impossible goal, its constraint + cited evidence per §11.4.35. Composes §11.4.6/.7/.8/.34/.90/.99. Propagation gate `CM-COVENANT-114-112-PROPAGATION` (literal `11.4.112`) + recommended `CM-WONT-FIX-STRUCTURAL-IMPOSSIBILITY` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.112. Release blocker. No `--wont-fix-without-proof` / `--reattempt-closed-impossible` / `--skip-impossibility-evidence` / `--impossible-equals-broken` escape.

§11.4.113 — Absolute no-force-push + merge-onto-latest-main mandate (User mandate, 2026-06-03)

Forensic anchor — verbatim user mandate (2026-06-03): “Any force-push is strictly forbidden! We must for every Submodule take as a base latest commit on Submodule’s main (or master) branch, then on top of it carefully to merge all changes that have to be pushed! Once all merging is carefully done we perform commit and push to all Submodule’s upstreams!” Force-push is STRICTLY FORBIDDEN with NO exception — `git push --force`, `--force-with-lease`, `+<ref>`, any history-rewriting overwrite of a remote ref — against EVERY repository this Constitution governs (main repo, this constitution submodule, every owned + nested submodule, every upstream). No operator-approval path, no “after

merge-first audit” path. Mandated 6-step integration for any repo/submodule with commits to publish OR diverged mirrors: (1) `git fetch --all --prune --tags` all remotes; (2) base = LATEST commit on canonical `main` / `master` (most-advanced mirror tip); (3) carefully MERGE every change on top — union, preserve BOTH sides, NEVER `-s ours` / rebase / reset that drops commits (§9 no-commit-loss); (4) resolve conflicts carefully — no markers, no file dropped, gates/tests still pass; (5) commit the merge (stage only intended files, NEVER `git add -A` in a submodule per §11.4.30); (6) push to ALL upstreams — each is a fast-forward because the merge commit descends from every mirror tip, so NO force is needed (an upstream still rejecting → return to step 1 for it, merge its new tip, re-validate, re-push). TIGHTENS §11.4.41 / §11.4.71 / §9.2 / CONST-043 — force-push escape hatch REMOVED: even WITH operator approval, even after a clean merge-first audit, force-push is forbidden, because the merge-onto-latest-main path is always available so force is never necessary; those clauses’ merge-first/fetch-first machinery stays, only their terminal “…then force-push” step is struck. Classification: universal (§11.4.17). Composes §2.1 / §9 / §9.2 / §11.4.4 / §11.4.6 / §11.4.26 / §11.4.37 / §11.4.40 / §11.4.41 / §11.4.71 / §11.4.88 / CONST-043. Propagation gate `CM-COVENANT-114-113-PROPAGATION` (literal `11.4.113`) + recommended `CM-NO-FORCE-PUSH-ABSOLUTE` (scan tracked scripts/hooks for `push --force` / `--force-with-lease` / `push +<ref>` + reject; §11.4.109-class PreToolUse guard blocks the class) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.113. Release blocker. No `--force` / `--force-with-lease` / `--allow-force-push` / `--force-push-authorised` / `--skip-merge-onto-main` escape.

§11.4.114 — Last-known-good-tag regression isolation mandate (1.1.8-dev remediation, 2026-06-03)

When a previously-working feature/behaviour is observed broken, the FIRST diagnostic action MUST be to identify the last release tag (or commit/build/deploy) at which it was KNOWN-GOOD and diff/bisect the broken state against it — BEFORE any open-ended root-cause hunt or speculative fix. The known-good revision is the regression oracle: bounds the search to commits between good and now (`git diff <good-tag>..HEAD --stat` of the feature’s files = captured evidence), marks it a regression REPAIR not from-scratch design, gives each

suspect file a behavioural oracle. An operator-volunteered known-good tag is load-bearing → act on it first. Default to SURGICAL forward-fix (keep post-good-tag features, revert ONLY the broken sub-part) over wholesale revert (which loses the batch's other working features) unless operator prefers wholesale. Honest boundary (§11.4.6): "it worked before" is a HYPOTHESIS until the tag is identified AND the feature confirmed working there; "probably regressed last batch" without the diff is a guess; a never-worked feature is not a regression. Classification: universal (§11.4.17). Composes §11.4.4 / §11.4.6 / §11.4.7 / §11.4.40 / §11.4.43 / §11.4.102 / §11.4.108. Propagation gate `CM-COVENANT-114-114-PROPAGATION` (literal `11.4.114`) + recommended `CM-REGRESSION-ISOLATED-AGAINST-KNOWN-GOOD` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.114. Release blocker. No `--skip-known-good-diff` / `--root-cause-from-scratch` / `--assume-regression-source` / `--wholesale-revert-without-isolation` escape.

§11.4.115 — RED-baseline-on-the-broken-artifact + polarity-switch mandate (1.1.8-dev remediation, 2026-06-03)

Strict refinement of §11.4.43's RED step. Every RED test MUST REPRODUCE the defect on the CURRENT pre-fix artifact (the actual broken build/deploy), capturing positive evidence per §11.4.5 / §11.4.69 / §11.4.107 that the defect is genuinely present — never a synthetic failure the fix is then written to agree with. The SAME source MUST carry a single polarity switch (env flag / parameter, canonical `RED_MODE` , default `1` = reproduce-and-assert-defect-present) that flipped to `0` post-fix becomes the GREEN regression-guard asserting the defect is ABSENT. One source, two roles: the bug-catcher IS the regression-guard; no separate happy-path test is the primary guard (that only proves the test agrees with the fix — the §11.4.43 PASS-bluff). RED-on-broken then GREEN-on-fixed (clean target per §11.4.108) both captured. Honest boundary (§11.4.6): a RED run that does NOT fail on the broken artifact is a finding (close per §11.4.7 with negative evidence, or fix the blind test). Classification: universal (§11.4.17). Composes §11.4.1 / §11.4.2 / §11.4.5 / §11.4.69 / §11.4.107 / §11.4.4 / §11.4.7 / §11.4.43 / §11.4.50 / §11.4.108 / §11.4.114. Propagation gate `CM-COVENANT-114-115-PROPAGATION` (literal `11.4.115`) + recommended `CM-RED-POLARITY-SWITCH-PRESENT` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.115. Release blocker. No `--green-only-test` / `--skip-red-reproduction` / `--separate-happy-path-suffices` / `--synthetic-red-OK` escape.

§11.4.116 — Real-time conductor↔autonomous-test-framework sync channel mandate (1.1.8-dev remediation, 2026-06-03)

Any autonomous, long-running test/QA/validation framework an external orchestrator (conductor agent / operator) depends on for real-time decisions MUST expose a real-time sync channel: (1) a structured append-only event stream (JSONL or equivalent, one event per line, never rewritten — session-start / phase-transition / per-test-or-challenge-start / captured-evidence-path / external-call (LLM/vision/sink-probe) / error / per-item-verdict); (2) an atomically-rewritten status snapshot (single small file, write-temp-then-rename so no torn read) with current session/phase/item + counters + last verdict. Verdicts use the closed PASS / FAIL / SKIP / OPERATOR-BLOCKED vocabulary (§11.4.45). The conductor tails it live → real-time sync, §11.4.4 interrupt on a fresh defect, never idles blindly (§11.4.94 / §11.4.97). Anti-bluff: every verdict event MUST carry the evidence path that backs it (§11.4.69) — a PASS event with no evidence path is a channel-layer bluff; a snapshot PASS contradicted by a stream with no evidence event → treat as FAIL; no start-event → no verdict-event. Owned project-agnostic submodule (§11.4.28): channel stays project-neutral, the consumer registers its data via the public API at runtime, never hardcoded. Classification: universal (§11.4.17). Composes §11.4.4 / §11.4.5 / §11.4.69 / §11.4.27 / §11.4.28 / §11.4.45 / §11.4.52 / §11.4.89 / §11.4.94 / §11.4.97. Propagation gate `CM-COVENANT-114-116-PROPAGATION` (literal `11.4.116`) + recommended `CM-AUTONOMOUS-FRAMEWORK-SYNC-CHANNEL` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.116. Release blocker. No `--no-sync-channel` / `--end-of-session-report-suffices` / `--verdict-without-evidence-path` / `--torn-status-write-OK` escape.

§11.4.117 — Computer-vision / OCR pixel-oracle fallback for non-introspectable UIs mandate (1.1.8-dev remediation, 2026-06-03)

Any test that drives a UI control OR asserts on-screen content MUST NOT assume the accessibility/semantic/DOM hierarchy is the source of truth for what the user sees. When the hierarchy is blank/partial/known-unreliable (TV-Compose, leanback, canvas/ `SurfaceView` /GL, games, custom-rendered UIs), the test MUST fall back to a PIXEL ORACLE: (1) DRIVE input by computer-vision template-match (locate a control by rendered appearance → tap coordinates), not a node id; (2) ASSERT content by ROI OCR (read the rendered text the user reads) with a per-word confidence floor + ROI per §11.4.107(12), not a hierarchy text attribute that may be absent/stale. The tool MUST both drive input AND read pixels — a hierarchy-only tool is NOT a content oracle. Makes §11.4.52's "near-empty hierarchy → INFEASIBLE" constructive: pixel-drive, not only SKIP. Anti-bluff (§11.4.107(10)): the CV/OCR analyzer is self-validated (golden-good PASS, golden-bad FAIL, wired into meta-test); thresholds calibrated on the project's own frames, not hardcoded (§11.4.6). Honest boundary: BOTH hierarchy blank AND pixel oracle infeasible (secure surface black-captures §11.4.112, geo-unreachable) → SKIP-with-reason §11.4.3 + tracked operator-attended migration §11.4.52, never fake PASS. Classification: universal (§11.4.17). Composes §11.4.5 / §11.4.6 / §11.4.48 / §11.4.49 / §11.4.52 / §11.4.107 / §11.4.112. Propagation gate `CM-COVENANT-114-117-PROPAGATION` (literal `11.4.117`) + recommended `CM-CV-OCR-PIXEL-ORACLE-FALLBACK` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.117. Release blocker. No `--hierarchy-is-content-oracle` / `--skip-pixel-fallback` / `--unvalidated-ocr-OK` / `--hardcoded-ocr-threshold-OK` escape.

§11.4.118 — Discovery-pressure to confirm known-issue-set completeness mandate (1.1.8-dev remediation, 2026-06-03)

A remediation/release cycle MUST NOT treat "every reported defect is fixed" as "the build is good" — the reported set is biased by what the operator happened to test ("we only see what we test"). After/alongside fixing the reported set, run a discovery + stress pass across ALL target devices/environments that deliberately exercises subsystems/journeys/edge-cases BEYOND the reported defects — to

CONFIRM the reported set is the COMPLETE critical set + surface unreported defects before the user does. The pass MUST produce PROVABLE coverage: an enumerated list of subsystems/journeys/stress-scenarios exercised, each with outcome (no-new-issue, or a new tracker entry per §11.4.15 / §11.4.16). “We found no other issues” is a §11.4 bluff unless paired with “here is the enumerated set we exercised” — absence of evidence of looking is not evidence of absence. New findings trigger §11.4.4 interrupt + §11.4.114/§11.4.115 isolation→RED→fix. Honest boundary (§11.4.6): discovery reduces the unknown-unknown surface, does not prove zero remaining defects — earned claim is “reported set + enumerated discovery set addressed,” un-exercised subsystems stated as honest coverage gaps (§11.4.3), never silently implied clean. Classification: universal (§11.4.17). Composes §11.4.4 / §11.4.5 / §11.4.69 / §11.4.25 / §11.4.40 / §11.4.42 / §11.4.52 / §11.4.85 / §11.4.114 / §11.4.115 / §11.4.119. Propagation gate `CM-COVENANT-114-118-PROPAGATION` (literal `11.4.118`) + recommended `CM-DISCOVERY-COVERAGE-ENUMERATED` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.118. Release blocker. No `--reported-set-suffices` / `--skip-discovery-pass` / `--no-issues-without-coverage-list` / `--assume-complete` escape.

§11.4.119 — Single-resource-owner partitioning for parallel hardware testing mandate (1.1.8-dev remediation, 2026-06-03)

Strict refinement of §11.4.58 / §11.4.103 for hardware contention. When multiple parallel streams exercise SHARED hardware / any exclusive-access resource (a media-playback path, a single HDMI/audio sink, an exclusive device handle, a serial/JTAG line, a GPU under exclusive capture), exactly ONE stream MUST own each such resource at a time. The owner drives it (playback, input injection, capture); every other concurrent stream targeting it MUST be READ-ONLY (passive probes — `dumpsys` / `/proc` / `/sys` reads / sink-side network probes / log tails). Parallelism partitioned by resource: distinct devices/sinks/handles fully concurrent (stream-per-device), but the same device’s exclusive resource is single-owner. Ownership enforced by an advisory lock/token (§11.4.58 L3 + the hardware analogue of §11.4.84 quiescence), event-driven (claim when the resource frees, release the moment work completes so the next queued stream claims it per

§11.4.103 auto-backfill). Why: concurrent drivers of one exclusive resource produce CROSS-CONTAMINATED evidence (sink reports the wrong stream's audio, foreground belongs to whichever `am start` landed last, input events interleave) — a PASS under contention is a §11.4 evidence-integrity bluff. Honest boundary (§11.4.6): a "read-only" probe stream MUST issue NO state-changing command against a device it does not own; a non-partitionable resource → streams serialize (single-owner over time), not run concurrently and hope. Classification: universal (§11.4.17). Composes §11.4.5 / §11.4.69 / §11.4.13 / §11.4.50 / §11.4.58 / §11.4.82 / §11.4.84 / §11.4.103 / §11.4.118. Propagation gate `CM-COVENANT-114-119-PROPAGATION` (literal `11.4.119`) + recommended `CM-SINGLE-RESOURCE-OWNER-PARTITION` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.119. Release blocker. No `--allow-concurrent-resource-drivers` / `--skip-ownership-lock` / `--read-only-may-mutate` / `--contended-evidence-OK` escape.

§11.4.120 — Fix-breaks-its-own-gate reconciliation mandate (1.1.8-dev remediation, 2026-06-03)

When a correct fix causes a pre-existing gate/test to FAIL because that gate asserted the OLD (now-removed-or-changed) behaviour, the gate FAIL is the CORRECT signal the fix landed — it MUST NOT be suppressed by either forbidden response: (1) FAKE-PASSING the gate (edit to `always pass`, weaken to a tautology, or delete — a §11.4 gate-layer bluff + a §11.4.84 mutation-residue risk); (2) REVERTING the correct fix to satisfy the stale gate (re-introducing the defect). Required response = RECONCILIATION: rewrite the gate to assert the NEW mechanism the fix introduced, backed by captured evidence of the new correct behaviour, AND update its paired §1.1 mutation so the mutation breaks the NEW invariant. Discriminator vs bluffing: after reconciliation gate + mutation still form a valid §1.1 pair (mutate the new invariant → gate FAILs); a bluffed gate's mutation no longer makes it FAIL. The reconciliation MUST be a visible evidence-cited change, never a silent assertion-weakening. Honest boundary (§11.4.6): a post-fix gate FAIL is NOT automatically "stale gate" — investigate per §11.4.102 first; the FAIL may be the gate correctly catching a REGRESSION the fix introduced (then the FIX is wrong). Reconcile ONLY when investigation PROVES the gate asserted old-correct-now-removed behaviour AND the new behaviour is intended +

evidence-confirmed. Classification: universal (§11.4.17). Composes §11.4.1 / §11.4.4 / §11.4.6 / §11.4.84 / §11.4.102 / §11.4.108 / §1.1. Propagation gate `CM-COVENANT-114-120-PROPAGATION` (literal `11.4.120`) + recommended `CM-GATE-RECONCILED-NOT-FAKE-PASSED` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.120. Release blocker. No `--fake-pass-stale-gate` / `--revert-fix-for-gate` / `--weaken-assertion-to-pass` / `--delete-failing-gate` escape.

§11.4.121 — No-commit-while-build-writes-tracked-artifacts mandate (1.1.8-dev remediation, 2026-06-03)

A commit (especially `git add -A` / any broad stage) MUST NOT run while a build/packaging/generation step is actively writing artifacts INTO tracked (version-controlled) directories — it races the writer and stages a PARTIAL or stale artifact (a §11.4.108 SOURCE→ARTIFACT integrity failure landed in version control). The commit MUST be deferred until the build step that writes tracked artifacts has COMPLETED, so the tree is quiescent at the artifact layer AND the committed artifacts are the FRESH, whole outputs (no stale pre-rebuild artifact, no half-written file). Before committing tracked build outputs, verify the writing step finished (process exit / completion marker / per-artifact mtime ≥ build-start). Build-output analogue of §11.4.84 (no commit while a mutation gate is in flight): both close “commit captures transient non-final tree state” at two write-sources. Gitignored build outputs (`out/` / `dist/`) are exempt — the race doesn’t apply. Honest boundary (§11.4.6): “the build probably finished” is not “the build finished” — verify with a completion signal; source changes NOT touching the build’s tracked-artifact dirs are fine mid-build. PROJECT-SPECIFIC instance (which tracked dir + which build steps write it) recorded in the consuming project’s own governance per §11.4.35. Classification: universal (§11.4.17). Composes §11.4.6 / §11.4.30 / §11.4.58 / §11.4.84 / §11.4.88 / §11.4.96 / §11.4.103 / §11.4.108. Propagation gate `CM-COVENANT-114-121-PROPAGATION` (literal `11.4.121`) + recommended `CM-NO-COMMIT-DURING-ARTIFACT-WRITE` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.121. Release blocker. No `--commit-during-build` / `--stage-partial-artifact-OK` / `--assume-build-finished` / `--skip-build-completion-check` escape.

§11.4.122 — No-silent-removal-of-existing-components-without-operator-confirmation mandate (User mandate, 2026-06-03)

Forensic anchor — verbatim user mandate (2026-06-03): “Never ever remove any application, system component or service from already existing codebase / System without interactively asked question to us! THIS IS MANDATORY RULE / CONSTRAINT!” Case study (FACT): in the 1.1.8-dev burn-down, F2 (an Apple-TV-class app) + F4 (a Huawei HMS component) were removed WITHOUT asking; the operator reversed both.

No application / system component / service / package / feature / driver / module / library / prebuilt asset — any already-existing end-user capability — may be removed (deleted, dropped from the package set, disabled-into-non-shipping, un-bundled, de-listed) from the existing codebase / shipped System WITHOUT FIRST interactively asking the operator (via the §11.4.66 interactive mechanism — `AskUserQuestion` on Claude Code, never free-text, never silent, never an autonomous decision) and receiving an EXPLICIT keep-or-remove decision. A removal is: a deletion from the package/service/module manifest set whose NET EFFECT is “a capability that shipped before no longer ships.” Adding / replacing-with-operator-approved-equivalent / fixing is NOT a removal; when uncertain, treat AS a removal and ask (§11.4.6 no-guessing). A silent removal is a release blocker regardless of rationale (dedup / “broken anyway” / geo-restricted / incompatible / superseded). The tracked DROP path: ask → operator approves → mark item `Obsolete (→ Fixed.md)` with reason `feature-removed` + operator-approval citation (§11.4.90) → then remove. Classification: universal (§11.4.17). Composes §11.4.66 / §11.4.101 (removal of a live capability is exactly the high-blast-radius class §11.4.101 reserves for an operator question, never an autonomous decision) / §11.4.90 / §11.4.112 / §11.4.6 / §11.4.40 / §11.4.42. Propagation gate `CM-COVENANT-114-122-PROPAGATION` (literal `11.4.122`) + recommended gate `CM-NO-SILENT-COMPONENT-REMOVAL` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.122. Release blocker. No `--remove-without-asking` / `--silent-removal` / `--autonomous-removal-OK` / `--dedup-removal-exempt` / `--it-was-broken-anyway` escape.

§11.4.123 — Rock-solid-proof-or-deep-research mandate (User mandate, 2026-06-03)

Forensic anchor — verbatim user mandate (2026-06-03): “Every single reported issue MUST BE fully and 100% validated with rock solid proofs! Nothing can be considered fixed or completed without hard evidence! No false results or bluff(s) of any kind is allowed! If we are not sure on how to achieve full testing, validation and verification of something we MUST ALWAYS perform deep web research for all possible data (articles, documentation, guides, and other resources) and opensourced codebases which we can use to solve our problems and perform testing with validation and verification which produces rock-solid evidence(s) and leaves no space for false results or any kind of bluff!” Case study (FACT): in the 1.1.8-dev remediation the validation method for FLAG_SECURE secondary-display capture (black frames) + non-introspectable streaming-app UIs (blank accessibility tree) was unclear; deep web research (`docs/research/testing_frameworks_20260603/`) yielded the CV/OCR/liveness/sink-probe oracle stack (now §11.4.107/§11.4.112/§11.4.117), making rock-solid evidence possible — “unclear how to validate” is a research trigger, never a bluff licence.

Every reported issue / fix / claimed completion MUST be 100% validated with rock-solid CAPTURED proof (§11.4.5/§11.4.69/§11.4.107) before it is marked fixed/implemented/completed (§11.4.33). Nothing is fixed/complete without hard captured evidence — metadata-only / config-only / absence-of-error / grep-without-runtime PASS forbidden (§11.4/§11.4.1); no false results, no bluff of any kind. Operative addition: when UNSURE how to fully test/validate/verify something (no obvious evidence-producing method, OR the candidate yields only metadata/config/absence-of-error evidence), the agent MUST ALWAYS FIRST perform deep web research per §11.4.8+§11.4.99 (official docs, articles, guides, vendor refs, standards, issue trackers, reusable open-source codebases) to DISCOVER or BUILD an evidence-producing validation method that leaves no room for a false result. Declaring “untestable”/“not automatable” or accepting a metadata-only PASS WITHOUT first exhausting this deep-research path is itself a §11.4.123 violation (same severity as a PASS-bluff); the cited research (URLs + the method, OR “NO

external solution found — original work” per §11.4.8) is the proof the path was exhausted. Only after that research genuinely fails may the item be `PENDING_FORENSICS:` / `Operator-blocked` (§11.4.21)/ `structurally-impossible` won’t-fix (§11.4.112) with the cited research as earned evidence. Classification: universal (§11.4.17). Composes §11.4.5/§11.4.6/§11.4.8/§11.4.52/§11.4.69/§11.4.99/§11.4.107/§11.4.118/§11.4.21/§11.4.112. Propagation gate `CM-COVENANT-114-123-PROPAGATION` (literal `11.4.123`) + recommended gate `CM-ROCK-SOLID-PROOF-OR-RESEARCH` + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.123. Release blocker. No `--metadata-pass-suffices` / `--skip-proof` / `--untestable-without-research` / `--config-only-closure-OK` / `--bluff-when-unsure` escape.

§11.4.124 — Dead/unwired-code investigate-before-remove mandate (User mandate, 2026-06-04)

Forensic anchor — verbatim user mandate (2026-06-04): “Before removing any seemingly-dead (zero-importer / unwired) codebase, we MUST investigate via git history where/how it was originally used and how it became dead. Removal is permitted ONLY when we have captured PROOF it is genuinely no longer needed — and that removal MUST be its own separate commit with a proper descriptive message. If there is no such proof, the code MUST be investigated for where/how it should be wired in properly, and any missing or unwired tests MUST be added. We MUST ALWAYS be extra careful with any codebase removal.”

“Zero importers / never called / unwired ⇒ dead ⇒ delete” is a GUESS (§11.4.6), never a finding — a “no references” result proves only current non-reference, not genuinely-unneeded. Before removing ANY seemingly-dead element (zero-importer / never-called / unwired function / method / type / file / module / package / asset / config / build target) the agent MUST FIRST investigate via git history (`git log --follow`, `git log -S` / `-G` pickaxe, blame on the deleted call-site) and capture as FACT: (1) WHERE/HOW it was originally wired in, (2) WHEN/HOW it became dead (call-site deleted deliberately / by mistake-regression / never-completed / refactored-unreachable), (3) whether “no references” is real OR a hidden reference the static tool cannot see (reflection / dynamic dispatch / build-tags / codegen / DI / plugin registry / FFI / config-driven). Removal is permitted

ONLY with captured PROOF the element is genuinely no longer needed; that removal MUST be its OWN SEPARATE COMMIT (independently reviewable + revertible per §11.4.84/§11.4.92) with a descriptive message citing the git-history evidence (+ §11.4.122 operator-confirmation if it is an end-user capability; tracked via §11.4.90 `Obsolete (→ Fixed.md)`). With NO such proof the element MUST NOT be removed — instead investigate WHERE/HOW to wire it in properly (restore a mistakenly-deleted call-site per §11.4.114; finish never-completed wiring) AND add any missing / unwired tests (§11.4.27/§11.4.43/§11.4.115). ALWAYS be extra careful with removal: when uncertain, default to NOT removing (investigate + wire + test) per §11.4.6 + §11.4.101 + §11.4.122; “probably dead” is never sufficient — the bar is captured proof. Classification: universal (§11.4.17). Composes §11.4.6 / §11.4.8 / §11.4.84 / §11.4.90 / §11.4.92 / §11.4.101 / §11.4.114 / §11.4.122 / §11.4.27 / §11.4.43 / §11.4.115. Propagation gate `CM-COVENANT-114-124-PROPAGATION` (literal `11.4.124`) + recommended gate `CM-DEAD-CODE-INVESTIGATE-BEFORE-REMOVE` (a net-deletion commit must be removal-only + cite the git-history investigation OR be part of a tracked Obsolete item) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.124. Release blocker. No `--zero-importers-means-dead` / `--delete-unwired-on-sight` / `--skip-git-history-investigation` / `--remove-without-proof` / `--bundle-removal-with-other-work` escape.

§11.4.125 — Code-review-agent gate before pre-build + main build (mandatory multi-layer review) (User mandate, 2026-06-04)

Forensic anchor — verbatim user mandate (2026-06-04): “After all fixes/changes/implementations are done, BEFORE running pre-build tests and the main build, dispatch code-review agent(s) that analyze all work done + all existing data/facts + the existing codebase + current git history to determine quality, safety, and whether the fixes/changes will REALLY work; they MUST validate and verify that every test covering the fixes/changes genuinely validates the work with NO chance of false results or bluff of any kind. Any finding MUST be fixed, polished, improved, and covered with additional tests before the build proceeds. Multiple strong layers of checks.”

After all fixes / changes / implementations in a batch are done, and BEFORE running the pre-build test sweep AND the main (artifact) build (for ANY project), the agent MUST dispatch one or more dedicated code-review agent(s) (subagent-driven by default per §11.4.70/§11.4.20) performing a multi-layer review that: (1) analyzes ALL work done in the batch (every fix/change + its source diff + stated intent); (2) analyzes ALL existing data + facts (captured evidence per §11.4.5/§11.4.69/§11.4.107, tracker entries, prior findings, the §11.4.108 runtime-signature registry); (3) analyzes the existing codebase (blast radius per §11.4.92, cross-feature interaction, contract integrity of every dependency); (4) analyzes current git history (what each change touched, how it composes with concurrent/recent work, whether it reproduces a known-broken pattern per §11.4.114/§11.4.124); (5) determines quality + safety + will-it-REALLY-work (robust + not error-prone — no solve-A-create-B; no host/data/security regression; genuinely delivers the end-user-visible behaviour per §11.4/§107); (6) validates + verifies the tests covering the work — every covering test genuinely exercises the work-under-test and catches its negation, with ZERO chance of a false result or bluff (a test that PASSes on broken-for-the-user work, a metadata-only/config-only/absence-of-error/grep-without-runtime assertion, or a gate whose paired §1.1 mutation does not make it FAIL is a finding). Any finding (defect / error-prone change / safety risk / will-not-really-work / bluff-or-false-result-capable test / missing-coverage gap) MUST be fixed, polished, improved, and covered with additional tests (four-layer per §11.4.4(b), TDD-RED-first per §11.4.43/§11.4.115) BEFORE the pre-build sweep + main build proceed; the review iterates (re-review after each remediation) until no blocking findings remain. The review is itself anti-bluff (its conclusions are captured evidence per §11.4.5/§11.4.69; a rubber-stamp review of a defective batch = PASS-bluff). It is one of MULTIPLE STRONG LAYERS — complementing, never replacing, the §1 pre-build sweep, §11.4.92 multi-pass (author-side self-review; §11.4.125 adds the structurally-separated reviewer seam per §11.4.70), §11.4.108 four-layer fix-verification, §11.4.110 build-readiness verdict, and the post-build / runtime-on-clean-target / user-visible layers. Composes §11.4 / §11.4.1 / §11.4.4 / §11.4.6 / §11.4.40 / §11.4.43 / §11.4.50 / §11.4.70 / §11.4.20 / §11.4.92 / §11.4.102 / §11.4.107 / §11.4.108 / §11.4.110. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-125-PROPAGATION` (literal `11.4.125`) + recommended gate `CM-CODE-REVIEW-GATE-BEFORE-BUILD` (build starts only with a fresh code-review-completed marker for the current batch, produced after the last fix + before the pre-build sweep + main build) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.125. Release blocker. No `--skip-code-review` / `--build-without-review` / `--no-review-gate` / `--review-optional` / `--trust-the-author` escape.

§11.4.126 — Default autonomous-loop working mode from first prompt (User mandate, 2026-06-04)

Forensic anchor — verbatim user mandate (2026-06-04): “Make sure that you continue work in endless fully autonomous loop, do not stop until new fully validated and verified version (tag) is created and published (all submodules and main repo) or IN A CASE OF some other main stream work until it is fully completed with all side work streams and nothing else is left in our working queue! THIS MUST BE ALWAYS the default working mode without us asking you! We tend to achieve ABSOLUTE EFFICIENCY, with this and all other projects which will incorporate this MANDATORY RULE / CONSTRAINT!!! This way of (your) working will be ALWAYS applied / followed / executed / fully respected, as soon as we assign / send first request (prompt) in the session! This stops only if we explicitly say so or nothing is left to be done in current working scope (release that will come / upcoming version)!!! Any mimicking (imitation) of this behavior / rules / mandatory constraints, false results or any kind of bluff(s) is ABSOLUTELY FORBIDDEN!!!”

The endless fully-autonomous loop is the **DEFAULT working mode**, engaged automatically the moment the operator sends the FIRST request / prompt of a session — the operator MUST NOT have to ask for it, request it, restate it, or re-enable it per session. §11.4.87 framed the endless-loop covenant as an explicit-instruction opt-in; §11.4.126 is the **capstone** that promotes the same covenant to always-on: from the first prompt onward, every agent operates in the §11.4.87 loop discipline as the standing default, with §11.4.94 zero-idle, §11.4.97 maximum-idle-use, §11.4.101 autonomous-decision-over-blocking, and §11.4.103 continuous-parallel-stream all engaged by default — no per-session activation handshake. The continuation contract: the loop continues until ONE of two terminal conditions holds — (A) **Release scope** — a new, fully-validated-and-verified version (tag) is created AND published across all owned submodules AND the main repo to all configured remotes (per §2.1 + §11.4.40 + §11.4.113); OR (B) **Non-release main-stream scope** — the main-stream goal is fully completed AND every side work stream is done AND the working queue holds nothing left for the current scope. Until (A) or (B) holds, the agent MUST keep working per §11.4.42 / §11.4.72 / §11.4.94 /

§11.4.103. The loop STOPS ONLY on: (1) the operator explicitly saying so (STOP / pause / end); (2) nothing left to do in the current working scope with the queue genuinely empty per the (A)/(B) terminal conditions; (3) a §12 host-session-safety demand. Idle-while-blocked parks one work unit, it does not stop the loop (§11.4.101 + §11.4.94 + §11.4.97). Goal — ABSOLUTE EFFICIENCY; applies to this project AND every project that incorporates this Constitution. Anti-bluff: mimicking / imitating this loop behaviour, narrating continuation without performing it, fabricating progress, or emitting false / bluff results of ANY kind is ABSOLUTELY FORBIDDEN (composes §11.4 / §11.4.1 / §11.4.2 / §11.4.5 / §11.4.6 / §11.4.50 / §11.4.69 / §11.4.107); a report claiming the loop ran while no real work / no captured evidence was produced is a §11.4 PASS-bluff at the operating-mode layer. Classification: universal (§11.4.17). Composes with §11.4.87 / §11.4.94 / §11.4.97 / §11.4.101 / §11.4.103 / §11.4.66 / §11.4.6 / §11.4.40 / §11.4.42 / §11.4.72 / §11.4.113 / §2.1 / §12. Propagation gate `CM-COVENANT-114-126-PROPAGATION` (literal `11.4.126`) + paired §1.1 mutation (gate-code = separate work item).

Canonical authority: `Constitution.md` §11.4.126. Release blocker. No `--ask-before-continuing` / `--single-turn-only` / `--not-default-loop` / `--mimic-OK` escape.

§11.4.127 — Session-handoff resumption-prompt mandate (User mandate, 2026-06-06)

Forensic anchor — verbatim user mandate (2026-06-06): “make sure that in situations like this now when new session is needed you ALWAYS prepera such sentence - which will be valid for particular moment and the phase of the project and enough for work to continue.”

When a fresh session is needed (context limits / degradation) OR the operator asks whether a new session is needed / requests a handoff, the agent MUST ALWAYS prepare + proactively provide a ready-to-paste **resumption prompt valid for that EXACT moment + project phase** — self-contained for ZERO-loss resume. Two variants: SHORT first-sentence (“Read `<handoff docs>`, then continue `<goal>` …”) + FULL block on demand. MUST (1) point to live handoff docs (`.remember/remember.md` if present + `docs/CONTINUATION.md` §12.10, read FIRST + `git fetch --all`); (2) state PHASE + NEXT action + terminal goal; (3) embed exact live-state anchors (build IDs/MD5, device/target serials, commit HEAD, in-flight PIDs+log paths, captured-evidence paths); (4) restate binding constraints (anti-bluff

§11.4, no-force-push §11.4.113, exact version/naming, hardware/target gotchas); (5) be MOMENT-VALID never generic. Handoff docs current BEFORE the prompt (§12.10). Missing/stale/generic = §11.4.127 violation. Composes §12.10 / §11.4.6 / §11.4.66 / §11.4.87 / §11.4.103 / §11.4.126. Classification: universal (§11.4.17). Gate `CM-COVENANT-114-127-PROPAGATION` (literal `11.4.127`) + paired §1.1 mutation.

Canonical authority: `Constitution.md` §11.4.127. Release blocker. No `--skip-handoff-prompt` / `--generic-prompt-OK` / `--no-resumption-sentence` / `--handoff-without-state` escape.

§11.4.128 — Always-on device-recording mandate (User mandate, 2026-06-06)

Forensic anchor — direct user mandate (2026-06-06): ALWAYS live-record all available data from all test/manual-testing devices, EXTRA carefully (never harm device / performance / no side effects); raw NOT processed without need (token-conscious); raw ALWAYS git-ignored + code-intelligence-excluded; only curated evidence committed, only at release prep.

For EVERY test/debug device + every device under known manual testing, across EVERY transport (USB / wireless ADB / SSH / serial / network introspection API), ALWAYS live-record all analysable data: activities, all logs, perf metrics (CPU/mem/I/O/thermal/load), every sink-side report (§11.4.13), any live-changeable param. (1) **Side-effect-free** — non-invasive read-only probes, bounded sampling + write-volume, observer-effect budget; a perturbing recorder = §11.4.128 violation. (2) **Background + parallel + subagent-driven** (§11.4.103 + §11.4.70) — never blocks main stream. (3) **Token-conscious record-now-analyse-later** — raw NOT processed without need; only analyse-trigger = release-tag prep (§11.4.40/§11.4.42) OR explicit operator ask. (4) **Raw git-ignored (w/ §11.4.77 regen declaration) + code-intelligence-excluded (§11.4.78/.79)** — only curated evidence committed, only at release prep under `docs/qa/<run-id>/` (§11.4.83). (5) **Layout** `<recording-root>/YYYY-MM-DD/<combined main+submodules hash>/<DEVICE>_<SERIAL>/recording_NNN/<files>` . (6) **Anti-bluff** — recorder-running-but-no-growing-corpus = §11.4 bluff; curated findings trace to real raw paths; recorder health = captured evidence (§11.4.5/.69). Composes

§11.4.2/.5/.13/.69/.40/.42/.70/.77/.78/.79/.83/.103/.119. Classification: universal (§11.4.17). Gate `CM-COVENANT-114-128-PROPAGATION` (literal `11.4.128`) + rec. gate `CM-DEVICE-RECORDING-ALWAYS-ON` + paired §1.1 mutation.

Canonical authority: `Constitution.md` §11.4.128. Release blocker. No `--skip-recording` / `--record-without-layout` / `--commit-raw-corpus` / `--index-raw-corpus` / `--analyse-corpus-always` / `--invasive-probe-OK` escape.

§11.4.129 — Huge-blocker release protocol (User mandate, 2026-06-06)

Forensic anchor — direct user mandate (2026-06-06): on a huge blocker during release validation: STOP all testing → fix ALL discovered issues → process all recorded data from last session → land rock-solid fixes → author NEW validation+verification tests of ALL supported test types → rebuild → reflash → RESTART full validation+verification of every fix/change from the last release tag to now — both devices in parallel, recorded, real physical proofs, no bluff.

On a HUGE BLOCKER (release-blocking-severity: core capability broken / regression invalidating the cycle / blast radius reaching the batch's other fixes), in order, NO shortcut: (1) STOP all testing every device (§11.4.4 STOP at release granularity). (2) Fix ALL discovered issues (not just the blocker) — root-cause §11.4.102 + isolate against last known-good tag §11.4.114. (3) Process all recorded data from last session (the §11.4.128(3) release-prep analyse-trigger). (4) Land rock-solid fixes §11.4.123 + §11.4.43/.115 + §11.4.9. (5) Author NEW tests of ALL supported types §11.4.27 + §11.4.85, anti-bluff + paired §1.1 mutation. (6) Rebuild (full, not module-only) + reflash CLEAN target §11.4.108. (7) RESTART full validation from last tag to now §11.4.40 — RESTART not resume — both/all devices parallel §11.4.103/.119, recorded §11.4.128, real physical proofs §11.4.5/.69/.107, no bluff. BINDS the existing release anchors for the huge-blocker case (STOP→fix-all→process-recordings→new-tests-all-types→rebuild→reflash→full-restart + restart-not-resume), citing not duplicating. Composes §11.4.4/.40/.42/.9/.27/.85/.102/.108/.114/.115/.123/.128/.103/.119. Classification: universal (§11.4.17). Gate `CM-COVENANT-114-129-PROPAGATION` (literal `11.4.129`) + rec. gate `CM-HUGE-BLOCKER-FULL-RESTART` + paired §1.1 mutation.

Canonical authority: `Constitution.md` §11.4.129. Release blocker. No `--resume-after-blocker` / `--spot-validate-after-fix` / `--skip-recording-analysis` / `--skip-new-tests` / `--module-only-after-blocker` / `--single-device-restart` escape.

§11.4.130 — Post-remediation validate-the-fix-FIRST-after-redeploy (User mandate, 2026-06-06)

Forensic anchor — direct user mandate (2026-06-06): when a blocker found during release validation is fixed and a new artifact (rebuild / new flashing image / redeploy) is produced + the target reflashed, FIRST re-test the SPECIFIC last-failing features + validate the just-incorporated fixes BEFORE the broader / full validation.

When a blocker / critical failure found during release validation is FIXED and a new artifact is produced + the target reflashed / redistributed / updated, in order, NO shortcut: (1) re-test the SPECIFIC last-failing features FIRST (targeted guard tests for exactly the defects this fix addressed) BEFORE any broader / full-suite validation; (2) validate the just-incorporated fixes with real captured evidence — the §11.4.115 RED test flips GREEN at `RED_MODE=0` on the new artifact AND the §11.4.108 runtime-signature verifies on the CLEAN target the redeploy produced (metadata-only/config-only/absence-of-error/grep-without-runtime PASS forbidden §11.4/.1; proof §11.4.5/.69/.107/.123); (3) ONLY after the targeted fix is CONFIRMED working proceed to the §11.4.40 full retest from last tag to now. Rationale: a first fix attempt may not work / be incomplete / regress again under the new artifact — confirming FIRST catches fix-did-not-take immediately instead of hours later at the END of a full cycle (then restarting §11.4.129); cheap-confirmation-first is §11.4.82 applied to the post-blocker reflash. This is §11.4.46 specialised for the post-blocker-reflash case + the targeted-confirmation phase that GATES §11.4.129's step-7 full-restart. Honest boundary §11.4.6: "probably took" ≠ "took"; a still-FAILing targeted re-test re-enters the §11.4.114/.115 isolate→RED→fix loop, never proceeds to the full cycle on a still-broken fix. Composes §11.4.4/.40/.46/.108/.114/.115/.123/.129/.82. Classification: universal (§11.4.17). Gate `CM-COVENANT-114-130-PROPAGATION` (literal `11.4.130`) + rec. gate `CM-FIX-FIRST-AFTER-REDEPLOY` + paired §1.1 mutation.

Canonical authority: `Constitution.md` §11.4.130. Release blocker. No `--skip-targeted-retest` / `--full-cycle-first` / `--assume-fix-took` / `--validate-fix-at-end` / `--skip-red-green-flip-on-new-artifact` escape.

§11.4.131 — Standing session-resumption file mandate (User mandate, 2026-06-07)

Forensic anchor — verbatim user mandate (2026-06-07): “Make this markdown a standard file which will be written EVERY TIME when we need fresh session out of the box! It MUST BE always up to date and in sync so whenever new session is created all we have to do is just point to it!”

Every project MUST maintain a SINGLE canonical, always-current **session-resumption file** at a fixed, project-declared standard path (declared once per §11.4.35, never moved without a §11.4.66 operator decision) — the OUT-OF-THE-BOX entry point for any fresh session (creating a new session = ONLY point the new agent at this one file). §11.4.131 promotes §11.4.127 (PREPARE-on-demand) into a STANDING, version-controlled ARTIFACT, ALWAYS present + ALWAYS in sync. (A) Exists at the declared path at all times (literal path per §11.4.35, never silently moved). (B) (Re)written whenever a fresh session is needed OR the live state materially changes (new HEAD / build-artifact id / phase / device state / in-flight job / blocking decision) — the §12.10 trigger set; a stale file = §11.4.131 violation (same class as a §12.10 stale-CONTINUATION). (C) Carries SHORT + FULL §11.4.127 variants; points to `.remember/remember.md` + `docs/CONTINUATION.md` read FIRST + `git fetch`; embeds exact live-state anchors (HEAD, artifact checksums, device serials, in-flight PIDs+logs, evidence paths); states PHASE + NEXT + terminal goal; restates binding constraints (anti-bluff §11.4, no-force-push §11.4.113, exact version/naming, hardware gotchas); MOMENT-VALID never generic (§11.4.6). (D) §11.4.65 export (`.html` / `.pdf` siblings) + §11.4.44 revision header. (E) Given ONLY this file’s path a fresh session fully resumes with zero additional context. Composes §12.10/.127/.65/.44/.6/.66/.126. Classification: universal (§11.4.17). Gate `CM-COVENANT-114-131-PROPAGATION` (literal `11.4.131`) + rec. gate `CM-SESSION-RESUMPTION-FILE-PRESENT` + paired §1.1 mutation.

Canonical authority: `Constitution.md` §11.4.131. Release blocker. No `--skip-resumption-file` / `--ephemeral-prompt-only` / `--stale-resumption-OK` / `--generic-template-OK` escape.

§11.4.132 — Risk-ordered validation priority mandate (User mandate, 2026-06-07)

Forensic anchor — verbatim user mandate (2026-06-07): “We MUST ALWAYS first test and validate features, functionalities and fixes/changes that have been worked most recently, the ones which were most problematic, which have the most chance to crash or break again, the ones which have been re-opened the most times! Then, after we validate and verify all this with real (physical) proofs and hard evidence, with no false results and bluffs of any kind, we continue with all other existing tests in the test suites! This IS MANDATORY.”

Tests / validations / verifications MUST run in RISK-DESCENDING order — the highest-risk set FIRST, and ONLY AFTER that set is fully GREEN with real (physical) captured evidence does the remainder of the suite run. Closed risk factors, highest-first: (a) most-recently-worked; (b) historically most-problematic (longest defect history, most prior fixes/failures); (c) highest crash/break/regress likelihood (greatest blast radius/complexity/dependency surface); (d) most-reopened per §11.4.55 reopens-count. Each highest-risk item MUST pass with real (physical) captured evidence per §11.4.5/§11.4.69/§11.4.107 — no metadata-only / config-only / absence-of-error / grep-without-runtime PASS (§11.4/§11.4.1), no false results, no bluff (§11.4.6). ONLY AFTER the entire highest-risk set is GREEN with captured proof does the rest of the suite run; arbitrary-order or lower-risk-before-highest-risk-GREEN = §11.4.132 violation. REFINES/STRENGTHENS §11.4.130 (generalises “validate-just-fixed-first” to the full risk-ordered set) + §11.4.46 (adds explicit risk-ordering) + §11.4.42 (applies implementation-priority discipline to VALIDATION ordering). Classification: universal (§11.4.17) — consuming project supplies its recency/problematic-history/reopen-count sources (e.g. §11.4.93 DB `reopens_count` + `last_modified`) per §11.4.35. Composes §11.4.4/.5/.6/.7/.40/.42/.46/.50/.55/.69/.107/.130. Gate `CM-COVENANT-114-132-PROPAGATION` (literal `11.4.132`) + rec. gate `CM-RISK-ORDERED-VALIDATION-PRIORITY` + paired §1.1 mutation.

Canonical authority: `Constitution.md` §11.4.132. Release blocker. No `--skip-risk-ordering` / `--any-order-OK` / `--suite-order-fixed` escape.

§11.4.133 — Target-System + hardware safety mandate (User mandate, 2026-06-08)

Forensic anchor — verbatim user mandate (2026-06-08): “Make sure that all changes we do to the System are ALWAYS safe for the System itself and for the hardware the system runs on! This is MANDATORY.”

Every change to the TARGET system (firmware, kernel, init/boot scripts, drivers, sysfs/devfreq/voltage/clock/thermal/regulator register writes, partition/bootloader/U-Boot, HAL, framework, prebuilts, device config) MUST ALWAYS be safe for BOTH (a) the target System itself — MUST NOT brick, boot-loop, corrupt data, or render the device unrecoverable — AND (b) the hardware it runs on — MUST NOT exceed safe electrical/thermal/voltage/clock limits or damage panels/storage/radios/regulators. Concrete obligations: (1) reversible-first — verify irreversible high-blast-radius changes (bootloader/U-Boot MD5, partition layout) against known-good values + capture a pre-op backup (§9.2) BEFORE applying; (2) NO unverified hardware-control writes — never write an unverified value to a voltage/clock/regulator/thermal-throttle/current-limit sysfs node or register that could exceed datasheet limits, the safe range established as FACT (§11.4.6), never guessed; (3) thermal/perf changes (forcing a performance governor, pinning the top OPP, disabling thermal management) MUST respect the device’s cooling design, validated by captured thermal evidence; (4) flashing MUST use the sanctioned tool + a freshly-built integrity-verified image — never an ad-hoc partition write or stale/unverified artifact; (5) unprovable-safety ⇒ blocked — a change whose target/hardware safety cannot be established from captured evidence is treated as UNSAFE and blocked (§11.4.6 + §11.4.101 reversible-first + §11.4.123 rock-solid-proof). DISTINCT from §12 host-session safety: §12 protects the DEVELOPER’s HOST + session; §11.4.133 protects the TARGET device + its hardware — both apply, neither weakens the other. Classification: universal (§11.4.17) — consuming project supplies its concrete hardware-control surfaces, datasheet-safe ranges, known-good bootloader/image hashes, and sanctioned flashing tool per §11.4.35. Composes §12 / §11.4.6 / §11.4.101 / §11.4.108 / §11.4.123. Gate `CM-COVENANT-114-133-PROPAGATION` (literal `11.4.133`) + rec. gate `CM-TARGET-HARDWARE-SAFETY` + paired §1.1 mutation.

Canonical authority: `Constitution.md` §11.4.133. Release blocker. No `--unsafe-hardware-write` / `--skip-system-safety` / `--brick-risk-accepted` escape.

§11.4.134 — Code-review iterate-until-GO + rock-solid-evidence mandate (User mandate, 2026-06-08)

Forensic anchor — verbatim user mandate (2026-06-08): “For any fixes/changes given back to us for re-work by the code-review process, once we fix/improve everything per the code-review’s requests, we MUST RE-RUN code-review AGAIN until we get a GO from it with NO new issues reported or warnings of any kind! All results produced by this whole process MUST ALWAYS give us rock-solid PHYSICAL evidence that the fixed/improved codebase really works now as expected, with no false results and no bluff(s) of any kind.”

When the §11.4.125 code-review returns ANY finding (BLOCKING, nit, or warning) and the author fixes/improves per that review, the code review MUST BE RE-RUN — and KEEP being re-run after each remediation round — until a clean GO with ZERO new issues AND ZERO warnings. A single “addressed the findings” pass is NOT sufficient: the corrected batch MUST pass a FRESH adversarial review (a re-review can surface NEW findings introduced by the fixes — the §11.4.1 fix-A-creates-B mode). The loop terminates ONLY on a clean GO; a residual warning is itself a finding that re-arms the loop. Every round’s verdict AND every fix’s validation MUST carry rock-solid PHYSICAL captured evidence per §11.4.5 / §11.4.69 / §11.4.107 (captured audio / video / sysfs / dumpsys / sink-side / runtime-signature) proving the fixed codebase REALLY works — never metadata-only / config-only / absence-of-error / grep-without-runtime; no false results, no bluff; a GO unbacked by physical evidence is a §11.4 PASS-bluff at the review-loop layer. REFINES / STRENGTHENS §11.4.125 (iterate-until-no-blocking): makes the loop EXPLICIT, raises termination to ZERO findings AND ZERO warnings, BINDS physical evidence to every round. Classification: universal (§11.4.17). Composes §11.4.125 / §11.4.1 / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.69 / §11.4.107 / §11.4.50 / §11.4.108 / §11.4.123. Gate `CM-COVENANT-114-134-PROPAGATION` (literal `11.4.134`) + rec. gate `CM-CODE-REVIEW-ITERATE-UNTIL-GO` + paired §1.1 mutation.

Canonical authority: `Constitution.md` §11.4.134. Release blocker. No `--skip-rereview` / `--single-review-pass` / `--warnings-ok` / `--evidence-optional` escape.

§11.4.135 — Standing regression-guard suite + every-fixed-defect-gets-a-permanent-regression-test (User mandate, 2026-06-08). Every project MUST maintain a STANDING regression-guard suite that runs on EVERY build+deploy and BLOCKS the release tag on any failure. Every closed defect (stable ticket id, e.g. ATM-NNN) MUST, in the SAME commit as its fix (extending the §11.4.43 DOCUMENT step), register a permanent §11.4.115 RED-on-broken-artifact regression test into the suite — `RED_MODE=1` capturing the historical defect on a pre-fix artifact (the proof the guard is real), `RED_MODE=0` the standing GREEN guard asserting the defect is ABSENT. A closure without a registered guard is a §11.4.123 violation. The suite runs FIRST in the post-deploy cycle (highest-risk set per §11.4.132) and is a §11.4.40 release-gate blocker. Forensic anchor (FACT): the wrong-subtitle-on-2nd-display defect was “fixed” via a source-side `CONTROL_MENU_LABEL_DENYLIST` that NO test mirrored or re-ran, so the NEXT chrome class recurred silently while the GREEN suite passed. Industry-standard bug-driven testing (Google content-driven testing; AOSP CTS/Tradefed) made mechanical + enforced. Composes §11.4.4 / §11.4.40 / §11.4.43 / §11.4.46 / §11.4.50 / §11.4.107 / §11.4.108 / §11.4.115 / §11.4.118 / §11.4.123 / §11.4.124 / §11.4.130 / §11.4.132. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-135-PROPAGATION` (literal `11.4.135`) + recommended gates `CM-REGRESSION-GUARD-REGISTERED` / `CM-REGRESSION-GUARD-SUITE-WIRED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.135. Non-compliance is a release blocker. No escape hatch — no `--skip-regression-guard`, `--no-guard-on-close`, `--guard-optional` flag.

§11.4.136 — Real-content end-to-end playback-test mandate (User mandate, 2026-06-08). Refines/strengthens §11.4.107. Any test asserting media playback works MUST drive REAL content (catalog stream or offline reference clip) through the user’s path (§11.4.48 UI-driven → §11.4.117 CV/OCR fallback) and assert it genuinely PLAYS via the §11.4.107 liveness battery PLUS a decoder-health census — a numeric drop-buffer budget, no buffer-timestamp re-order/discard, no codec-reject (cite Android/Media3 ExoPlayer OEM pre-OTA playback-test mandate: “too many dropped buffers” >25, “unexpected presentation timestamp”, “test timed out”). Metadata-only / launch-only / registration-only / single-frame / config-only PASS is forbidden (§11.4 / §11.4.1). A golden/reference clip corpus (BBC ExoPlayer testing

samples) is the offline ground-truth. Composes §11.4.5 / §11.4.48 / §11.4.50 / §11.4.107 / §11.4.117 / §11.4.123 / §11.4.13 / §11.4.69. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-136-PROPAGATION` (literal `11.4.136`) + recommended gate `CM-REAL-CONTENT-PLAYBACK-TEST` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.136. Non-compliance is a release blocker. No escape hatch — no `--launch-proves-playback`, `--skip-decoder-health`, `--metadata-playback-pass-suffices` flag.

§11.4.137 — Subtitle/caption content-correctness oracle + secure-display-proxy-honesty mandate (User mandate, 2026-06-08). Refines §11.4.117 + §11.4.107 + §11.4.112. Forensic anchor (FACT): tests tasked to “physically verify the 2nd-display subtitle” PASSed GREEN while subtitles did NOT show / showed WRONG — the streaming player surface is FLAG_SECURE so `screencap -d <secondary>` returns BLACK (autonomous PIXEL verification structurally impossible per §11.4.112), so the test fell back to the accessibility-scraped/ `persist.atmosphere.subdebug` proxy, and the proxy accepted a chrome/menu LABEL (`Аудио и субтитры`) as a valid subtitle because the prose floor accepted any multibyte prose and NO menu-label denylist + NO position/cadence check existed. The mandate: a subtitle-correctness test MUST classify the cue’s *content class* — a present cue is NOT a correct cue. CHROME (FAIL) if a known control/ menu label (closed multilingual deny-list MIRRORED from source, case-folded incl. non-ASCII), time/numeric chrome, not prose, OUTSIDE the lower safe-title band (CEA-708 9-anchor grid), OR STATIC across the window (real subtitle changes → ≥2 distinct prose cues, a metamorphic relation). DIALOGUE (PASS) only when prose + not-denied + not-chrome + position-ok + cadence ≥2 OR fuzzy-matches the SOURCE-extracted cue via normalized edit distance (§11.4.123 host ground truth). The oracle MUST be self-validated golden-good/golden-bad (§11.4.107(10)) and the deny-list MUST be verified present in the SHIPPED artifact (§11.4.108) — a source-green denylist with no test mirror + no artifact check is the exact recurrence pattern forbidden here. Secure-display honesty (§11.4.112): where FLAG_SECURE makes pixel verification impossible, the rock-solid autonomous proof is the player’s caption telemetry + source-track presence + content-class oracle — NEVER a faked pixel “physical” pass; human-eye pixel confirmation is `operator_attended` (§11.4.52) with a tracked migration item. App-agnostic (keys off content class). Composes §11.4.3 / §11.4.5 / §11.4.6 / §11.4.107 / §11.4.108 / §11.4.112 / §11.4.115 / §11.4.117 / §11.4.123 / §11.4.13 / §11.4.69. Classification:

universal (§11.4.17). Propagation gate `CM-COVENANT-114-137-PROPAGATION` (literal `11.4.137`) + recommended gate `CM-SUBTITLE-CONTENT-CORRECTNESS-ORACLE` + paired §1.1 mutation (strip the denylist/position/cadence check → golden-bad `Аудио и субтитры` PASSes → gate FAILs). **Canonical authority:** constitution submodule `Constitution.md` §11.4.137. Non-compliance is a release blocker. No escape hatch — no `--present-cue-is-correct`, `--skip-chrome-oracle`, `--length-heuristic-suffices`, `--pixel-pass-on-secure-display`, `--skip-position-check`, `--skip-cadence-check` flag.

§11.4.138 — Operator-escape => mandatory bluff-audit + permanent guard (User mandate, 2026-06-08). When the operator (or any out-of-band channel) finds a defect that the GREEN test suite passed, this is by definition a §11.4 PASS-bluff — it MUST trigger, before the fix is closed: (1) a §11.4.102 systematic-debugging pass to FACT-root-cause; (2) a bluff-audit identifying the EXACT assertion that should have caught it but didn't, cited to `file:line` (canonical example: `lib/subtitle_content_validation.sh:sub_is_prose()` returning TRUE for `Аудио и субтитры`); (3) a permanent §11.4.135 regression guard registered in the SAME commit as the fix, with its §11.4.115 RED capturing the operator-found defect; (4) the bluff-audit committed under `docs/research/<scope>/<defect>_bluff_audit/`. Closing an operator-found defect WITHOUT the bluff-audit + permanent guard is itself a §11.4 violation (the bluff that let it through is still live and the defect will recur). Composes §11.4 / §11.4.1 / §11.4.102 / §11.4.108 / §11.4.115 / §11.4.118 / §11.4.123 / §11.4.135. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-138-PROPAGATION` (literal `11.4.138`) + recommended gate `CM-OPERATOR-ESCAPE-BLUFF-AUDIT` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.138. Non-compliance is a release blocker. No escape hatch — no `--close-without-bluff-audit`, `--operator-find-is-just-a-bug`, `--skip-permanent-guard` flag.

§11.4.139 — Fresh-process clean-artifact runtime-signature mandate (User mandate, 2026-06-08). Refines §11.4.108. Before any post-deploy validation — ESPECIALLY a non-pixel proxy verification (the subdebug/accessibility-cue channel used for FLAG_SECURE displays) — the harness MUST assert running-artifact == built-artifact: the deploy yielded a CLEAN target (mutable-overlay/userdata wiped) OR a pre-validation check proves no stale overlay shadows the deployed code (e.g. every guarded package — incl. the Presenter that emits the subtitle cue — resolves to the system partition, no per-user override). A stale shadow of the cue-

emitting component (e.g. a Presenter APK predating the denylist) makes the proxy report on code that was never deployed — any PASS is a §11.4 PASS-bluff. Each fix declares ONE machine-checkable runtime signature verified on the clean target (the §11.4.108 registry IS the definition of done); for the subtitle class the signature is “the shipped Presenter APK contains the denylist literal (case-insensitive) AND the subdebug channel emits `candidate REJECTED reason=chrome-label` for a menu label.” Composes §11.4.46 / §11.4.108 / §11.4.130 / §11.4.135 / §11.4.137. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-139-PROPAGATION` (literal `11.4.139`) + recommended gate `CM-CLEAN-ARTIFACT-RUNTIME-SIGNATURE` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.139. Non-compliance is a release blocker. No escape hatch — no `--validate-against-running-state`, `--skip-clean-precondition`, `--shadow-OK` flag.

§11.4.140 — Universal action-prefix system (`ACTION_NAME ::`) (User mandate, 2026-06-09; GRAMMAR_ADDENDUM 2026-06-09). When a user prompt's FIRST non-blank line starts with a recognised action prefix, you MUST: (1) look the action token up in the action registry `constitution/actions/registry.yaml` (or `$HELIX_ACTION_REGISTRY`); (2) if it is a registered action, REPLACE the prefix with that action's `expansion` text and apply its `rules` ; (3) execute the remainder of the prompt under the expanded instruction. **Four EQUIVALENT forms** — same action, same expansion, same execution: (1) `ACTION_NAME :: <rest>` (bare `::`), (2) `PREFIX::ACTION_NAME :: <rest>` (namespaced `::`), (3) `/ACTION_NAME <rest>` (bare slash), (4) `/PREFIX::ACTION_NAME <rest>` (namespaced slash). Thus `BACKGROUND :: x` ≡ `DEFAULT::BACKGROUND :: x` ≡ `/BACKGROUND x` ≡ `/DEFAULT::BACKGROUND x` . `PREFIX` is an action NAMESPACE; the reserved default namespace is **`DEFAULT`** , and an action runs WITH or WITHOUT the prefix. Grammar (all hold): anchored at the FIRST non-blank line only (mid-prose tokens never match); the action token AND the namespace are UPPERCASE-only `[A-Z][A-Z0-9_]*` (lowercase never matches); the namespace separator `::` inside the token carries NO surrounding spaces (`PREFIX::ACTION_NAME`), DISTINCT from the action-body separator `" :: "` (one ASCII space on each side of `::` — avoids C++ `Foo::Bar` , YAML `key: value` , URLs) in forms 1/2 and the slash-body separator (one space) in forms 3/4; stacked prefixes (`A :: B :: rest`) apply outer-to-inner, left-to-right (expand `A` , re-scan, expand `B` , then the residual is the task); a leading `\` escapes the prefix for BOTH the `::` and the slash form (`\BACKGROUND :: x` , `\/BACKGROUND x` — treat literally, strip the backslash, NO expansion) so action names can be discussed. **Conflict rule (slash form):** `/ACTION_NAME` (form 3) is honored as the action ONLY when `ACTION_NAME` (case-folded) does not collide with a built-in/host slash command (registry `slash_bare: auto` + `slash_conflicts: [..]`); form 4 (`/PREFIX::ACTION_NAME`) is ALWAYS unambiguous and always honored. An unknown token that matches the grammar shape (any of the 4 forms) but is NOT registered is NEVER silently expanded or silently dropped — ask which registered action was meant (§11.4.66 / §11.4.105) or treat it as a literal prompt, NEVER invent an expansion (§11.4.6); any prompt not satisfying the grammar is an ordinary prompt and the system is a no-op. The registered

> action `BACKGROUND` expands to: *"The following prompt that we will provide MUST BE executed in background in parallel with all main work streams using the subagents-driven development approach! All work done MUST PRODUCE rock solid evidence covered with hard physical proof(s) that all done is working as expected and as specified without any false results and without any bluff!"* (composes §11.4.20 / §11.4.70 subagent-driven, §11.4.58 / §11.4.103 parallel streams, §11.4.89 background execution, §11.4.5 / §11.4.69 / §11.4.107 captured physical evidence, §11.4 anti-bluff). The system is UNIVERSAL (every CLI agent reads this block via its context carrier per §11.4.35), extensible (new action = new registry row), decoupled + reusable (§11.4.28), and loads out-of-the-box. Classification: universal (§11.4.17). **Canonical authority:** constitution submodule `Constitution.md` §11.4.140. Non-compliance is a release blocker. No escape hatch — no `--skip-action-prefix`, `--ignore-prefix`, `--no-registry`, `--invent-expansion-OK`, `--single-layer-only` flag.

§11.4.141 — Token-efficiency mandate (research-derived + operator mandate, 2026-06-09). Every project worked on by AI coding agents MUST cut token spend (input AND output) toward **30–40% of current (a 60–70% reduction)** WITHOUT degrading quality/performance/safety or breaking any existing mechanism, via a composable, safety-ranked measure set: (1) **prompt-cache the static governance prefix** — the always-loaded governance forms a byte-stable cache-breakpointed prefix with no volatile bytes ahead of it; cache reads cost ~0.1× base input (the dominant cost driver — measured ~170K tokens of governance re-sent every turn, externally corroborated by Claude Code issue #24147); caching is transparent so it removes no rule, weakens no gate, changes no verdict — only billing (PRIMARY, biggest + safest lever); (2) **subagent model-tiering + output-to-file** — mechanical non-judgment work (search/grep/status/doc-export/read-only probes) to a Haiku-class model, the strong model RESERVED for all reasoning/verdicts/fix-design (§11.4.102)/code-review (§11.4.125)/demotion (§11.4.7), large output persisted to a file not an inline 350–520K-token transcript; the cheap model never emits a PASS so §11.4.50 + anti-bluff are untouched; (3) **thin always-loaded INDEX + on-demand detail** — concise index (one line per fix/anchor, EACH carrying the literal `11.4.N` token so propagation gates pass) with the canonical full text kept gate-scanned in `constitution/Constitution.md` and reachable in one hop — a de-duplication realising §11.4.35, never a deletion; (4) **CodeGraph/retrieval-first over**

full-file loading (§11.4.78/§11.4.79); (5) **output-token reduction** — terse status + `effort:"low"` on the mechanical allowlist only; (6) **tool-call batching + no re-reads**; (7) **compaction/context-editing for long sessions**. **Mandatory measured proof:** a token-accounting harness measures tokens-per-development-cycle BEFORE vs AFTER on a frozen deterministic workload from the authoritative `usage` object (input/cache_read/cache_creation/output split; NEVER `tiktoken`, NEVER the client-side cost estimate), reproduced N times (§11.4.50), pass = AFTER ≤ 40% of BEFORE OR the measured best-safe reduction with a cited cold-cache reason; the AFTER run MUST show ZERO regression on the pre-build sweep + meta-test mutation sweep + propagation gates + a strong-model reasoning probe + a cache-warm proof (`cache_read_input_tokens > 0`) — cost reduction with quality regression is a §11.4 FAIL. The headline number is the *measured* reduction, never the design estimate (§11.4.6/§11.4.123). No measure may break/degrade any existing mechanism, and the rule is structured so none can. Composes §11.4.5/.6/.20/.40/.50/.58/.69/.70/.78/.79/.80/.103/.106/.123/.125/.128/§12.6/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-141-PROPAGATION` (literal `11.4.141`) + recommended gate `CM-TOKEN-EFFICIENCY` + paired §1.1 mutation (inject a pre-breakpoint volatile token → cache collapses → measured reduction falls below bar → gate FAILs). **Canonical authority:** constitution submodule `Constitution.md` §11.4.141. **Project instantiation (§11.4.35):** [the consuming project fills in: confirm the Claude Code governance-prefix cache is warm + free of pre-breakpoint volatiles; tier the §11.4.96-SAFE mechanical subagents (search/grep/status/doc-export) to Haiku with output-to-file under `qa-results/` ; replace the consumer `CLAUDE.md` 112-row Applied-Fixes inline table + verbatim anchor restatements with a literal-anchor index pointing at `constitution/Constitution.md` ; keep CodeGraph/lumen-first navigation; run the harness BEFORE/AFTER to prove the measured reduction.] Non-compliance is a release blocker. No escape hatch — no `--skip-token-efficiency` , `--no-cache-governance` , `--assert-reduction-without-measuring` , `--tier-down-reasoning` , `--inline-all-governance` , `--tiktoken-estimate-OK` flag.

§11.4.142 — Universal code-review mandate — every change reviewed, always, no exception (User mandate, 2026-06-09). Verbatim operator mandate: “ALL changes we do MUST pass through the code review step!!! ALWAYS!!!” EVERY change made to ANY repository this Constitution governs — without exception — MUST pass through an INDEPENDENT code-review step BEFORE it

is accepted, committed, or built. NO change class is exempt: source code, fixes, tests, gates, meta-test mutations, documentation, doc-tooling, build/CI scripts, configuration, governance files (Constitution / CLAUDE / AGENTS / QWEN), conductor main-stream edits, sub-agent output, refactors, single-line edits — if a diff exists, it gets an independent review. This is the ABSOLUTE form of §11.4.125 (code-review agent gate after a batch, before pre-build sweep + main build): §11.4.142 strips every implicit scoping §11.4.125's "after all fixes/changes/ implementations are done" phrasing could leave open — no "just a doc edit", no "just a one-liner", no "the author already self-reviewed (§11.4.92)", no "trivial change" carve-out. **Independence is load-bearing** — the reviewer MUST be structurally separated from the author (a dedicated code-review agent, subagent-driven by default per §11.4.70 / §11.4.20, or a distinct human), NEVER the author re-reading their own work; §11.4.92's multi-pass self-evaluation is the AUTHOR-side discipline and PRECEDES (never satisfies) §11.4.142. **The review is itself anti-bluff** (§11.4 / §11.4.1) — a rubber-stamp "looks good" is not a review; it MUST genuinely analyse correctness, safety (no host §12 / data §9 / target-hardware §11.4.133 regression), will-it-really-work (no solve-A-create-B), end-user behaviour (§11.4 / §107), and test-genuineness (§1.1), its conclusions captured evidence per §11.4.5 / §11.4.69, and **it iterates to a clean GO per §11.4.134** — any finding (BLOCKING / nit / warning) re-arms the loop, acceptance only on ZERO new findings + ZERO warnings with rock-solid physical evidence. Honest boundary (§11.4.6): a passing review does NOT replace §11.4.108 four-layer runtime-signature verification nor the §11.4.40 full-suite retest — it is one of MULTIPLE STRONG LAYERS, and the FIRST one every change crosses. Composes §11.4.1 / §11.4.4 / §11.4.5 / §11.4.6 / §11.4.20 / §11.4.40 / §11.4.69 / §11.4.70 / §11.4.92 / §11.4.108 / §11.4.110 / §11.4.125 / §11.4.134 / §107 / §1.1. Classification: universal (§11.4.17) — the consuming project supplies its reviewer-dispatch mechanism + change-acceptance seam (commit wrapper / merge queue / PR gate) per §11.4.35. Propagation gate `CM-COVENANT-114-142-PROPAGATION` (literal `11.4.142`) + recommended gate `CM-EVERY-CHANGE-REVIEWED` (every accepted change carries a fresh independent-review-completed marker for its diff, produced by a reviewer distinct from the author, before acceptance/commit/build) + paired §1.1 mutation (strip the literal → propagation gate FAILs; accept a diff with no independent-review marker → `CM-EVERY-CHANGE-REVIEWED` FAILs). **Canonical authority:** constitution submodule `Constitution.md` §11.4.142. Non-compliance is a release blocker.

No escape hatch — no `--skip-review`, `--no-review`, `--trivial-change-exempt`, `--doc-edit-exempt`, `--self-review-suffices`, `--review-after-commit` flag.

§11.4.143 — Real-user-journey mandate for video-streaming-app full-automation tests (User mandate, 2026-06-10). Verbatim operator mandate: “All video streaming apps … require to choose some title and to press proper UI button to start or resume playing! Proper UI interaction to play exact show with proper content and subtitles is MANDATORY! Without it we just eventually play on 2nd display sample, and that’s it mostly! THIS MUST BE ADDED as MANDATORY RULE regarding testing of any video streaming app in general with full automation tests! Universal in root constitution + a consuming project’s extensions.” Any full-automation test that asserts a video player / streaming application plays content MUST drive the REAL end-user journey through the app’s OWN UI — launch → BROWSE the actual catalog → choose a SPECIFIC title → press the real Play/Resume button → confirm THAT chosen content is genuinely playing, with its correct subtitles, on the intended routing target. A test that bypasses the journey with a sample / demo / built-in-loop clip, a deep-link / `am start -a VIEW` / intent shortcut, a synthetic or pre-staged stream, or any path that does NOT exercise the app’s own browse-select-play UI is a §11.4 PASS-bluff at the user-journey layer: it validates ROUTING (that *something* reaches the display) while leaving the user-visible behaviour the operator cares about — “the show I picked actually plays” — unproven (the operator’s “we just eventually play on 2nd display sample, and that’s it mostly” gap). The mandate (ALL hold): (1) **Real journey, not a shortcut** — launch → catalog browse → specific-title selection → real Play/Resume press through the app’s own UI (§11.4.48 UI-driven), NEVER a deep-link / intent / sample / loop-clip shortcut; (2) **Chosen-content confirmation** — the PASS proves the SPECIFIC selected title is playing (not merely that pixels move on the target) via the §11.4.107 liveness battery on the §11.4.136 real-content path + the §11.4.137 subtitle content-correctness oracle for the chosen title’s captions; (3) **Non-introspectable UIs use the pixel oracle** — when the app’s accessibility hierarchy is blank/unreliable (TV-Compose / leanback / canvas / GL), DRIVE input + ASSERT content via the §11.4.117 CV/OCR pixel oracle, never a hierarchy-only tool; (4) **Login via the credential single-source** (§11.4.10), never hardcoded, never logged; (5) **Honest SKIP, never a faked PASS** — where the autonomous journey is genuinely infeasible (hard human-only login / CAPTCHA, geo-block per §11.4.3, secure-surface pixel-blanking per §11.4.112) the test is

`operator_attended` SKIP-with-reason per §11.4.52 + §11.4.3 with a tracked migration item — NEVER a metadata-only / sample-played / routing-only PASS. Honest boundary (§11.4.6): “the routing fired so the title is playing” is a guess — only the chosen-content liveness + subtitle oracle on the real journey proves it. Composes §11.4.48 / §11.4.107 / §11.4.117 / §11.4.136 / §11.4.137 / §11.4.52 / §11.4.3 / §107 / §1.1. Classification: universal (§11.4.17) — the consuming project supplies its concrete app roster, login-credential source, routing target, and UI-driving / pixel-oracle harness per §11.4.35. Propagation gate `CM-COVENANT-114-143-PROPAGATION` (literal `11.4.143`) + recommended gate `CM-VIDEO-REAL-JOURNEY-TEST` (every video-streaming-app playback test drives the real browse-select-play journey + confirms the chosen content + subtitles, or SKIPs-with-reason) + paired §1.1 mutation (replace a real-journey test’s browse-select-play path with a deep-link / sample shortcut → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.143. Non-compliance is a release blocker. No escape hatch — no `--sample-playback-ok`, `--skip-real-journey`, `--deep-link-suffices`, `--routing-only-pass`, `--no-title-selection` flag.

§11.4.144 — Tracked/recorded-device availability-following mandate (User mandate, 2026-06-10). Direct operator mandate: every device the project is tracking / following / recording (test / debug / manual-testing device, across every reachable transport — USB / wireless ADB / SSH / serial / network introspection API) MUST be availability-FOLLOWED — its connection state continuously monitored, any drop handled, never silently abandoned and never presented as a continuous recording. The §11.4.128 always-on recorder KNOWS a tracked device is absent (its per-device loop guards on the reachable state) but, lacking a following discipline, merely spins idle — no data captured, no offline event logged, no resume, no escalation — so the recording corpus presents a continuous timeline with a silent hole, a §11.4 PASS-bluff at the recording-integrity layer (it claims continuous capture while no data exists for the gap). On a tracked device leaving its reachable state the system MUST, automatically: (a) **DETECT** the drop and **log an honest offline event** into the recording corpus (§11.4.6 — a silent gap presented as continuous capture is a fabricated-continuity bluff; §11.4.128 — a silent recording gap defeats always-on recording); (b) **WAIT** for the device to return using the project’s ALREADY-DEFINED reconnection timings — never invented numbers (§11.4.6), the SAME grace / reconnect / poll budgets the project’s recovery path

already uses; (c) **RE-ATTACH** — resume recording / tracking the moment the device returns and log an honest online / resume event; (d) **ESCALATE** to the project's sanctioned device-recovery path (per §11.4.69 feature class `device_recovery`) if the device does not return within the defined timeout — through the sanctioned, authorization-gated recovery entry point ONLY, never bypassing its gate and never performing a destructive recovery (e.g. a power-cycle) autonomously without that authorization (§11.4.21 / §11.4.101 — high-blast-radius recovery is gated; while blocked the system keeps following the device, logging the blocked-escalation honestly). A tracked-device drop that produces a silent corpus hole, a never-resumed recording, or a never-escalated permanent absence is the bluff this anchor forbids. Composes §11.4.128 (always-on recording — §11.4.144 closes its drop-handling gap) / §11.4.69 (`device_recovery` sink-side positive evidence) / §11.4.6 (honest offline / online events, reused-not-invented timings) / §11.4.14 (watchdog children reaped on stop) / §11.4.21 + §11.4.101 (gated, non-autonomous escalation). Classification: universal (§11.4.17) — the consuming project supplies its concrete tracking transport, device set, already-defined reconnection timings, and sanctioned recovery entry point per §11.4.35. Propagation gate `CM-COVENANT-114-144-PROPAGATION` (literal `11.4.144`) + recommended gate `CM-DEVICE-AVAILABILITY-FOLLOWED` + paired §1.1 meta-test mutation (strip the watchdog wiring / let an absence go unlogged → gate FAILs; strip the literal → propagation gate FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.144. Non-compliance is a release blocker. No escape hatch — no `--skip-availability-following`, `--silent-recording-gap-OK`, `--no-reconnect-wait`, `--invent-reconnect-timing`, `--skip-recovery-escalation`, `--autonomous-power-cycle-OK` flag.

§11.4.145 — Independent multi-angle impact-research per change (User mandate, 2026-06-10). For EVERY fix / change / new feature, INDEPENDENT impact-research agents (subagent-driven §11.4.70/§11.4.20, structurally separate from the author — §11.4.92 self-eval PRECEDES, never satisfies — adversarial "refute-the-change" stance to defeat the documented LLM confirmation-bias failure mode) MUST research the change AND its connected/dependent features across a CLOSED SET OF EIGHT ANGLES — (1) correctness/logic, (2) regression (what existing working feature could break — via call-graph impact enumerating every direct + transitive caller and contract-dependent feature, each mapped to its test), (3) latent/dangerous-code — the recents class (runtime-only failure, interface/ABI/

contract mismatch, concurrency/data-race, resource lifecycle), (4) security (taint/injection/secret/unsafe-API), (5) performance (latency/memory/thermal under load vs baseline), (6) host/data/target-hardware safety per §12/§9/§11.4.133, (7) cross-feature interaction (shared state/timing/hardware contention), (8) business-logic conformance (matches the spec AND the connected features' contracts, not merely compiles). Forensic anchor (FACT, project-internal): the recents / 3-button-nav path shipped a latent AIDL-interface-contract mismatch that PASSED code review yet failed at RUNTIME ONLY (ANR on a recents-reaping gesture) because no review angle interrogated the interface contract / concurrency-lifecycle / silently-broken connected feature — the §11.4.108 SOURCE→ARTIFACT→RUNTIME→USER-VISIBLE gap caught only after the fact; §11.4.145 shifts that discovery LEFT. Each angle MUST name the TOOL(S) used + obtain the required DEPTH (the diff, the full changed unit, its declared contract, the spec section, every connected feature — NEVER the diff lines alone) + emit a captured-evidence conclusion per §11.4.5/§11.4.69 ("looks fine" forbidden, §11.4.1); a genuinely-N/A angle is recorded `NOT_APPLICABLE: <reason>` per §11.4.6, never silently skipped. Output = a per-change impact-research REPORT (one section per angle: tool + evidence path + verdict) + a single GO/NO-GO that BLOCKS acceptance/commit/build on ANY unmitigated risk in ANY angle; the change is fixed/mitigated + affected angles re-researched, iterating to a CLEAN GO (zero unmitigated risk + zero warning) per §11.4.134 before the §11.4.125 review gate. Honest boundary (§11.4.6): a GO proves cross-angle internal-consistency + bounded blast-radius — it does NOT replace §11.4.108 runtime-signature verification nor §11.4.40 full-suite retest; it is one of MULTIPLE STRONG LAYERS, the FIRST research pass every change crosses. Composes/strengthens §11.4.92/.125/.108/.110/.134/.78/.79/.107/.85/.133/§12/§9/.99/.6/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-145-PROPAGATION` (literal `11.4.145`) + recommended gate `CM-IMPACT-RESEARCH-PER-CHANGE` + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; accept a change with a report missing an angle or an unmitigated NO-GO → `CM-IMPACT-RESEARCH-PER-CHANGE` FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.145. Non-compliance is a release blocker. No escape hatch — no `--skip-impact-research`, `--single-angle-suffices`, `--author-researches-own-change`, `--diff-skim-OK`, `--no-go-overridable`, `--trivial-change-no-research` flag.

§11.4.146 — Reproduce-first test + same-test-confirms-fix + mandatory extend-to-all-cases workflow (User mandate, 2026-06-10). Every reported problem MUST be handled by a NAMED three-step test workflow — §11.4.146 does NOT re-author its component disciplines, it NAMES + BINDS them into the operator's exact per-defect sequence and adds ONLY the two emphases the components leave implicit. **(STEP 1 — REPRODUCE-FIRST + INVESTIGATE)** before any fix, author the §11.4.43 RED test as a §11.4.115 RED-baseline-on-the-broken-artifact (reproduce the defect on the CURRENT pre-fix artifact, capture defect-present physical evidence per §11.4.5/§11.4.69/§11.4.107); NEW EMPHASIS (D1) that same RED test is ALSO a deliberate investigation instrument per §11.4.102(A) — it MUST gather ADDITIONAL forensic data characterising the defect (triggers, boundaries, input/topology scope, adjacent failure modes) that feeds BOTH the fix design AND the STEP-3 extend scope; a RED test used only as a binary present/absent gate with no characterisation satisfies §11.4.115 but NOT §11.4.146 STEP 1. **(STEP 2 — SAME-TEST-CONFIRMS-FIX)** after the fix the SAME test source (the §11.4.115 polarity switch flipped `RED_MODE=1→0`) confirms the defect ABSENT — RED-on-broken then GREEN-on-fixed, both captured, on a clean target per §11.4.108/§11.4.139, validated-first-after-redeploy per §11.4.130, deterministic per §11.4.50; no separate happy-path test substitutes (the §11.4.43/§11.4.115 PASS-bluff). **(STEP 3 — EXTEND-TO-ALL-CASES, mandatory per-fix)** NEW EMPHASIS (D2) immediately after STEP 2 the test MUST be FANNED OUT across the full case-space of the SAME functionality — all flows, valid + invalid + boundary (§11.4.85 empty/max/off-by-one) + concurrent (§11.4.85 contention) + failure-injection (§11.4.85 chaos) + topology variants (§11.4.3) — confirming no issue of any kind, with PROVABLE enumerated coverage per §11.4.118 (a listed case-set with per-case outcome, never "no other issues found"); a REQUIRED step, NOT deferred to a release-cycle discovery sweep and NOT reduced to a single guard; each user-visible case carries rock-solid physical evidence per §11.4.123 + is registered into the §11.4.135 standing regression-guard suite; a newly-discovered fan-out defect triggers §11.4.4 test-interrupt + re-enters STEP 1. (ANTI-BLUFF, all steps) every PASS is rock-solid CAPTURED physical evidence per §11.4.123 (§11.4.5/§11.4.69/§11.4.107) — metadata-only / config-only / absence-of-error / grep-without-runtime PASS forbidden (§11.4/§11.4.1); unclear validation method ⇒ deep-research-before-declaring-untestable per §11.4.123/§11.4.8/§11.4.99; an operator-found defect the green suite missed triggers §11.4.138. Honest boundary (§11.4.6): STEP 3 reduces the unknown-unknown surface but does not prove zero remaining defects (§11.4.118) — un-exercised

cases stated as honest gaps, never silently implied clean. Composes §11.4.43/.115/.102/.130/.108/.139/.50/.85/.118/.135/.3/.123/.5/.69/.107/.138/.4/§107/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-146-PROPAGATION` (literal `11.4.146`) + recommended gate `CM-REPRODUCE-FIRST-THEN-EXTEND` (every closed defect's fix carries a §11.4.115 reproduce-first polarity test with captured defect-characterisation [STEP 1] + its `RED_MODE=0` GREEN confirmation [STEP 2] + an enumerated per-functionality extend case-set registered into the §11.4.135 suite [STEP 3]; a closure with the reproduce→confirm pair but NO enumerated extend case-set FAILs) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; close a defect with only the reproduce→confirm pair and no extend-case-set → `CM-REPRODUCE-FIRST-THEN-EXTEND` FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.146. Non-compliance is a release blocker. No escape hatch — no `--skip-reproduce-first`, `--fix-without-red`, `--skip-extend-to-all-cases`, `--reproduce-confirm-suffices`, `--defer-extend-to-release-sweep`, `--single-guard-suffices`, `--no-defect-characterisation` flag.

§11.4.147 — Crashed-agent respawn-until-complete + no-work-loss registry mandate (User mandate, 2026-06-10). Verbatim operator intent: "any agent that crashed because of something MUST BE respawned and finish its work at some point! We MUST NOT lose any work, forget about or have it corrupted!" Forensic case study (FACT): dispatched subagents died on TRANSIENT causes (`API Error: Server is temporarily limiting requests` rate-limit killed 5 at once, `API Error: socket connection closed unexpectedly` killed 2, one left PARTIAL edits — two source files written, the dependent SystemUI sliders + doc + test NOT done), each re-dispatched by pure conductor vigilance with NO mechanical guarantee — the moment conductor context is lost / compacted / the conductor itself crashes, an in-flight-but-dead work unit is silently dropped (lost / forgotten / corrupted). Every dispatched agent/subagent MUST be tracked through its full lifecycle so a crash NEVER loses, forgets, or corrupts its work; a crashed agent is NOT a completed agent (§11.4.6 — "it probably finished before dying" is a forbidden guess; abnormal termination is positive evidence the unit is OPEN). The mandate (ALL hold): **(a) durable agent REGISTRY** — every dispatched agent has an append-only machine-readable registry entry (stable id, task + §11.4.58 file-scope, declared output path(s) + §11.4.5/§11.4.69 evidence dir, dispatch

timestamp, status from the CLOSED SET `{dispatched | in-flight | crashed | respawned | complete}`), reusing the §11.4.116 sync substrate (JSONL event stream + atomic status snapshot; “an entry with no dispatch-event cannot show `complete`”); the registry is the SINGLE SOURCE OF TRUTH for “is this work owed?” and an unregistered dispatch is itself a violation; **(b) mechanical CRASH-DETECTION + RESPAWN-until-complete** — any abnormal termination (transient rate-limit/socket-close, any non-completion exit, or a terminal error after the runtime’s own retries are exhausted) flips the entry to `crashed` + keeps the unit OPEN, and the unit is respawned (fresh agent claims the same id + scope + preserved state) and re-respawned until an agent reaches `complete`; respawn is the safe/reversible/bounded decision taken autonomously per §11.4.101 (never blocks the loop for a human), transient causes use the project’s ALREADY-DEFINED backoff budgets (never invented, §11.4.6), a deterministically-reproducible non-transient terminal error is investigated per §11.4.102 before further respawn (never a blind retry loop); **(c) PARTIAL-STATE preserve → §11.4.84-check → resume-or-clean-restart** — the crashed agent’s uncommitted edits + output doc are PRESERVED (never silently discarded → lost, never blindly committed → corruption), the respawn runs the §11.4.84 quiescence check on the preserved tree (every modified file accounted-for vs scope, no mutation/ `// always pass` / `_mutated_*` residue, no half-written/torn artifact), then EITHER RESUMES idempotently when it passes OR CLEAN-RESTARTS from a known-good base (§9.2 pre-op backup, reversible §11.4.101) when inconsistent — nothing lost, nothing corrupted; per-agent `git worktree` isolation (§11.4.58 L4 / §11.4.84) keeps the partial tree from contaminating other streams; **(d) “crash ≠ done” COMPLETION criterion** — a unit is DONE only when an agent reaches `complete` with its required captured evidence/output landed (§11.4.5/§11.4.69 + the §11.4.116 verdict-carries-evidence-path rule); the endless-loop done-condition (§11.4.87/§11.4.94/§11.4.97/§11.4.126) MUST NOT read satisfied while any entry is `dispatched` / `in-flight` / `crashed` / `respawned` -not-yet- `complete`, the zero-idle survey (§11.4.94) treats every non-`complete` entry as an OPEN item to reclaim, and a registry showing `complete` without landed evidence is a §11.4 PASS-bluff at the agent-lifecycle layer. Honest boundary (§11.4.6): the registry + respawn guarantee work is not lost/corrupted, NOT that it is correct — the respawned output still crosses §11.4.108 + §11.4.125/§11.4.142 review + §11.4.40 retest; §11.4.147 is the durability layer beneath those, the agent-side analogue of §11.4.144 (same detect→wait/backoff→re-attach/

respawn→escalate shape). Composes §11.4.6/.58/.84/.87/.94/.97/.101/.116/.102/.108/.125/.142/.40/.128/.144/§9.2/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-147-PROPAGATION` (literal `11.4.147`) + recommended gate `CM-CRASHED-AGENT-RESPAWN-TRACKED` + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; mark a `crashed` entry as the loop's done-condition `complete` without landed evidence, OR drop a dispatched agent without a registry entry → `CM-CRASHED-AGENT-RESPAWN-TRACKED` FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.147. Non-compliance is a release blocker. No escape hatch — no `--skip-agent-registry`, `--crash-equals-done`, `--no-respawn`, `--discard-partial-state`, `--blind-commit-partial`, `--forget-dead-agent`, `--loop-done-with-crashed-entries` flag.

§11.4.148 — Workable-item integrity (status+type+id) + comprehensive structured description + bidirectional external-tracker sync + BLOCKED unblock-choices mandate (User mandate, 2026-06-10). BINDS + STRENGTHENS the workable-item discipline (§11.4.15 status / §11.4.16 type / §11.4.54 id / §11.4.21 Operator-blocked / §11.4.91 description-clarity / §11.4.93 + §11.4.95 SQLite SSoT / §11.4.106 docs_chain) into ONE integrity contract spanning DB ↔ docs ↔ external tracker, adding the operator's three emphases: **(D1) no item without a valid status + valid type + stable id — on ALL three surfaces** (an item missing any of the three in the DB, the rendered docs, OR the external tracker FAILs the validator — release-blocker; the id is the cross-surface binding key); **(D2) comprehensive structured description per item** — WHAT it is (§11.4.91 ≥6-word/≥40-char clear meaning) + HOW it manifests + HOW to reproduce (§11.4.115/.146) + ACCEPTANCE CRITERIA (§11.4.123/.69 captured-evidence verdict), in the §11.4.93 DB `description` column AND the docs AND the tracker, never a stub/§11.4.91-anti-pattern fragment; **(D3) BLOCKED items carry WHY + enumerated unblock CHOICES** — tightens §11.4.21: every `Operator-blocked` item (incl. `Blocked` / `BLOCKED` documented alias normalised to canonical `Operator-blocked`, never a silent fork §11.4.6) MUST enumerate the closed list of decisions/actions that would unblock it (`[A]…·[B]…·[C]…`, mirroring §11.4.66's 2–4-option shape); a BLOCKED item with no enumerated choices FAILs; **(D4) regular never-missed bidirectional DB↔docs↔tracker sync** — the §11.4.93/.95 git-tracked SQLite DB is the SSoT, docs + tracker DERIVED; full sync (`db-to-md` / `md-to-db` byte-identical round-trip §11.4.93 +

tracker push) runs regularly + on every change, §11.4.86 drift-proof fingerprint (sha256 of the sorted item keyset, NOT mtime) gates freshness, §11.4.106 docs_chain-bound so drift is mechanically caught not vigilance-dependent; **(D5) generic external-tracker push** carries statuses (collapsed onto the tracker's native set per §11.4.33/.112 when fixed, precise value preserved in a header, never lost), types, assignee (§11.4.104 participant handle, UNSET defaults from a project env var, never hardcoded/logged §11.4.10), and sub-tasks, IDEMPOTENT (dry-run-then-real, match-by-stable-key `[<ID>]` prefix / id custom-field, present⇒UPDATE/absent⇒CREATE, rate-limited, credential-redacted, sink-side `created=N updated=M failed=0` proof §11.4.69), the MACHINERY project-agnostic §11.4.28 (consumer registers its tracker/list/field-map at runtime). Anti-bluff §11.4: every sync + validator pass carries captured evidence §11.4.5/.69; honest boundary §11.4.6 — guarantees well-formed items + agreeing surfaces, NOT that the underlying work is correct (still crosses §11.4.108/.40/.123). Composes §11.4.15/.16/.21/.33/.34/.54/.66/.86/.91/.93/.95/.104/.106/.112/.123/.10/.28/.69/.6/§1.1. Classification: universal (§11.4.17) — consumer supplies its DB path, id prefix, tracker service + list/board id + field map + default-assignee env var, docs_chain context per §11.4.35. Propagation gate `CM-COVENANT-114-148-PROPAGATION` (literal `11.4.148`) + recommended gates `CM-ITEM-INTEGRITY-STATUS-TYPE-ID` / `CM-ITEM-COMPREHENSIVE-DESCRIPTION` / `CM-BLOCKED-UNBLOCK-CHOICES` / `CM-TRACKER-SYNC-IDEMPOTENT` + paired §1.1 mutation (gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.148. Non-compliance is a release blocker. No escape hatch — no `--item-without-status`, `--item-without-type`, `--item-without-id`, `--stub-description-OK`, `--blocked-without-choices`, `--skip-tracker-sync`, `--one-way-sync-OK`, `--non-idempotent-tracker-push`, `--hardcode-assignee` flag.

§11.4.149 — Per-workable-item testing-diary mandate (User mandate, 2026-06-10). Every workable item MUST carry an append-only TESTING DIARY — a chronological record of every test run against it — distinct from §11.4.93 `item_history` (lifecycle STATE transitions) + §11.4.55 Reopens (reopen cycles): the diary records TEST EXECUTIONS (which may or may not change status). The mandate (ALL hold): **(a)** a `test_diary` table additive to the §11.4.93/.95 SQLite SSoT, one row per run soft-keyed to the item id, capturing ISO-8601-UTC `date_time` + `tested_by` (closed set `User|Operator|AI-agent|HelixQA`) + `result` (§11.4.45 `PASS|FAIL|SKIP` + detail) + in-depth `observations` (long

markdown, facts or explicit `UNCONFIRMED:` / `PENDING_FORENSICS:` §11.4.6, the background a future fix needs) + `action_taken` + `status_changed` +from/to (did this run change status + WHY) + §11.4.69 `evidence_path` + `feature_class`, with a SCHEMA CONSTRAINT making a `PASS` row WITHOUT a non-empty evidence path impossible (a PASS-bluff rejected by the schema itself); **(b)** BOTH an in-depth diary doc + a derived at-a-glance summary VIEW (total/pass/fail/skip + last-verdict + last-run + status-changes + distinct testers/feature-classes, DERIVED never duplicated §11.4.93) feeding §11.4.132 risk-ordering; **(c)** four-format per-item `Diary` / `Diary_Summary` exports §11.4.65 + §11.4.86-drift-proof-fingerprinted (sha256 of the sorted diary keyset) + §11.4.106 docs_chain-bound so a diary row not exported/pushed FAILs a gate (never-missed); **(d)** external-tracker SUB-TASK model — the item task's description stays the item (§11.4.148 D2, NOT polluted with diary text), each run a CHILD sub-task (type `task`) with a `{TODO|In-progress|Completed}` lifecycle (collapsed per §11.4.33/.112), observations + action + an Evidence: line (path not raw artefact §11.4.10/.13) + a diary-entry idempotency key, reusing the §11.4.148 D5 idempotent rate-limited credential-redacted sink-side-proven push, mapper project-agnostic §11.4.28; **(e)** MINIMAL-LLM deterministic bash/Go tooling (zero LLM in the data path — the `observations` prose is authored by whoever ran the test; tooling only stores/renders/pushes/validates), 100% test-covered §11.4.27 (unit + integration-against-the-real-tracker with an honest §11.4.3 SKIP-when-token-absent never a faked PASS + export + HelixQA Challenge + paired §1.1 mutation) so a diary PASS-bluff is mechanically impossible. Honest boundary §11.4.6 — the diary guarantees a complete auditable per-item test-history, NOT that any single run's verdict is correct (rests on its own §11.4.69/.107/.123 evidence). Composes §11.4.6/.27/.45/.50/.55/.65/.69/.86/.93/.95/.106/.107/.123/.132/.148/.10/.13/.28/.33/.112/§1.1. Classification: universal (§11.4.17) — consumer supplies its DB path, diary doc layout, export formats, tracker sub-task field map, docs_chain context per §11.4.35. Propagation gate `CM-COVENANT-114-149-PROPAGATION` (literal `11.4.149`) + recommended gates `CM-TEST-DIARY-SYNC` / `CM-DIARY-PASS-REQUIRES-EVIDENCE` + paired §1.1 mutation (gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.149. Non-compliance is a release blocker. No escape hatch — no `--skip-testing-diary`, `--diary-without-evidence`, `--no-diary-summary`, `--diary-without-tracker-subtask`, `--llm-in-diary-data-path`, `--diary-export-optional` flag.

§11.4.150 — Mandatory deep multi-angle web research per change/issue, before declaring fixed or structural (User mandate, 2026-06-11). Verbatim operator mandate: “For every single issue we fix or improvement we make — besides tight systematic-debugging, fixing, code review by independent agents, comprehensive tests — we MUST ALWAYS do deep web research from various angles! No matter how big/small/simple/complex, dig deep the internet for articles, technical documentation, APIs, open-source code — EVERYTHING that can help make the best possible solution OR confirm we don’t have a more serious problem we’re unaware of! We MUST do everything possible to STOP the constant issue-reopening and finally start closing items as fixed+working! Do this ALWAYS in parallel with the main work stream!” For EVERY fix / improvement / change / closure — no matter how big, small, simple, or complex — the agent MUST, IN ADDITION to tight systematic-debugging (§11.4.102), the fix, independent-agent code review (§11.4.125 / §11.4.142), and comprehensive multi-layer tests (§11.4.4(b) / §11.4.40), perform a DOCUMENTED **deep multi-angle web research pass** digging the internet from VARIOUS angles — articles, official + vendor technical documentation, API references, standards, issue trackers, maintainer guidance, reusable open-source code — to BOTH (i) discover the best possible solution AND (ii) confirm there is NOT a more serious underlying problem the team is unaware of. Forensic case study (FACT, 2026-06-11): a subtitle-on-secondary-display goal verdicted `Won't-fix: structurally-impossible` (§11.4.112 secure-surface pixel-blanking) was OVERTURNED by a deep multi-angle research pass that surfaced the real OCR-of-the-PRIMARY-display path — a “we can’t” became a shipped capability; the canonical anti-pattern of a structural verdict reached for want of research. The mandate (ALL hold): **(A) No closure-as-fixed/structural WITHOUT a documented deep-research pass** — no item may be marked `Fixed` / `Implemented` / `Completed` (§11.4.33) OR classified `structurally-impossible` won’t-fix (§11.4.112) until a deep multi-angle pass is documented; §11.4.150 makes §11.4.8’s pass UNCONDITIONAL (every item, however trivial, at the closure AND structural-verdict gates). **(B) Multiple angles, not a single lookup** — ≥ 2 genuinely-distinct angles (official-docs / standards / known-bug-trackers / alternative-approach / failure-mode / security / performance / platform-constraint / open-source-precedent) so a single-source confirmation-bias miss (§11.4.145) cannot pass; a one-link drive-by is NOT deep multi-angle. **(C) Confirm-no-bigger-problem, not just find-a-fix** — explicitly seek evidence the fix does not mask a deeper defect AND the closure/verdict is genuinely safe; “we found nothing worse” requires the enumerated search (§11.4.118). **(D) Latest-**

source + cited — LATEST authoritative versions per §11.4.99 (never training-data/memory), each cited by URL + access date in the research artefact AND the closure commit footer (`Deep-research <date>: <urls>` OR the literal `NO external solution found — original work` per §11.4.8). **(E) Reopen-breaking is the PURPOSE** — STOP the constant Fixed↔Reopened churn (§11.4.34 / §11.4.55); a reopen whose root cause a deep pass would have surfaced is a §11.4.150 miss; composes §11.4.7 (demotion-evidence) + §11.4.112 (structural verdict now REQUIRES the cited-authorities pass). **(F) ALWAYS in parallel with the main stream** — background subagent-driven (§11.4.70 / §11.4.20 / §11.4.103 / §11.4.89) concurrent with main fix/build/test work, NEVER serialising it or stalling the loop (§11.4.94 / §11.4.97 / §11.4.101 / §11.4.126). **(G) Apply ASAP to every workable item** — retroactively + going forward across the §11.4.93 / §11.4.95 SSoT; items closed without a documented pass are re-audited in the §11.4.40 / §11.4.42 release-gate sweep. Honest boundary (§11.4.6): the pass reduces the unknown-unknown surface + breaks the most common reopen causes — it does NOT prove zero remaining defects (§11.4.118) and does NOT replace §11.4.108 runtime-signature verification, §11.4.125 / §11.4.142 review, or §11.4.40 retest; it is the research layer every fix/closure/structural-verdict additionally crosses, and “we probably don’t have a bigger problem” without the enumerated multi-angle search is a guess (§11.4.6), never a finding. Composes §11.4.8 / §11.4.99 / §11.4.123 / §11.4.118 / §11.4.145 / §11.4.125 / §11.4.142 / §11.4.7 / §11.4.112 / §11.4.34 / §11.4.55 / §11.4.70 / §11.4.20 / §11.4.89 / §11.4.103 / §11.4.40 / §11.4.42 / §11.4.93 / §11.4.95 / §11.4.6 / §1.1. Classification: universal (§11.4.17) — the consuming project supplies its concrete research corpora, angle set, item tracker, and closure-commit-footer convention per §11.4.35. Propagation gate `CM-COVENANT-114-150-PROPAGATION` (literal `11.4.150`) + recommended gate `CM-DEEP-RESEARCH-PER-ISSUE` (every closed/structural-verdicted item carries a documented multi-angle deep-research artefact + cited-source closure footer, run in parallel, before the closure/structural verdict is accepted) + paired §1.1 mutation (strip the literal → propagation gate FAILs; close an item or reach a `structural` verdict with no documented multi-angle research artefact / cited footer → `CM-DEEP-RESEARCH-PER-ISSUE` FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.150. Non-compliance is a release blocker. No escape hatch — no `--skip-deep-research` ,

`--single-source-suffices`, `--trivial-change-no-research`, `--close-without-research`, `--structural-without-research`, `--serialise-research`, `--research-from-memory-OK` flag.

§11.4.151 — Project-prefixed release-tag/version-naming mandate (User mandate, 2026-06-12). Verbatim operator mandate: “Every release tag and version name we create — on the main repository and on every Submodule we own — MUST be prefixed with the project’s release prefix, e.g. `myproject-1.0.0-dev-0.0.1`. The prefix MUST come from `HELIX_RELEASE_PREFIX` in our `.env` if it is set, otherwise from the lowercased project root directory name. The SAME prefix MUST be used across the main repo and all owned Submodules in one release so a release is greppable across every repository.” Every release tag AND every version name created on the main repository AND on every owned-by-us submodule (§11.4.28) MUST be prefixed with the project’s release prefix, form `<PREFIX>-<version>` (canonical example: `myproject-1.0.0-dev-0.0.1`), so a release is identifiable + greppable across every repository it spans (one `git tag -l '<PREFIX>-*'` enumerates the whole release surface). **Prefix resolution order (closed-set, deterministic — §11.4.6 no-guessing):** (1) `HELIX_RELEASE_PREFIX` from the project’s `.env` — authoritative when set; `.env` is git-ignored per §11.4.30 and the variable is documented in the tracked `.env.example` (a §11.4.77 re-obtain mechanism, never committed); (2) fallback = the lowercased snake_case form of the project root directory name (no spaces) per §11.4.29 — used whenever the env var is unset/empty, so a prefix is ALWAYS resolvable from the checkout with zero operator input. **The prefix MUST be IDENTICAL across the main repo and all owned submodules within a single release** — a release that tags the main repo `<PREFIX>-1.2.0` while tagging an owned submodule with a different/unprefixed value is a §11.4.151 violation (the cross-repo grep no longer enumerates the release). Version codes increment monotonically within the prefix (`<PREFIX>-…-0.0.1` → `<PREFIX>-…-0.0.2` → …), never reset, never skipped. Honest boundary (§11.4.6): the prefix guarantees a release is identifiable + uniform across every repository, NOT that its contents are correct — the tag is still created only after the §11.4.40 full-suite retest GREEN and reaches every upstream via the §11.4.113 merge-onto-latest-main path (NEVER a force-push), fanned out per §2.1. Composes §2.1 / §11.4.29 / §11.4.30 / §11.4.40 / §11.4.113 / §11.4.126 (the release-scope terminal condition is a published, prefixed tag). Classification: universal (§11.4.17) — the consuming project supplies its concrete prefix value + the `HELIX_RELEASE_PREFIX` env var per §11.4.35.

Propagation gate `CM-COVENANT-114-151-PROPAGATION` (literal `11.4.151`) + recommended gate `CM-RELEASE-PREFIX-NAMING` (every release tag/version on the main repo + every owned submodule carries the resolved `<PREFIX>-` prefix, identical across the release) + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; create an unprefixed or differing-prefix release tag → `CM-RELEASE-PREFIX-NAMING` FAILs; gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md` §11.4.151. Non-compliance is a release blocker. No escape hatch — no `--no-release-prefix`, `--unprefixed-tag`, `--prefix-optional`, `--differing-submodule-prefix` flag.

§11.4.152 — Crashlytics-recorded-data continuous monitoring + systematic-debug + regression-test-coverage mandate (User mandate, 2026-06-13). Verbatim operator intent: “For every project that has Firebase Crashlytics enabled / wired, we MUST continuously monitor ALL of the Crashlytics-recorded data — crashes, ANRs, performance traces, and non-fatals — systematically debug each, fix and improve, and cover everything with validation and verification tests. This MUST be checked regularly, with no false results and no bluff of any kind!” Every project that has Firebase Crashlytics enabled/wired (SDK linked into a shipping artifact, crash+non-fatal+ANR reporting active) MUST treat the Crashlytics console as a first-class captured-evidence channel from real end-user devices and continuously process every datum it records — an open Crashlytics issue with a green test suite is a §11.4 PASS-bluff at the field-telemetry layer (a real user hitting a broken feature while the suite reports green). STRENGTHENS+COMPLETES §11.4.47: §11.4.47 owns the periodic REVIEW + dedup + Issue-creation pass that SURFACES Crashlytics/Analytics/Performance findings; §11.4.152 owns what happens to each surfaced item AFTER — the systematic-debug → fix/improve → regression-test-coverage lifecycle that drives it to a proven regression-immune closure (§11.4.47 finds it; §11.4.152 fixes it + proves it stays fixed; both mandatory, neither substitutes). The mandate (ALL hold): (1) **continuous monitoring of ALL four surfaces** — fatal crashes, ANRs, performance traces/regressions, AND non-fatals (skipping any one is a PASS-bluff; non-fatals are the silent class — every catch/fallback/recovered-error path is a real degraded UX and MUST be triaged, not ignored because the app did not crash); (2) **systematic-debugging of each (reproduce-before-fix)** per §11.4.102 Iron Law + §11.4.115 RED-baseline-on-the-broken-artifact (the recorded stacktrace/ANR-trace/non-fatal-context IS the §11.4.5/.69 captured defect-present evidence) BEFORE the fix — a fix without a prior reproducing test is a §11.4.43/.123 violation; unreproducible ⇒ deep-research-

before-declaring-untestable §11.4.123/.150; (3) **a fix/improvement for every confirmed issue** against its proven root cause (§11.4.9/.43/.108), “acknowledged in console” is not a fix, muting a recurring issue without a tracked rationale is a §11.4.6/.90 violation; (4) **validation+verification regression-test coverage per closed issue** — in the SAME commit as the fix register a permanent §11.4.135 regression guard (§11.4.115 polarity test: `RED_MODE=1` captures the recorded defect on the pre-fix artifact, `RED_MODE=0` standing GREEN guard asserting ABSENT) exercising the same user-reachable path the stacktrace identifies, with rock-solid captured evidence §11.4.5/.69/.107/.123 + a closure log citing the console issue id/URL + root-cause + fix commit + the validation/verification test paths; a Crashlytics issue marked resolved WITHOUT a falsifiable regression test is FORBIDDEN (the silent-recurrence vector, §11.4.138 operator-escape class applied to field telemetry); (5) **regular cadence** reusing the §11.4.47 five-trigger set (pre-build/pre-flash/pre-distribute/pre-tag blocking; daily + post-deployment-burn-in non-blocking) — a one-time sweep never repeated is a violation; (6) **no false results, no bluff** (§11.4/.1/.6 — “no new issues” requires the enumerated monitored-surfaces+window §11.4.118; “fixed” requires the RED→GREEN flip §11.4.115/.130; a guard whose §1.1 mutation does not FAIL it is a bluff gate). Honest boundary §11.4.6: processing every recorded issue breaks the silent-recurrence vector — it does NOT prove zero remaining field defects nor replace §11.4.108/.40; an issue is “closed” only when its guard is GREEN on a clean artifact, never when the console mark is flipped. Classification: universal (§11.4.17) — the consuming project supplies its Crashlytics handle, console-access credential (§11.4.10, never logged), severity table, and per-issue closure-log path per §11.4.35; the reference project-level instantiation is a consuming project’s §6.O (Crashlytics-Resolved Issue Coverage — per-issue validation test + Challenge test + `.lava-ci-evidence/crashlytics-resolved/<date>-<slug>.md` closure log) + §6.AC (Comprehensive Non-Fatal Telemetry — every catch/fallback records a non-fatal with triage context). Composes §11.4/.1/.5/.6/.9/.34/.40/.43/.47/.69/.90/.102/.107/.108/.115/.118/.123/.130/.135/.138/.150/§1.1. Propagation gate `CM-COVENANT-114-152-PROPAGATION` (literal `11.4.152`) + recommended gate `CM-CRASHLYTICS-ISSUE-FULLY-COVERED` (every closed Crashlytics issue carries a systematic-debug root-cause record + a registered §11.4.135 regression guard + a closure-log entry citing the console issue id/URL + the validation/verification test paths; a console-resolved issue with no falsifiable regression guard FAILs) + paired §1.1 meta-test mutation (gate-code = separate work item). **Canonical authority:** constitution submodule `Constitution.md`

§11.4.152. Non-compliance is a release blocker. No escape hatch — no `--skip-crashlytics-monitoring`, `--monitor-crashes-only`, `--skip-non-fatals`, `--resolve-without-regression-test`, `--console-mark-is-fixed`, `--monitor-once`, `--mute-without-rationale` flag.

§11.4.153 — Comprehensive per-feature Status + Status_Summary document set with mandatory video-recording confirmation (User mandate, 2026-06-15). Every project MUST maintain, under `docs/features/`, a comprehensive feature Status document set (`Status.md` + its §11.4.56 `Status_Summary.md` companion) enumerating EVERY system component, EVERY client app/binary/surface (TUI / CLI / Web / desktop / mobile / API / gRPC / library / submodule / infrastructure), and EVERY feature — including features ported from any incorporated CLI-agent / submodule catalogue (§11.4.74) — with NO single feature left out. (1) **Total categorized coverage** — per-feature table organized Component→Category, reconciled against the actual codebase (a code-present feature missing from the table, or a row with no code, is a §11.4.6/§11.4.118 bluff). (2) **Per-feature fields** — Component / Feature / Category / Implementation / Wiring (genuinely end-user-reachable, not merely compiled, §11.4.108) / Real-use / Tests-coverage (four-layer §11.4.4(b)) / Validation (PASS / FAIL / SKIP / PENDING_FORENSICS / OPERATOR-BLOCKED per §11.4.45) / **Video-recording confirmation** (path to the real-use video or honest gap marker). (3) **Mandatory per-feature real-use video** — every user-visible “confirmed” claim backed by a recorded real-use video of a genuine end-user scenario (real prompts → real LLM/service responses → real results), NEVER a frozen/stale frame (§11.4.107), NEVER faked/mocked/demo-loop/bluff response or LLM error passed off as success (§11.4.2/§11.4.5), stored at the project-declared recording path (§11.4.35); a confirmed row with no real video or a bluff video = §11.4 PASS-bluff; autonomous-infeasible ⇒ honest §11.4.3/§11.4.52 SKIP + tracked migration item, NEVER a faked confirmation. (4) **Video-analysis remediation loop** — every video analysed; any defect it surfaces triggers §11.4.102 systematic-debug → fix → §11.4.146 retest → re-record → clean GO per §11.4.134 before “confirmed” (a video exposing a broken feature is a §11.4.4 test-interrupt). (5) **Always-in-sync** — §11.4.45-class roster/corpus-backed, §11.4.106 docs_chain-bound + §11.4.86 drift-proof fingerprint (sha256 of sorted feature-key roster AND sorted video-artefact roster, NOT mtime), re-syncs out-of-the-box; stale = violation. (6) **Four-format export** — HTML + PDF + **DOCX** (this doc class ADDS DOCX to the §11.4.65 HTML+PDF set; other classes unchanged), in sync per §11.4.60. (7) Follows §11.4.44/.45/.56/.57/.59/.60.

(8) **MP4 format REQUIRED.** All video confirmations MUST be in `.mp4` format (H.264). Window-specific capture ONLY (§11.4.159(A)). Vision validation REQUIRED (§11.4.159(D)). `.cast` files are supplementary only. Honest boundary §11.4.6 — guarantees a complete video-confirmed always-synced ledger, NOT per-feature correctness (rests on each row's §11.4.69/.107/.123 evidence) and NOT a §11.4.40 retest substitute. Classification: universal (§11.4.17). Composes §11.4.2/.5/.44/.45/.52/.56/.57/.59/.60/.65/.86/.102/.106/.107/.108/.118/.123/.134/.146/§1.1. Propagation gate `CM-COVENANT-114-153-PROPAGATION` (literal `11.4.153`) + recommended gates `CM-FEATURE-STATUS-COMPLETE` + `CM-FEATURE-STATUS-VIDEO-CONFIRMED` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.153. Non-compliance is a release blocker. No escape hatch — no `--skip-feature-status`, `--feature-without-video`, `--frozen-video-OK`, `--bluff-response-video-OK`, `--skip-video-analysis`, `--feature-ledger-incomplete-OK`, `--no-docx-export`, `--allow-feature-ledger-drift` flag.

§11.4.154 — Window-scoped capture + fresh-corpus rotation for feature/QA recordings (User mandate, 2026-06-15). Verbatim: "recording you perform does record the window containing apps and services, not the whole desktop or monitor screen!" + "All old recording files MUST BE removed when new one starts!" Refines §11.4.2/.5/.107/.153 recording discipline with two capture-hygiene invariants. **(A) Window-scoped, NOT whole-screen** — every feature/QA video MUST capture ONLY the window/surface of the app/service under test (GUI window / CLI-TUI terminal pane / web tab-viewport / device-emulator-simulator frame), NEVER the whole desktop/monitor or unrelated windows; whole-desktop capture leaks operator-private content (§11.4.10/.83), dilutes the §11.4.107 liveness/freeze oracle, and breaks the §11.4.137 OCR/ROI content oracle. Target the window/region by stable identity (window id/title, device serial, browser context, tmux target) per §11.4.111 — never a fixed full-screen device index. Platform genuinely cannot capture below whole-screen ⇒ honest §11.4.3 SKIP + tracked migration item, never a whole-screen pass-off. **(B) Fresh-corpus rotation** — when a new recording run for a scope begins, the agent's OWN prior in-scope stale recordings at the raw recording path MUST be removed FIRST so the live corpus reflects the current run (§11.4.107 not-stale + §11.4.86 roster-freshness). Honest boundary (§11.4.6 + §9.2): "remove old" = the agent's own prior recordings for the SAME scope/project ONLY — NEVER another project's/scope's/operator-authored files; uncertain ⇒ surface, don't delete (§11.4.122); committed `docs/qa/<run-id>/` evidence

(§11.4.83) is the durable record, NOT rotated. Classification: universal (§11.4.17). Composes §11.4.2/.5/.10/.83/.86/.107/.111/.122/.128/.137/.153/§9.2/.6. Propagation gate `CM-COVENANT-114-154-PROPAGATION` (literal `11.4.154`) + recommended gate `CM-WINDOW-SCOPED-FRESH-CORPUS-RECORDING` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.154. Non-compliance is a release blocker. No escape hatch — no `--whole-screen-capture-OK`, `--skip-window-scope`, `--keep-stale-recordings`, `--no-corpus-rotation`, `--full-desktop-recording` flag. **(C) MP4 auto-conversion REQUIRED.** Any `.cast` file produced MUST be auto-converted to `.mp4` via `agg` + `ffmpeg` immediately after capture. The `.mp4` is the primary evidence; `.cast` is supplementary only.

§11.4.155 — Project-name-prefixed feature/QA recording filenames (User mandate, 2026-06-15). Verbatim: “All recorded videos MUST START with prefix: the PROJECT NAME (ALWAYS USE THE PROJECT NAME). Project name MUST be obtained according to the constitution’s own project-name resolution.” Every recorded video the project produces — every feature/QA real-use recording (§11.4.153), every window-scoped capture (§11.4.154), every always-on device recording (§11.4.128), every raw or curated artefact at the project-declared recording path (§11.4.35) + the committed `docs/qa/<run-id>/` trail (§11.4.83) — MUST have a filename that STARTS WITH the PROJECT-NAME prefix, ALWAYS; an unprefixed recording is a §11.4.155 violation (a multi-project/scope corpus on one host per §11.4.128/.103 becomes un-greppable + un-attributable — the §11.4.151 identify-and-grep failure on the recording-corpus axis). **Prefix resolution (closed-set, deterministic — §11.4.6, IDENTICAL to §11.4.151):** (1) `HELIX_RELEASE_PREFIX` from `.env` (authoritative, git-ignored §11.4.30, documented in tracked `.env.example` §11.4.77) else (2) lowercased snake_case project-root dir name §11.4.29 — ALWAYS resolvable, zero operator input. SAME prefix for EVERY recording in a checkout so `ls '<PREFIX>---'*` enumerates the corpus; canonical form `<PREFIX>---<feature-or-scope>---<run-id>.<ext>` (`---` delimits the prefix unambiguously); MUST equal the §11.4.151-resolved release-tag prefix (divergence is itself a §11.4.155 violation — one project, one name). Honest boundary (§11.4.6): the prefix guarantees attribution + greppability, NOT content validity (still §11.4.107/.137/.153) and does NOT relax §11.4.154’s window-scope/rotation (rotation removes the agent’s OWN `<PREFIX>---*` only; foreign/operator files surfaced not deleted §11.4.122/§9.2). Classification: universal (§11.4.17). Composes §11.4.151/.128/.153/.154/.111/.83/.6/.29/.30/.35/.77/.86/§1.1.

Propagation gate `CM-COVENANT-114-155-PROPAGATION` (literal `11.4.155`) + recommended gate `CM-RECORDING-PROJECT-NAME-PREFIX` + paired §1.1 mutation.

Canonical authority: constitution submodule `Constitution.md` §11.4.155. Non-compliance is a release blocker. No escape hatch — no `--no-recording-prefix`, `--recording-without-project-name`, `--unprefixed-recording`, `--prefix-optional-for-recording`, `--differing-recording-prefix` flag.

§11.4.156 — All CI/CD automation (GitHub Actions / GitLab pipelines / equivalents) MUST be disabled (User mandate, 2026-06-15). Verbatim operator mandate: “Any GitHub actions or GitLab pipelines MUST BE disabled! Add this critical mandatory rule / mandatory constraint into the root constitution Submodule, commit and fetch all its changes to all upstreams and make sure we respect and follow this rule (we do apply it) ASAP!!!” Every repository this Constitution governs — main repo, this constitution submodule, every owned + nested submodule we author and push — MUST ship with ALL server-side CI/CD automation DISABLED: no push to any owned upstream may trigger a GitHub Actions run, GitLab pipeline, or equivalent (Jenkins/CircleCI/Travis/Drone/Woodpecker/Bitbucket/Azure, any `on: push` / `schedule` / `workflow_dispatch`). GENERALISES + makes ABSOLUTE the §11.4.75 Layer-5 posture (remote CI DISABLED, workflow preserved at a `…disabled-local-only` non-`.yml` name a provider ignores) across ALL governed repos; enforcement migrates to the LOCAL §11.4.75 git-hook ritual + §11.4.40 pre-tag sweep, never a remote runner. ALL hold: **(A)** zero active `.github/workflows/*.yml|yaml` / `.gitlab-ci.yml` / `.gitlab/**` / equivalent at the ROOT of any governed repo/submodule (the only place a provider executes); **(B)** “disabled” = a push triggers ZERO runs — delete OR rename to a non-trigger name (§11.4.75 `.disabled` / `.disabled-local-only`); a live-`on:` + `if:false` workflow still queues runs, NOT compliant; **(C)** scope = repos we author+push — vendored/third-party nested configs below the root (AOSP `external/**`, `prebuilts/**`, vendored submodules) are INERT (a provider never runs a non-root config), OUT of scope, MUST NOT be mass-edited (§11.4.29 vendor-exempt); test = “does a push to OUR upstream trigger a run?” yes⇒disable, inert⇒document+leave (§11.4.6 verify-not-assume); **(D)** no new CI may be added (release blocker); **(E)** pre-push verify `git ls-files | grep -E '^\.github/workflows/.*\.ya?ml$|^\.gitlab-ci\.yml$'` empty for authored repos, §11.4.109-class PreToolUse guard + gate enforce mechanically. Honest boundary (§11.4.6): file-level disabling stops FILE-triggered runs, NOT provider-side server

settings (org-default required workflows, branch-protection required checks, provider scheduled exports) — the operator turns those off; the agent documents what it cannot reach, never claims unachieved completeness. Composes §11.4.75/.29/.6/.40/.42/.109/.113/§2.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-156-PROPAGATION` (literal `11.4.156`) + recommended gate `CM-NO-ACTIVE-CI` + paired §1.1 meta-test mutation (strip the literal → propagation gate FAILs; add a root `.github/workflows/x.yml` to an authored repo → `CM-NO-ACTIVE-CI` FAILs). **Canonical authority:** constitution submodule `Constitution.md` §11.4.156. Non-compliance is a release blocker. No escape hatch — no `--allow-ci`, `--enable-workflow`, `--keep-pipeline`, `--remote-ci-OK`, `--ci-exempt` flag.

§11.4.157 — GEMINI.md maintained in lockstep with CLAUDE.md / AGENTS.md / QWEN.md (User mandate, 2026-06-15). Verbatim operator mandate: “Make sure with CLAUDE.md, AGENTS.md, QWEN.md we maintain GEMINI.md too! Add this mandatory fact / rule to root constitution Submodule we are inheriting / extending - CONSITUTION.md, CLAUDE.md, QWEN.md, AGENTS.md, GEMINI.md and other related relevant files!” Forensic FACT (2026-06-15): when §11.4.156/§11.4.157 were authored, Constitution/CLAUDE/AGENTS/QWEN carried the family through §11.4.155 but GEMINI.md had silently drifted to §11.4.141 — 14 mandates (§11.4.142–155) never propagated — a §11.4 propagation-bluff (a Gemini-CLI agent reads a stale Constitution). GEMINI.md is a FIRST-CLASS governance context carrier EQUAL to CLAUDE.md/AGENTS.md/QWEN.md, never optional/best-effort. ALL hold: **(A)** five-carrier lockstep — no governance change is complete until GEMINI.md carries it alongside the other three mirrors; GEMINI.md is added to the §11.4.26 propagation + cross-reference set explicitly; **(B)** no silent drift — GEMINI.md lagging the other mirrors’ highest rule is a §11.4.157 violation (§11.4.65-class), back-fill required; **(C)** equal status — GEMINI.md restates the SAME literal `11.4.N` anchors the propagation gates require (§11.4.35), fleet count INCLUDES GEMINI.md; **(D)** consumer projects’ own CLAUDE/AGENTS/QWEN/GEMINI bind too (§11.4.35). Honest boundary (§11.4.6): the §11.4.142–155 GEMINI.md back-fill is a tracked release-blocking remediation; claiming GEMINI.md “in sync” while the back-fill is incomplete is itself a §11.4.157 violation. Composes §11.4.26/.35/.17/.44/.65/.140/.156/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-157-PROPAGATION` (literal `11.4.157`, GEMINI.md INCLUDED) + recommended gate `CM-GEMINI-MD-LOCKSTEP` + paired §1.1 meta-test mutation. **Canonical authority:** constitution

submodule `Constitution.md` §11.4.157. Non-compliance is a release blocker. No escape hatch — no `--skip-gemini-md`, `--gemini-optional`, `--gemini-lag-OK`, `--four-carrier-suffices` flag.

§11.4.158 — Intensive all-feature/flow/edge-case video-recording + read-the-screen content-verification mandate (User mandate, 2026-06-16). Every project MUST be covered by intensive automated testing that exercises + RECORDS every feature/flow/use-case/edge-case (valid/invalid/boundary/concurrent/failure-injection §11.4.85), each recording showing the feature GENUINELY WORKING with REAL results (real prompts→real responses→real outputs §11.4.153) and NO false/simulated/stale/frozen result (§11.4.2/.5/.107) — a recording showing an error/bluff/non-working feature is a FINDING (→§11.4.153(4)/§11.4.4 fix→retest→re-record), never a confirmation. The testing System MUST ACTUALLY READ every shown log line / message / UI label / dialog / toast / status text and VERIFY it is a genuine working result (OCR/ROI §11.4.117/.137 + confidence floor for pixel surfaces; direct capture for terminal/log; §11.4.107(10) self-validated golden-good/golden-bad analyzer) — "a video was produced" is NOT evidence, "the System read the screen + confirmed a genuine result" is. HelixQA (§11.4.27) MUST drive this exercise→record→read→score pass, PASS only on a read-confirmed genuine result + captured artefact path (§11.4.69). Default recording save path = `$HOME/Downloads` (host user's home Downloads, resolved at runtime never hardcoded) unless a project declares an override per §11.4.35; §11.4.155 project-prefix + §11.4.154 window-scope/rotation + §11.4.128 git-ignored raw corpus + §11.4.83 curated docs/qa evidence apply. **Vision analysis MANDATORY for every recording** — after every recording, the agent MUST read the terminal output / video content and verify the feature actually works. A recording without a vision-confirmed verdict is a §11.4.158 violation. Composes §11.4.2/.5/.25/.27/.52/.69/.83/.85/.107/.108/.117/.118/.128/.137/.138/.153/.154/.155/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-158-PROPAGATION` (literal `11.4.158`) + recommended gates `CM-INTENSIVE-RECORDING-COVERAGE` + `CM-RECORDING-CONTENT-READ-VERIFIED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.158. Non-compliance is a release blocker. No escape hatch — no `--skip-recording-coverage`, `--video-without-content-read`, `--happy-path-recording-suffices`, `--recording-path-anywhere`, `--unread-recording-OK`, `--skip-helixqa-read` flag.

§11.4.159 — Mandatory window-specific video recording + vision validation mandate (User mandate, 2026-06-20). Every feature test, validation, verification, challenge, and QA session that produces video evidence MUST comply with ALL of the following: **(A) Window-specific recording ONLY** — every video MUST record ONLY the target application window (Terminal pane, TUI app, browser tab, emulator frame), NEVER the whole desktop/monitor screen; use macOS `screencapture -l<window_id>`, Linux `xdotool` + `ffmpeg`, or equivalent; whole-desktop capture leaks operator-private content (§11.4.10), dilutes the §11.4.107 liveness oracle, and breaks §11.4.137 OCR/ROI; platform genuinely cannot capture below whole-screen => honest §11.4.3 SKIP + tracked migration item. **(B) MP4 format REQUIRED** — `.mp4` (H.264, `movflags +faststart`, `pix_fmt yuv420p`); `.cast` files are supplementary only; auto-convert via `agg` + `ffmpeg` per §11.4.154(C). **(C) Project-name prefix REQUIRED** — filename MUST start with project name in snake_case from `HELIX_RELEASE_PREFIX` in `.env` (§11.4.151) or lowercased project root dir name (§11.4.29); format `<project_name>-<feature>-<timestamp>.mp4`; unprefixed = violation. **(D) Mandatory vision validation** — after EVERY recording, the agent MUST read the terminal output / video content and verify: (i) feature ACTUALLY WORKS, (ii) LLM responses are REAL, (iii) all tests show PASS, (iv) no “TODO implement” / “simulate” / “for now” patterns, (v) output demonstrates end-user working feature; MUST produce a verdict (PASS/FAIL) with evidence path (§11.4.69). **(E) Terminal window cleanup** — after each recording, dismiss/close ONLY the Terminal window used for that recording (window-specific close via `osascript` / `xdotool`); MUST NOT close windows belonging to other processes. **(F) Real results ONLY** — every recording MUST show REAL working features; errors/empty output/simulated responses trigger fix→retest→re-record. **(G) Re-runnable evidence** — the command shown MUST be re-runnable to produce the same results. **(H) Fresh-corpus rotation** — remove agent’s own prior in-scope stale recordings FIRST (§11.4.154). **(I) Content verification MANDATORY — not duration-based** — the value of a recording is NOT its duration but its CONTENT; a recording MUST demonstrate the ACTUAL feature being used with REAL results; before accepting ANY recording, the agent MUST verify: expected output patterns ARE present (e.g., test PASS lines, API response data, feature-specific output), the feature ACTUALLY WORKS as demonstrated (not just “something ran”), LLM responses are REAL content (not simulated, not placeholder, not empty), every claim of “working” is backed by visible evidence; a 5-second recording showing a feature working correctly is MORE valuable than a 60-second recording of empty terminal; duration is NOT a proxy for

quality. **(J) Expected-content specification REQUIRED** — before recording, the agent MUST specify what content SHOULD appear (expected patterns, expected test results, expected API responses); after recording, the agent MUST verify these patterns ARE present; if expected content is MISSING, the recording is REJECTED regardless of duration. **(K) Content-verification recording workflow** — mandated workflow: (1) SPECIFY expected content patterns, (2) RECORD the feature execution, (3) EXTRACT all text from the recording, (4) VERIFY expected patterns are present, (5) CHECK for simulated/placeholder content, (6) ACCEPT only if ALL patterns found AND zero bluffs detected, (7) REJECT and re-record if ANY pattern missing or bluff detected. **(L) Root cause analysis REQUIRED for rejected recordings** — when a recording is rejected (missing expected content, bluff detected, or empty capture), the agent MUST investigate WHY before re-recording per §11.4.102; determine the root cause (timing issue, wrong command, tool failure, etc.) and fix it; simply re-recording without understanding WHY the first attempt failed is a §11.4.159 violation. **(M) Real-time monitoring RECOMMENDED** — for complex features, use real-time monitoring that analyzes output DURING recording (not after); this catches issues immediately and allows corrective action before the recording completes. Classification: universal (§11.4.17). Composes §11.4.2/.3/.5/.10/.29/.69/.83/.107/.111/.128/.137/.151/.153/.154/.155/.158/§1.1. Propagation gate `CM-COVENANT-114-159-PROPAGATION` (literal `11.4.159`) + recommended gate `CM-WINDOW-VIDEO-VALIDATED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.159. Non-compliance is a release blocker. No escape hatch — no `--whole-screen-ok`, `--cast-only`, `--skip-vision-validation`, `--no-cleanup`, `--simulated-recording-ok`, `--unprefixed-recording` flag.

§11.4.160 — Vision-verified recording + HelixQA bridge mandate (User mandate, 2026-06-21). Compact summary: every video recording for feature/QA evidence MUST be processed through a vision/OCR pipeline that reads on-screen content and confirms expected results BEFORE acceptance; the recording system MUST provide a bridge feeding captured frames to HelixQA's test infrastructure (or equivalent) for automated read-the-screen verification against SPECIFY-phase expected patterns (§11.4.159(J)); capture frames at ≤5s intervals, run OCR/vision with self-validated analyzer (§11.4.107(10)), compare text against patterns, produce per-frame PASS/FAIL with evidence path, surface failures immediately for re-record (§11.4.159(L)). Project-configured parameters per §11.4.35. Honest boundary: does NOT replace §11.4.5 quality analysis nor §11.4.108 runtime-signature;

FLAG_SECURE uses §11.4.117 proxy oracle. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-160-PROPAGATION` + recommended gate `CM-VISION-VERIFIED-RECORDING-BRIDGE` + paired §1.1 mutation.

§11.4.161 — Rootless container runtime mandate (User mandate, 2026-06-21). Compact summary: every project MUST use Podman rootless (or equivalent) for ALL containerized workloads — rootful Docker/sudo FORBIDDEN unless §11.4.112 platform constraint documented; `vasic-digital/containers` Submodule (§11.4.76) sole orchestration layer via `pkg/boot` / `pkg/compose` / `pkg/health`; extend upstream per §11.4.74 for missing capabilities; integration tests boot infra on-demand. Honest boundary: does NOT replace §11.4.10 credentials nor §12.3 hygiene. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-161-PROPAGATION` + recommended gate `CM-ROOTLESS-CONTAINER-RUNTIME` + paired §1.1 mutation.

§11.4.162 — OpenDesign UI design system mandate (User mandate, 2026-06-21). Compact summary: every project with user-facing interfaces MUST use OpenDesign (https://github.com/nexu-io/open-design) as mandatory UI design system — NOT ad-hoc CSS; install as dependency, use token system for all color (light+dark from brand assets), typography, spacing, component tokens; extend upstream (§11.4.74) for unsupported patterns; every UI component ships light+dark + token-diff regression tests via visual regression; no element overlap/font collision/ label overlay. Honest boundary: does NOT replace §11.4.27 functional testing nor §11.4.48/.49 UI testing. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-162-PROPAGATION` + recommended gate `CM-OPENDESIGN-UI-SYSTEM` + paired §1.1 mutation.

§11.4.163 — Universal Media Validation (User mandate). Every recorded artifact MUST pass a MEDIA VALIDATION pipeline — OCR for video/screenshots, transcription for audio, text parsing; compare against SPECIFY-phase patterns (§11.4.159(J)); self-validated analyzer (§11.4.107(10)); structured verdict with pinpoint on FAIL; post-recording and real-time triggers. Paired §1.1 mutation ensures golden-bad fixture FAILs. Classification: universal. Propagation gate `CM-COVENANT-114-163-PROPAGATION` + paired §1.1 mutation.

§11.4.164 — Universal Auto-Propagation Hook (User mandate). Every constitutional pull triggers `post_update_hook.sh` — detects changed files, registers skills/MCP/hooks/scripts, logs summary. Consumer MUST invoke after every pull. Classification: universal. Propagation gate `CM-COVENANT-114-164-PROPAGATION` + paired §1.1 mutation.

§11.4.165 — Universal Independent Verification Agent (User mandate). Every code/media/docs/config output MUST pass INDEPENDENT verifier (§11.4.70/.20), iterate to GO per §11.4.134. CODE: review+build+mutations+runtime-signature. MEDIA: pipeline+genuine+format. DOCS: exports+revision. CONFIG: syntax+schema+leaks. Self-validated by §1.1 mutation. Classification: universal. Propagation gate `CM-COVENANT-114-165-PROPAGATION` + paired §1.1 mutation.

§11.4.166 — REPEALED (operator decision, 2026-06-22). The Universal Semgrep static-analysis mandate is repealed — Semgrep is NO LONGER mandatory. Static analysis remains encouraged, not mandated. Scaffolding, `submodules/semgrep`, MCP/pre-commit/PATH wiring, docs_chain `semgrep_status` context, and the `CM-COVENANT-114-166-PROPAGATION` / `CM-SEMGREP-WIRED` gates removed. Anchor 11.4.166 retired. See constitution `Constitution.md` §11.4.166.

§11.4.167 — Big-work-item feature work-stream lifecycle (User mandate, 2026-06-23). Every BIG work item (new feature OR drastic/large fix) MUST be its own isolated feature work-stream — own sibling project copy, own `feature/<slug>` branch, own per-feature flashable builds + feature tags, SEPARATE from trunk until operator manually approves after testing, trunk merged INTO every stream regularly. (A) one big item = one sibling project copy (`<project>_<slug>`); small changes stay on trunk. (B) disk-feasible CoW/reflink clone MANDATORY (`cp -a --reflink=auto` on btrfs/XFS/ZFS/APFS, else `git worktree`); naive deep copies FORBIDDEN; per-stream `out/` + shared ccache; ~0-disk verified (§11.4.69) before relied on (§11.4.6). (C) own branch + greppable tags (§11.4.151 `<base>-feat-<slug>`), NEVER merged to trunk until §11.4.40 GREEN on clean target → §11.4.41 merge-first → §11.4.113 ff-only. (D) trunk merged INTO every live stream per trunk-tag/daily (merge, never rebase). (E) feature branch + tag cascades to EVERY touched submodule (§11.4.113). (F) single-builder build QUEUE (§11.4.58/.12.7/.12.8) + finite-device queue (§11.4.119); idle `out/` GC'd (§11.4.77). (G) every change crosses full gauntlet (§11.4.142/.125/.134/.145 + anti-bluff). (H) stream registry (§11.4.116/.147) + atomic `.partial` →rename +

§11.4.84 resume + §9.2 retire-backup; bounded §12/.12.6. Honest boundary (§11.4.6): isolation+disk-feasibility+greppable-identity+merge-discipline, NOT correctness (still §11.4.108/.40). Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-167-PROPAGATION` (literal `11.4.167`) + gates `CM-FEATURE-WORKSTREAM-COW-CLONE` / `-NO-MERGE-UNTIL-APPROVED` / `-TRUNK-SYNC-CADENCE` / `-SUBMODULE-CASCADE` + paired §1.1 mutations. **Canonical authority:** constitution `Constitution.md` §11.4.167.

§11.4.168 — Exported-document independent content + textual + full-visual validation mandate (User mandate, 2026-06-23). Every generated/exported document (HTML/PDF/DOCX/any §11.4.65 export-scope format) MUST pass INDEPENDENT validation by a reviewer structurally separate from the generator (§11.4.70/.142, NEVER author self-check), on BOTH source AND every exported artifact, ALWAYS, across THREE layers: (1) CONTENT — export faithfully carries source intent+data, nothing dropped/truncated/garbled; (2) TEXTUAL — human-readable, NO raw markup/diagram-source (Mermaid `gantt` / `graph` / `flowchart` / `sequenceDiagram` etc.)/unrendered code-fence leaking as body text (the exact 2026-06-23 raw-gantt-in-PDF failure); (3) FULL VISUAL — diagrams render as IMAGES not source, layout intact, no overlap/cut-off/garble, verified by RENDERING the export (`pdftotext` catches raw source + `pdfimages` confirms rendered images + `pdftoppm` →OCR confirms human-readable content) with captured evidence §11.4.5/.69/.107. Anti-bluff: rubber-stamp ≠ validation; analyzer self-validated golden-good/golden-bad §11.4.107(10) (golden-bad seeded with raw `gantt` MUST FAIL); iterate to clean GO §11.4.134. Forensic FACT (2026-06-23): a “green” Mermaid fix shipped raw gantt source as readable text into user PDFs because validation grepped HTML for `class="mermaid"` and NEVER checked the exported PDF — operator caught it (§11.4.138). Honest boundary (§11.4.6): confirms the exported artifact is faithful/readable/visually-rendered — does NOT prove source content is correct (§11.4.142/.135) nor replace the §11.4.65 mtime freshness gate (this is the READABILITY/FIDELITY layer the mtime gate cannot see); FLAG_SECURE/un-rasterisable surfaces document the gap §11.4.112 + §11.4.117 proxy, never a faked PASS. Classification: universal (§11.4.17). Composes §11.4.65/.73/.107/.117/.135/.138/.142/.159/.163/.165/§1.1. Propagation gate `CM-COVENANT-114-168-PROPAGATION` (literal `11.4.168`) + recommended gate `CM-EXPORTED-DOC-VISUALLY-VALIDATED` + paired §1.1 mutation. **Canonical authority:** constitution `Constitution.md` §11.4.168.

§11.4.169 — Mandatory comprehensive test-type coverage with anti-bluff captured evidence (User mandate, 2026-06-25). Every project MUST cover a CLOSED enumerated test-type set, each to as-close-to-100% as the domain permits, every PASS citing rock-solid captured PHYSICAL evidence (§11.4.5/.69/.107), zero false results/zero bluff (§11.4/.1/.6): (1) unit (only layer mocks allowed §11.4.27(A)); (2) integration (real System, infra via containers submodule §11.4.76); (3) e2e; (4) full-automation (autonomous, re-runnable, no manual step §11.4.25/.52/.98, deterministic §11.4.50); (5) Challenges (challenges submodule banks §11.4.27(B)); (6) HelixQA (helix_qa submodule — written test banks/suites per app/service/platform + comprehensive fully-autonomous QA sessions §11.4.27); (7) DDoS/load-flood (clean refuse or graceful degrade); (8) security (authz/injection/secret-leak §11.4.10/crypto/CVE/default-deny + security submodule); (9) stress+chaos (load+contention+boundary+failure-injection, categorised recovery §11.4.85); (10) concurrency/atomicity (no lost updates, transactional atomicity, idempotent replay); (11) race-condition/deadlock (race detector + lock-order analysis; no blocking op in shared-lock region §1); (12) memory (leak census + peak-RSS ceiling + 24h/N-iter soak, no unbounded growth); (13) benchmarking/performance (p50/p95/p99 + throughput vs recorded baseline). The ONLY permitted absence is an honest §11.4.3 SKIP-with-reason, never a silent gap; four-layer §11.4.4(b) enforcement + coverage ledger (§11.4.25/.52); missing required type or OPERATOR_ATTENDED_ONLY = release blocker. Strict enumerated expansion of §11.4.27. Composes §11.4.4/.5/.6/.25/.27/.50/.52/.69/.76/.85/.98/.107/.123/.135/.146/§1.1. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-169-PROPAGATION` (literal `11.4.169`) + recommended gate `CM-MANDATORY-TEST-TYPES-COVERED` + paired §1.1 mutation. **Canonical authority:** constitution `Constitution.md` §11.4.169.

§11.4.170 — Device-independent host-side rendered-UI visual-proof mandate (User mandate, 2026-06-25). Compact summary: every change to ANY user-facing UI surface (screen/component/view/widget/layout/theme on any platform — Android-Compose, iOS-SwiftUI/UIKit, web, desktop, TUI) MUST be proven by DEVICE-INDEPENDENT host-side RENDERED PIXELS before it may be claimed correct — the real component rendered to a PNG ON THE HOST (NO device, NO emulator, NO running app) via a host-render harness (Compose→Roborazzi/Paparazzi on the JVM; web→Playwright/Storybook snapshot; SwiftUI→snapshot-testing; equivalent per stack), for EVERY relevant screen×state×{light,dark} theme,

with the PNG validated by BOTH (i) golden image-diff (per-pixel/perceptual PASS/FAIL) AND (ii) an OCR/vision oracle reading rendered text+labels+control bounds to assert legibility + NO overlap / NO label-over-label / NO clipping / NO off-screen control / NO collapsed-or-giant-unbounded widget (the §11.4.162 layout invariants PROVEN on real pixels). Forensic FACT (2026-06-25): UI work was “validated” by VALUE-EQUALITY unit tests (a hex colour string / an sp/dp size integer / a design-token field == a constant) that NEVER RENDERED A PIXEL — GREEN while the operator opened a broken unbounded giant-button screen. A value/token-equality / property-assertion UI test is FORBIDDEN as the PROOF a UI is correct (it may SUPPLEMENT, NEVER SUBSTITUTE the rendered-pixel proof); a “green” UI suite with no rendered-pixel artefact for the changed surface is a §11.4 PASS-bluff at the visual layer. Self-validated golden-good/golden-bad analyzer §11.4.107(10), thresholds calibrated on the project’s own fixtures §11.4.6. “device offline” is NEVER a valid skip — host-render IS the device-independent path; honest §11.4.3 SKIP only where the platform genuinely has no host-render harness (tracked migration item, never a value-only fallback). Honest boundary §11.4.6: host-render proves the COMPONENT renders correctly device-independently per commit — it does NOT prove the running app on real hardware (that remains §11.4.153/.158/.159/.160’s live-device recording + read-the-screen layer, which §11.4.170 COMPLEMENTS not replaces — both mandatory), does NOT replace §11.4.162 OpenDesign token/theme governance (§11.4.170 PROVES the rendered RESULT of those tokens), DISTINCT from §11.4.168 exported-document visual validation. Classification: universal (§11.4.17). Composes §11.4.3/.5/.27/.40/.69/.107/.108/.117/.153/.158/.159/.160/.162/.163/.168/.169/§1.1. Propagation gate `CM-COVENANT-114-170-PROPAGATION` (literal `11.4.170`) + recommended gate `CM-HOST-RENDERED-UI-VISUAL-PROOF` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md` §11.4.170. Non-compliance is a release blocker. No escape hatch — no `--value-assertion-proves-ui` , `--skip-rendered-pixel-proof` , `--token-equality-suffices` , `--device-offline-skip-visual` , `--single-theme-only` , `--unvalidated-image-diff-OK` , `--no-ocr-layout-oracle` flag.

Companion documents

§11.4.171 — Mandatory comprehensive human-readable workable-item descriptions (User mandate, 2026-06-29). Every workable item (ATM-NNN ticket) in the project's workable-items database, Issues.md, Fixed.md, Issues_Summary.md, Fixed_Summary.md, and ALL derived documents MUST carry a comprehensive human-readable description that explains the item in plain, non-technical language suitable for non-developers (project managers, stakeholders, team leads, business partners). The description MUST be 5-7 sentences minimum, written so that ANY person — regardless of technical background — can understand: (a) WHAT the item is (the feature, fix, or task in simple terms); (b) WHY it matters (the user/business value); (c) HOW it works (the mechanism, without code references); (d) WHO benefits (the end user or stakeholder); (e) WHAT the expected outcome is (the measurable result). The description is MANDATORY — an item without a human-readable description is a §11.4 violation at the documentation layer. Descriptions MUST NOT use jargon, acronyms without expansion, code references, or technical implementation details as the primary explanation. Technical details MAY appear as supplementary context AFTER the plain-language explanation. The `description` column in the SQLite database (`docs/workable_items.db`) is the SINGLE SOURCE OF TRUTH for item descriptions; all derived documents (Issues.md, Fixed.md, summaries, exports) MUST carry the SAME description text. Pre-build gate `CM-HUMAN-READABLE-DESCRIPTION` (when implemented) walks every item in the DB and asserts the `description` field exists, is non-empty, and is ≥ 50 characters. Paired §1.1 meta-test mutation strips a description → gate FAILs. Classification: universal (§11.4.17). Composes §11.4.15 (item-status tracking), §11.4.16 (item-type tracking), §11.4.19 (column alignment), §11.4.91 (summary clarity), §11.4.93 (SQLite SSoT), §11.4.148 (item integrity), §11.4.65 (universal export). Propagation gate `CM-COVENANT-114-171-PROPAGATION` (literal `11.4.171`) + paired §1.1 mutation.

§11.4.172 — Mandatory production-readiness planning with realistic timeline projection (User mandate, 2026-06-29). Every project under this Constitution MUST maintain a living production-readiness planning document that: (a) provides REALISTIC timeline projections for all product milestones (MVP, beta, production-ready) based on actual development velocity data (items completed per week, reopening rate, blocking dependencies); (b) identifies ALL danger zones,

weaknesses, and risk areas with mitigation strategies; (c) accounts for HW delivery timelines, licensing delays, and external dependency risks; (d) defines the critical path and identifies which items can slip without affecting MVP; (e) includes workstation/build-capacity analysis and optimization recommendations; (f) is updated at least monthly or whenever the item count changes by ≥ 10%; (g) is cross-referenced against the workable-items database for accuracy. The planning document MUST live at `docs/research/production_planning_<YYYYMMDD>/ANALYSIS.md` with HTML+PDF exports per §11.4.65. Honest boundary (§11.4.6): projections are estimates based on measured velocity — they guarantee the methodology is sound, NOT that specific dates will be met. Classification: universal (§11.4.17). Composes §11.4.6 (no-guessing — projections based on data, not wishful thinking), §11.4.40 (full retest before tag), §11.4.42 (iteration discipline), §11.4.93 (SQLite SSoT), §11.4.108 (four-layer verification). Propagation gate `CM-COVENANT-114-172-PROPAGATION` (literal `11.4.172`) + paired §1.1 mutation.

§11.4.173 — Shared-host process-ownership verification mandate (User mandate, 2026-06-29). On any shared host where other projects/users/agents may run concurrently, every inspection of or action on a process, build, daemon, port, lock, or system-state signal MUST first positively VERIFY the target is OURS before diagnosing or acting. Attribute a process to us ONLY by a strong discriminator — cwd inside this project's tree, argv naming our scripts/artifacts/repo path, a PID we launched + recorded (§11.4.147), or a lock/port path under our tree — NEVER a loose `pgrep java/gradle/node` name match (it matches every project on the host). NEVER kill/signal/restart a process not positively ours (other projects' gradle/java/podman/zig are off-limits — operator-workload-safety, §12/§11.4.133); the §12.8 daemon-kill remediation MUST be scoped to OUR daemons. When our work waits on host contention from another project's process, surface it (§11.4.66/§11.4.101) — block-don't-break. Every pgrep/ps filter MUST exclude self-matches + other-project matches. Forensic anchor (FACT 2026-06-29): a 200%-CPU java+gradlew on the shared host was a separate `catalogizer` project's gradle test (cwd=Projects/catalogizer), nearly mis-attributed to our build. Composes §11.4.6/§11.4.66/§11.4.101/§11.4.147/§12/§12.8/§11.4.133. Classification: universal (§11.4.17). Propagation gate `CM-COVENANT-114-173-PROPAGATION` (literal `11.4.173`) + recommended gate `CM-PROCESS-OWNERSHIP-VERIFIED` + paired §1.1 mutation. **Canonical authority:** constitution submodule `Constitution.md`

§11.4.173. Non-compliance is a release blocker. No escape hatch — no `--assume-process-ours`, `--loose-pgrep-ok`, `--kill-any-matching-process`, `--skip-ownership-check` flag.